FIT TRACK APP

Session : BSCS Spring 2020-2024

Project Advisor: Abdullah Sahi

Submitted By

Hira Nawaz *20BSCS29644*

Department of Computer Science & Information Technology
University of Sargodha

Statement of Submission

This is to certify that this final year project entitled Design and Implementation of **a Fit Track App**, which is submitted **by Hira Nawaz** (**20BSCS29644)** in partial fulfilment of the requirement for the award of degree for BS in Computer Science to the Department of Computer Science, University of Sargodha, comprises of only my original work and due acknowledgement has been made in the text to all other materials used.

Date: ----------

Project supervisor Sir Abdullah Sahi	Project Coordination Office DOCS&IT -UOS
External Examiner	Chairman DOCS&IT -UOS

Acknowledgement

I am grateful to **ALLAH** Almighty who gave us this opportunity to learn, investigate, adventure things critically, and evaluate concepts and ideas. I would like to express gratitude to my advisor **Prof. Irfan Ullah**. I value the mentoring, and guidance you have provided. Also, very thankful for answering queries and correcting me. Finally, I would like to thank Zamindar College, who provided us our learning platform and healthy environment for fulfilling our dreams.

Abstract

The Fit Track is an app and solution for people get fit and healthy without being worried about time and specific location. It lets users track their steps, set fitness goals, perform exercises based on their age, water reminder, GPS run tracker and Nutrition suggestions to stay fit and healthy. The app is designed to be easy to use and helps people take care of both their body and mind. The aim of this app is to provide simplicity and all features related to fitness /health in just single app using language xml for frontend and java for backend, and tool will be Android Studio and stores information on Firebase and SQLite database.

Table of Contents

CHAPTER 1: FINAL PROJECT PROPOSAL 12
1.1 INTRODUCTION 12
1.2. PROJECT TITLE: 12
1.3. PROJECT OVERVIEW STATEMENT: 12
1.4. PROJECT GOALS & OBJECTIVES: 13
1.5. HIGH-LEVEL SYSTEM COMPONENTS: 13
1.6. LIST OF OPTIONAL FUNCTIONAL UNITS: 14
1.7. EXCLUSIONS: 14
1.8. APPLICATION ARCHITECTURE: 15
1.9. GANTT CHART: 16
1.10. HARDWARE AND SOFTWARE SPECIFICATION: 17
1.11. TOOLS AND TECHNOLOGIES USED WITH REASONING: 18
1.12. APP PERMISSIONS: 20
CHAPTER 2: FIRST DELIVERABLE 21
2.1. INTRODUCTION 21
2.2. PROJECT/PRODUCT FEASIBILITY REPORT 22
2.2.1. Technical Feasibility 22
2.2.2. Operational Feasibility 22
2.2.3. Economic Feasibility 22
2.2.4. Schedule Feasibility 22
2.2.5. Specification Feasibility 22
2.2.6. Information Feasibility 22
2.2.7. Motivational Feasibility 23
2.2.8. Legal & Ethical Feasibility 23
2.4. PROJECT/PRODUCT SCOPE 23
2.5. PROJECT/PRODUCT COSTING 25
2.3.1. Project Cost Estimation By Function Point Analysis 26
2.3.2. Project Cost Estimation by using COCOMO'81 (Constructive Cost Model) 27
2.3.3. Activity Based Costing 28
2.5. TASK DEPENDENCY TABLE 29
2.6. CPM - CRITICAL PATH METHOD 30
2.7. INTRODUCTION TO TEAM MEMBER AND THEIR SKILL SET 34
2.8. VISION DOCUMENT 34
2.9. RISK LIST 34
2.10. PRODUCT FEATURES/ PRODUCT DECOMPOSITION 35
CHAPTER 3: SECOND DELIVERABLE FOR OBJECT ORIENTED APPROACH
3.1 INTRODUCTION: 36
3.1.1 Systems Specifications 37
3.1.2. Identifying External Entities 38
3.1.3. Context Level Data Flow Diagram: 39

3.1.4. Capture "shall" Statements: *40*
3.1.5. Functional Requirements: *41*
3.1.6. Non-Functional Requirements: *45*
3.1.7. Requirements Trace-ability Matrix: *47*
3.1.8. High Level Usecase Diagram: *49*
3.1.9. Analysis Level Usecase Diagram: *50*
3.1.10. Usecase Description: *51*
CHAPTER 4: DESIGN DOCUMENT (FOR OBJECT ORIENTED APPROACH) **63**
4.1 INTRODUCTION: 63
4.2 DOMAIN MODEL: 63
4.3 SYSTEM SEQUENCE DIAGRAM: 64
4.4. SEQUENCE DIAGRAM: 66
4.5 COLLABORATION DIAGRAM: 69
4.5.1. Contents of Collaboration Diagrams: *69*
4.5.2. Constructs of Collaboration Diagram: *70*
4.6 OPERATIONAL CONTRACTS: 72
4.7. DESIGN CLASS DIAGRAM: 74
4.8 STATE CHART DIAGRAM: 75
4.9. DATA MODEL: 76
CHAPTER 5: 2ND & 3RD DELIVERABLE FOR STRUCTURED APPROACH **82**
5.1. INTRODUCTION: 82
5.2. ARCHITECTURAL DESIGN: 82
5.3. COMPONENT LEVEL DESIGN : 83
CHAPTER 6: 4TH DELIVERABLE (USER INTERFACE DESIGN) **93**
6.1. INTRODUCTION: 93
6.2. SITE MAPS: 93
6.3 STORY BOARDS: 94
6.4. NAVIGATIONAL MAPS: 103
6.5.GUI TABLES 103
CHAPTER 7: 5TH DELIVERABLE (SOFTWARE TESTING) **108**
7.1 INTRODUCTION: 108
7.2. TEST PLAN: 108
7.2.1. Purpose: *108*
7.2.2. Outline: *108*
7.3. TEST DESIGN SPECIFICATION: 111
7.3.1. Purpose: *111*
7.3.2. Outline: *111*
7.4. TEST CASE SPECIFICATION: 111
7.4.1. Purpose: *111*
7.4.2. Outline: *111*
7.5. TEST PROCEDURE SPECIFICATION: 123
7.5.1. Purpose: *123*
7.5.2 Outline: *123*
7.6. TEST ITEM TRANSMITTAL REPORT: 124
7.6.1. Purpose: *124*

7.6.2. Outline: *124*
7.7. TEST LOG : 125
7.7.1. Purpose: *125*
7.7.2. Outline: *125*
7.8. TEST INCIDENT REPORT 126
7.8.1. Purpose: *126*
7.8.2. Outline: *126*
7.9. TEST SUMMARY REPORT : 127
7.9.1. Purpose: *127*
7.9.2. Outline: *127*

CHAPTER 8: USER TECHNICAL MANUAL 129
A. GENERAL INFORMATION 129
8.1 SYSTEM OVERVIEW :..... 129
B. SYSTEM SUMMARY :..... 129
C. LOGGING ON:..... 141
8.2 SYSTEM MENU :..... 142
8.3 CHANGING USER ID AND PASSWORD : 143
8.4 EXIT SYSTEM : 143
D. USING THE SYSTEM :..... 145
8.5 [SYSTEM FUNCTION NAME] 145
8.5.1 [Exercises] *145*
8.5.2 [Step Counter] *145*
8.5.3 [Run Tracker] *146*
8.5.4 [BMI Calculator] *146*
8.5.5 [Water Reminder] *146*
8.5.6 [Nutrition Tracking] *146*
8.5.7 [Workout Plan] *147*
8.5.8 [AI chatbot] *147*
8.6 SPECIAL INSTRUCTIONS FOR ERROR CORRECTION: 147
8.7 CAVEATS AND EXCEPTIONS: 147
CHAPTER 9: REFERENCES..... 148

List of Tables

Table 1: Problem Overview Statement 12
Table 2: Gantt Chart.....16
Table 3: Schedule Feasibility.....22
Table 4: 30
Table 5: Task Dependency Table 31
Table 6:CMP 32
Table 7: Identify Critical Path 33
Table 8: Risk Identification 34
Table 9: Risk Drivers 34
Table 10: Risk Mitigation.....35
Table 11: Capture Shell Statements Table.....40
Table 12: Functional Requirements Tables.....42
Table 13:.....42
Table 14:.....42
Table 15:.....43
Table 16:.....43
Table 17:.....43
Table 18:.....44
Table 19:.....44
Table 20:.....45
Table 21: Non-Functional Requirements Table 45
Table 22 Requirements Traceability Matrix 48
Table 23: Usecase Tables 51
Table 24:.....52
Table 25:.....54
Table 26:.....55
Table 27:.....56
Table 28:.....57
Table 29:.....58
Table :30: 59
Table 31:.....60
Table 32:Data Model Tables 78
Table 33:.....78
Table 34:.....78
Table 35:.....79
Table 36:.....79
Table 37:.....79
Table 38:.....80
Table 39:.....80
Table 40:.....103
Table 41:.....104

Table 42:.....104
Table 43:.....105
Table 44:.....105
Table 45:.....106
Table 46:.....106
Table 47:.....107
Table 48:.....107
Table 49:Test Schedule Table: 110
Table 50:Testing Approval Table: 111
Table 51:Test case Specification Tables:.....111
Table 52:.....112
Table 53:.....113
Table 54:.....114
Table 55: 115
Table 56:.....116
Table 57:.....117
Table 58:.....121
Table 59:.....122
Table 60: Transmittal Approval Table:.....125
Table 61: Summary Table: 128
Table 60: Approvals Table: 128

List of Figures

Figure 1: Gantt Chart: 16
Figure 2: Network Diagram: 32
Figure 3: System Specification: 36
Figure 4:Organizational Chart: 37
Figure 5:Context Level DFD: 39
Figure 6: High Level Use Case Diagram: 49
Figure 7: Analysis Level Use Case Diagram: 50
Figure 8:Domain Model: 64
Figure 9: SSD Diagram: 65
Figure 10: Sequence Diagram: 66
Figure 11: 66
Figure 12: 67
Figure 13: 67
Figure 14: 68
Figure 15: 68
Figure 16: 69
Figure 17: Collaboration Diagram: 70
Figure 18: 70
Figure 19: 71
Figure 20: 71
Figure 21: Class Diagram.: 74
Figure 22: State Chart Diagram: 76
Figure 23: ERD: 82
Figure 24: DFD Level 0: 83
Figure 25: DFD Level 1: 84
Figure 26: Component Level Diagrams: 85
Figure 27: 86
Figure 28: 87
Figure 29: 88
Figure 30: 89
Figure 31: 90
Figure 32: 91
Figure 33: 92
Figure 34: Site Maps: 93
Figure 35: Story Boards: 94
Figure 36: 95
Figure 37: 95
Figure 39: 96
Figure 40: 96
Figure 41: 97
Figure 42: 97
Figure 43: 98

Figure 44:. 99
Figure 45: 100
Figure 46: 100
Figure 47: 101
Figure 48: 101
Figure 49: 102
Figure 50: 102
Figure 51: 103
Figure 52: System Summary 130
Figure 53: 130
Figure 54: 131
Figure 55: 132
Figure 56: 132
Figure 57: 133
Figure 58: 133
Figure 59: 134
Figure 60: 134
Figure 61: 135
Figure 62: 136
Figure 63: 137
Figure 64: 137
Figure 65: 138
Figure 66: 138
Figure 67: 139
Figure 68: 139
Figure 69: 140
Figure 70: 140
Figure 71: 141
Figure 72: 141
Figure 73: 142
Figure 74: 142
Figure 75: 141
Figure 76: 142
Figure 78: 144
Figure 79: 144
Figure 80: 141

Chapter 1: Final Project Proposal

1.1. Introduction

The fit Track app is designed to monitor and enhance user fitness levels through various features, including step counting, workout planning, water intake reminders, BMI calculation, and nutrition tracking.

The app will allow users to track physical activities, set fitness goals, and monitor progress, providing a personalized fitness experience. Integrated sensors and user-inputted data are used to give real-time feedback on activities like walking, running, and workouts.

1.2. *Project Title:*

" Fit Track App"

1.3.*Project Overview statement:*

Table 1: Project Overview Statement

Project Title: FitTrack App

Project Manager: Sir Abdullah

Project Members:

Name	Registration #	Email Address	Signature
Hira Nawaz		Nawazhira48@gmail.com	

Project Goal:
Create app that allow users to easily perform health related tasks without being worried about location and time constraints.

Objectives:

Sr.#	
1	To motive the interest of health and physical fitness.
2	To create a connection between physical fitness and health in one application.
3	To develop an android application that allows users to use it at anytime and anywhere.

Project Success criteria:

- User adoption and consistent usage of the app for health tracking.
- Positive feedback and ratings from users.
- Achievement of the defined health goals within the app (e.g., step goals, water intake, etc.).
- Completion of the planned features (step tracking, workout plan, nutrition logging).

Assumptions, Risks and Obstacles:
Assumption: Users will have access to smartphones with Android OS.
Risk: Data security and privacy concerns while handling user health data.
Obstacle: Unforeseen technical challenges while integrating features like step counting and nutrition tracking.

Organization Address (if any): None

Type of project: ☐Research ☐**Development**

Target End users:

Development Technology:	☐**Object Oriented** ☐Structured
Platform: ☐Web based ☐Distributed ☐**Mobile Application** ☐Setup Configurations ☐Other_____	
Approved By:	
Date:	

1.4. Project Goals & Objectives:

The main aim of this project is to develop an app to help improve the health of the user and achieve physical fitness by helping and motiving them to doing exercise and keeping track. The objectives of this project are:

• To motive the interest of health and physical fitness.
• To create a connection between physical fitness and health in one application.
• To develop an android application that allows users to use it at anytime and anywhere.

1.5. High-level system components:

- **Frontend/UI Layer**
  - Provides user interface components and interactions (e.g., buttons, charts, forms).
  - Includes fragments and activities for step counting, water tracking, nutrition tracking, and workout plans.
- **Backend/Business Logic Layer**
  - Implements core functionalities like water reminder, step counting algorithms, nutrition calculations, and data analysis etc.
  - Manages communication between the UI and database/network layers.
- **Database Layer**
  - Stores user data, including step history, nutrition logs, workout plans, and goals.
  - Uses **SQLite** for local storage and Firebase for cloud-based data (e.g., user authentication).
- **Networking Layer**
  - Handles online data, such as fetching nutrition tips or synchronizing data across devices.
  - Integrates with **Firebase** or other APIs for AI features and cloud backups.
- **AI/ML Module**
  - Powers the AI bot for nutrition suggestions.
  - Includes Watson Machine Learning for nutrition image classification and AI-driven insights.
- **Sensor Integration**
  - Interfaces with device sensors (e.g., accelerometer, gyroscope) for step counting, motion detection, and activity tracking.
- **Notification/Reminder System**
  - Manages scheduled notifications for water intake, workouts, or nutrition reminders.
- **User Authentication**
  - Firebase Authentication for login, signup, and profile management.

1.6. List of optional functional units:

Performance:

- **Features:** The app ensures efficient step counting, quick goal setting, and smooth transitions between screens.
- **Constraints:** Response times are optimized for basic operations but may slow under large data loads due to hardware limitations.

Ease of Learning and Use:

- **Features:** Intuitive navigation and minimal training requirements.
- **Constraints:** No extensive tutorials; users rely on self-guided exploration.

Budget and Costs:

- **Features:** Free access to core features with optional premium subscriptions for advanced modules (e.g., AI nutrition bot).
- **Constraints:** Financial costs for third-party integrations limit feature expansion.

Timetables and Deadlines:

- **Features:** Incremental delivery of features with milestones set for step counter, water reminder, and BMI calculator.
- **Constraints:** Strict deadlines might leave some features partially implemented or deferred.

Documentation and Training Needs:

- **Features:** A basic user manual and quick-start guide are provided.
- **Constraints:** Limited in-depth documentation for advanced functionalities.

Quality Management:

- **Features:** Regular performance testing and user feedback loops.
- **Constraints:** Limited testing environments may impact quality under diverse usage conditions.

Security and Internal Auditing Controls:

- **Features:** Data encryption and user authentication for privacy.
- **Constraints:** Advanced security measures (e.g., biometric authentication) are not included in the current phase.

1.7. Exclusions:

The following functional units are excluded from the scope of the Fit Track App due to time constraints, resource limitations, or project complexity:

1. **Wearable Device Integration**
    - The app will not directly connect or sync with external fitness wearables like smartwatches or fitness bands (e.g., Fitbit, Apple Watch).
    - Reason: Development of cross-platform compatibility and APIs for multiple devices is beyond the current project's scope.
2. **Advanced AI-Powered Insights**
    - While the app will provide basic analytics like step counts and BMI calculations, it will not include advanced AI-powered insights such as personalized fitness recommendations or predictive health alerts.
    - Reason: Implementation requires significant resources and expertise in AI/ML, which is outside the project's current capacity.
3. **Social Networking Features**

- The app will not include social media integration, leaderboards, or group challenges.
- Reason: Focus is on individual fitness tracking rather than community engagement.

4. **Third-Party Nutrition Database Integration**
   - The app will not access external nutrition databases or APIs to fetch detailed nutritional information for meals logged by users.
   - Reason: Time constraints and reliance on external data sources.
5. **Multi-Language Support**
   - The app will not offer multi-language support and will be developed only in English.
   - Reason: Resource constraints and the need to focus on core functionalities.
6. **Offline Full Data Sync**
   - The app will not provide offline syncing of all data with cloud services. Only limited functionalities, like step count tracking, will work offline.
   - Reason: Implementation of robust offline data sync mechanisms is resource-intensive.
7. **Medical Monitoring Features**
   - Features such as heart rate monitoring, blood oxygen level tracking, or integration with medical devices are excluded.
   - Reason: These features require specialized hardware and certifications, which are beyond the current scope.

1.8. Application Architecture:

Mobile App Interface

- Built using **Java** for native Android development.
- Utilizes Android Studio for development and testing.
- Follows **Material Design Guidelines** for a user-friendly and consistent UI experience.

Backend Architecture

- SQLite database for local data storage and efficient offline access to user data.
- Firebase for user authentication and cloud-based features, such as storing user profiles and syncing step goals across devices.

System Architecture

- **Two-Tier Architecture**
  1. **Client Tier**:
     - Mobile app built with Java, handling all user interactions, data visualization (step counter, nutrition tracker, workout progress), and sensor-based features (e.g., pedometer, GPS).
     - Android's built-in APIs for sensors and notifications.
  2. **Data Tier**:
     - Local SQLite database for user data such as step history, water logs, and BMI calculations.
     - Firebase cloud services for authentication and remote backup.

1.9. Gantt chart:

A Gantt chart visually represents the project timeline, showing task start and end dates, dependencies, and milestones. This helps in tracking progress and resource allocation efficiently.

Table 2

Task Name	Duration	Start Date	End Date
Idea Discussion	2d	Mar 02,2024	Mar 04,2024
Domain Study	3d	Mar 04,2024	Mar 05,2024
Selection	1d	Mar 05,2024	Apr 06,2024
Proposal Submission	4w	Apr 06,2024	Apr 21,2024
SRS Writing	2w	Apr 22,2024	May 20,2024
Design Document	4w	May 20,2024	June 6,2024
Evaluation	2w	June 6,2024	June 20,2024

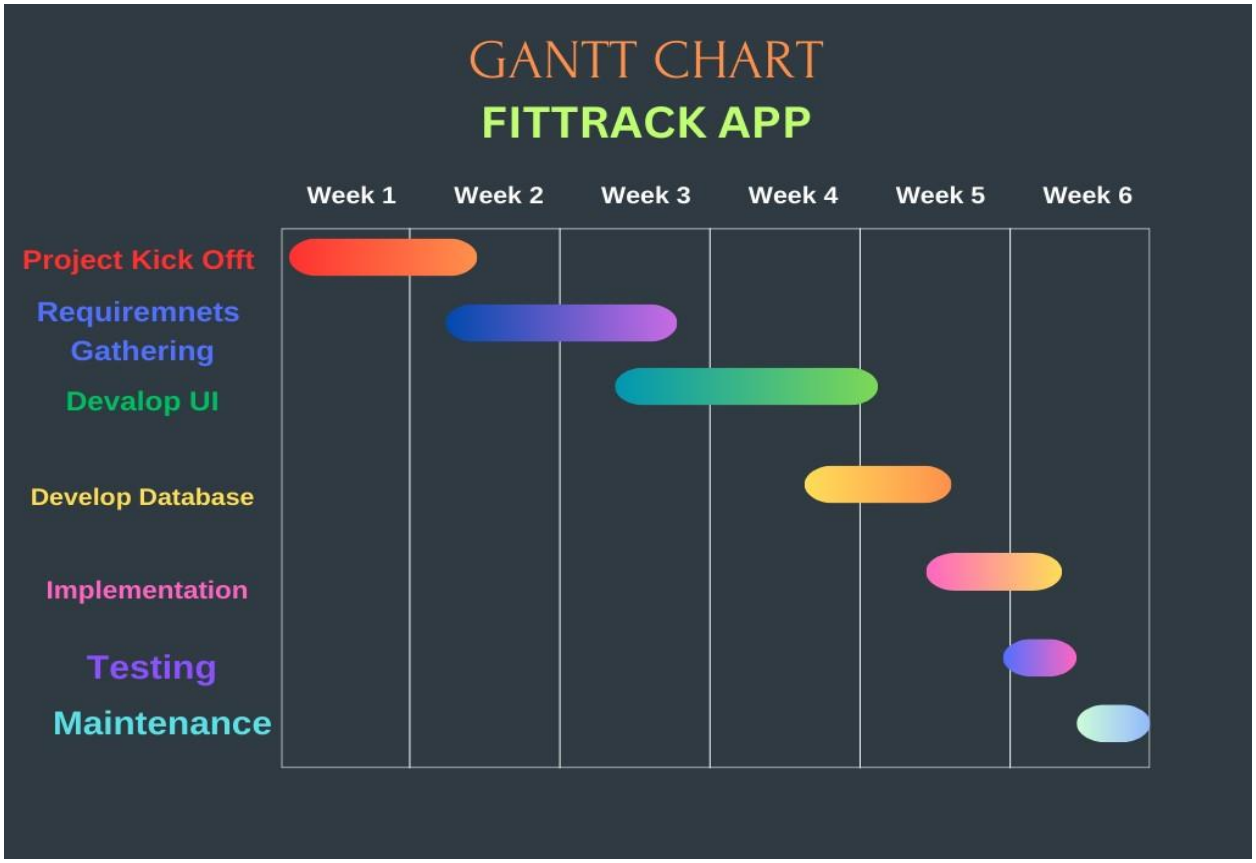

Figure 1

1.10. Hardware and Software Specification:

Hardware Specifications

1. **Development Environment**
    - **Machine Type**: Personal computer or laptop.
    - **Processor**: Intel Core i5 (or equivalent) or higher.
    - **RAM**: Minimum 8 GB (16 GB recommended for smooth development).
    - **Storage**: At least 256 GB of SSD (512 GB recommended).
    - **Graphics Card**: Integrated or discrete graphics for emulator performance.
2. **Mobile Devices**
    - **OS Compatibility**:
        - Android devices running **Android 6.0 (Marshmallow)** or above.
    - **Sensors**: Accelerometer, GPS, and pedometer support.
    - **Screen Size**: Any screen size with adaptable layouts.
3. **Testing Devices**
    - Variety of Android devices (smartphones, tablets) with different screen sizes and resolutions for testing.

Software Specifications

1. **Development Tools**
    - **IDE**: Android Studio (Giraffe 2023).
    - **Build Tools**: Gradle (Integrated with Android Studio).
    - **Java Development Kit (JDK)**: JDK 11 or higher.
    - **Android Emulator**: For testing across various virtual devices.
2. **Operating System**
    - **Development Machine**:
        - Windows 10
3. **Libraries and Frameworks**
    - Android Jetpack components (e.g., Room for database, Navigation for UI navigation).
    - Firebase SDKs (for authentication and cloud features).
4. **Database**
    - SQLite for local storage.
    - Firebase
5. **Additional Utilities**
    - **Version Control**: Git (GitHub or GitLab for repository management).
    - **Design Tools**: Figma or Adobe XD for UI prototyping.
    - **Testing Tools**: JUnit for unit testing, Android Debug Bridge (ADB) for device communication.

1.11. Tools and technologies used with reasoning:

The tools and technologies for the front-end and back-end of the fit track app are selected to ensure compatibility, scalability, and efficiency throughout the development process.

Front-End Tools

1. **Android Studio (IDE)**
    - **Reasoning**:
        - Comprehensive development environment specifically designed for Android app development.
        - Includes features like a code editor, debugger, emulator, and Gradle integration.
2. **XML (UI Design)**
    - **Reasoning**:
        - Allows for structured, responsive, and dynamic user interface design.
        - Supports integration with Android Views and Jetpack libraries.
3. **Java (Programming Language)**
    - **Reasoning**:
        - Native language for Android development, ensuring better performance and lower-level access to Android APIs.
        - Backward compatibility with older Android versions.
4. **Jetpack Libraries**
    - **Reasoning**:
        - Offers modern design patterns and components for handling lifecycle, navigation, and UI efficiently.
5. **Material Design**
    - **Reasoning**:
        - Provides guidelines and components to design intuitive, user-friendly, and visually appealing interfaces.

Back-End Tools

1. **SQLite (Database)**
    - **Reasoning**:
        - Lightweight, serverless database suitable for mobile applications.
        - Provides fast, efficient local data storage for tracking steps, goals, and history.
2. **Firebase (Authentication and Cloud Features)**
    - **Reasoning**:
        - Seamless user authentication and cloud data synchronization.
        - Provides scalable and cost-efficient real-time database solutions.
3. **Room Database (Jetpack)**
    - **Reasoning**:
        - Simplifies SQLite database management with object-oriented abstractions.
        - Offers compile-time verification of SQL queries.

Testing Tools

1. **JUnit (Unit Testing)**
    - **Reasoning**:
        - Ensures functionality and reliability of the app's individual components.
        - Provides automated and repeatable tests to catch issues early in development.

2. **Android Emulator**
    - **Reasoning**:
        - Enables testing the app on various virtual devices with different screen sizes and Android versions.
        - Reduces dependency on physical devices during development.

Version Control and Collaboration

1. **Git with GitHub/GitLab**
    - **Reasoning**:
        - Facilitates version control, code sharing, and collaboration among team members.
        - Tracks changes and provides a rollback option in case of issues.

Design Tools

1. **draw.io (Diagramming Tool)**
    - **Reasoning**:
        - Used for creating various system architecture and flow diagrams (e.g., class diagrams, data flow diagrams, network architecture).
        - Provides a user-friendly interface to design and export diagrams for integration into documentation.
        - Works well for team collaboration, allowing for shared diagrams, real-time editing, and cloud storage integration.

Reasons for Tool and Technology Selection

- **Development Process**:
    - Tools like JUnit, Android Studio, and Jetpack libraries support iterative development with frequent testing and updates.
- **Host and Target Platforms**:
    - Android Studio and Java ensure compatibility with the Android platform, supporting a wide range of devices and versions.
- **Proven Tools**:
    - Technologies like SQLite, Firebase, and Jetpack libraries have been thoroughly tested and are widely used in the industry.
- **Distribution of Development Organization**:
    - Git enables seamless collaboration, especially for distributed teams.
    - **draw.io** enables visually collaborative diagramming for team discussions and documentation.
- **Size of Development Effort**:
    - Jetpack components and pre-built libraries reduce development time and effort.
- **Budget and Time Constraints**:

  Open-source and free-to-use tools like Android Studio, SQLite, Git, and **draw.io** provide cost-effective solutions without compromising quality.

1.11. App Permissions:

- CAMERA Permission for Picture
- READ_EXTERNAL_STORAGE
- WRITE_EXTERNAL_STORAGE
- ACCESS_FINE_LOCATION
- ACCESS_COARSE_LOCATION
- POST_NOTIFICATION
- WAKE_LOCK
- SET_ALARM
- FOREGROUND_SERVICE
- ACTIVITY_RECOGNITION
- ACCESS_NETWORK_STATE
- INTERNET

Chapter 2: First Deliverable

2.1. Introduction

The first deliverable focuses on the planning and scheduling aspects of the project, ensuring a structured approach to its execution. This deliverable encompasses various key artifacts, each of which contributes to the overall planning and resource management needed for project success.

This chapter includes the following critical components:

- Project Feasibility
- Project Scope
- Project Costing
- Task Dependency Table
- Critical Path Method (CPM) Analysis
- Gantt Chart
- Introduction to Team Members
- Tasks and Member Assignment Table
- Tools and Technologies
- Vision Document
- Risk List
- Product Features

2.2. Project/Product Feasibility Report

There are different types of feasibility:

Types of Feasibility:

- Technical Feasibility
- Operational Feasibility
- Economic Feasibility
- Schedule Feasibility
- Specification Feasibility
- Information Feasibility
- Motivational Feasibility
- Legal and Ethical Feasibility

2.2.1. Technical Feasibility

It refers to the assessment of whether proposed system can be developed and implemented with the available technology, resources and skills. Fit Track app is totally an android based project. Each technology is freely available. Time limitation of the project development and the ease of implementation using these technologies are synchronized. The app is technically feasible, leveraging modern technologies such as :

Technologies:

- Java Programming
- SQLite for local data storage,
- Firebase for user authentication, and APIs for advanced functionalities like geolocation and notifications.
- XML

Tools:

- Microsoft Word for Documentation
- Android studio
- Draw.io for Diagrams

2.2.2. Operational Feasibility

I sure that the project will be operationally successful and raise no issues regarding different operations it performs. The user can easily operate the system as it provide users friendly environment. Therefore there will be no operational problem for the user of the system while using the app. From these it is clear that the project Fit Track App is Operationally feasible. The system is completely convenient for non technical user can easily use .

2.2.3. Economic Feasibility

The system will follow freeware software standards therefore no cost will be charged from the potential customers. Bug fixing and maintaining task will have an associated cost. The system being developed is economic with respect to user's point of view.

2.2.4. Schedule Feasibility

The project schedule is realistic, with milestones planned to ensure timely completion. The Gantt chart and CMP demonstrate the timeline feasibility. This is about assessment of amount of time required to complete the project and a determination of whether the project can be completed within the time constraints.

Table 3: Schedule feasibility

Task Name	Duration	Start Date	End Date
Idea Discussion	2d	April 02,2024	April 04,2024
Domain Study	3d	April 04,2024	April 06,2024
Selection	1d	April 06,2024	April 08,2024
Proposal Submission	5d	April 08,2024	May 20,2024
Feasibility Submission	2d	May 20,2024	May 26,2024

Proposal Defense	1w	May 26,2024	June 05,2024
SRS Writing	2w	June 05,2024	Jume 01,2024
Design Document	4w	Jume 01,2024	June 18,2024
Evaluation	2w	June 18,2024	July 2, 2024

2.2.5. Specification Feasibility

It refers to the evaluation of the project's technical and functional requirements and determination whether these requirements ca be achieved within the project limitations. The app adheres to functional specifications such as accurate step counting, BMI calculation, and geolocation-based run tracking while meeting non-functional specifications like performance, scalability, and security. Assesses whether the requirements are clear and defined. It ensures that the project's scope and boundaries are well understood.

2.2.6. Information Feasibility

The assessment of the availability and accessibility of the information required to support the project. All required information, including user data, and recommendations, is accessible through efficient data collection and storage mechanisms. The information that I gathered is meaningful, reliable and measurable. It can be authenticated once the system is complete.

2.2.7. Motivational Feasibility

The assessment of stakeholders willingness and motivation to support the project The app motivates users to adopt a healthier lifestyle by providing visual progress trackers, reminders, and rewards for achieving goals.

2.2.8. Legal & Ethical Feasibility

FitTracker App is freely available and provide the system an open source system. It has only maintenance cost. It will have great impact on peoples . The app is freely available by any user.
Legally and ethically I have rights to use tools that are used to develop my project like android studio etc. Which are legal in Pakistan.

2.3. Project/Product Scope

The **scope** refers to the boundaries of the features, functionalities, and goals that the app will achieve within its defined project constraints. The scope outlines what the app will include, such as step counting, workout tracking, calorie tracking, goal setting, progress monitoring, and integration with other fitness-related services, and what it will exclude, such as non-fitness-related features like social media integration or non-health-related tracking.

1. ***Core Features:***

1.Signup: User will create his/her profile.

2.Login: User will login his/her profile.

3. Step Counter: A step counter that will tracks daily steps to encourage users to stay active throughout the day and burnt calories will be shown and user will also be able to set goal steps its own and also can take suggestions by giving weight and height as input.

4. Workout planner: A simple workout planner which allows you to add and delete workout routines. And also save his data how much they drink water, his weight and height and also description of each workout plan.

5.BMI calculator: A BMI calculator to give the user an idea of their current health/fitness level based on height and weight.

6. Run Tracker: A geolocation tracker build using google maps to track a user's run and how much distance will be covered and have stopwatch that will note down time.

7. Exercises: Different exercises will be provided along with their instructions how to perform them. And also have countdown timer that calculate the time during exercises performed and all these exercises will be based on age such as before age eighteen and after age eighteen.

8. User-Interface: Interface will be more effective and user friendly.

9.Water Reminder: Users will be able to track how much they drink water and also this one remind them to drink water. Allow users to set a daily water intake goal based on their weight, height, and activity level. Display daily progress towards the water intake goal using visual indicators like progress bars.

10.Nutritions & Tips: User will be able to get tips and suggestions related to their health fitness.

11.AI chatbot/AI Nutrition (optional module): An AI-powered chatbot for instant answers to nutrition-related questions. It will help with meal planning, understanding nutritional labels, or finding healthy recipes, and also will provide personalized suggestions for improving diet to support overall health and fitness objectives.

2. Data Storage:

- **User Profiles**: Secure user profile storage, which includes personal data like age, weight, height, and fitness goals.
- **Local Database**: Data will be stored locally on the user's device for quick access (e.g., using SQLite) and synced with cloud services for backup if needed.

3. Notifications and Reminders:

- **Water Intake Reminders**: Reminders to drink water based on user preferences and activity level.
- **Step and Goal Reminders**: Notifications to encourage users to reach their daily step count or fitness goals.

4.User Interface and Experience:

- **User-Friendly UI**: An intuitive and visually appealing design to ensure ease of use.
- **Accessibility Features**: The app will support various accessibility features for users with disabilities, such as voice commands or screen readers.

5. Exclusions:

- **Non-Fitness Features**: The app will not include non-fitness functionalities, such as gaming or social media features.

- **Complex Health Metrics**: While the app may include basic health data (e.g., calories burned), it will not attempt to diagnose medical conditions or provide detailed health advice (e.g., blood pressure tracking, unless explicitly planned).

2.4. Project/Product Costing

Costing is essential for tracking expenses and resources. It involves estimating costs and monitoring the expenses over the course of the project. The estimation process can use various metrics and models.

2.4.1. Project Cost Estimation By Function Point Analysis

Step 1: Define Information Domain Values

First identify and count the **information domain values** for the **FitTrack** app. Here are the components of the app that would contribute to the function point count:

1. **Number of user inputs**: The data input by the user. In the FitTrack app, this could include inputs such as:
    - User profile details (age, weight, height)
    - Step goal
    - Meal logging (food items, quantity)
    - Water intake goal
2. **Number of user outputs**: These are the outputs shown to the user. For FitTrack, outputs may include:
    - Daily/weekly summary reports of steps, workouts, and calories
    - Visual progress indicators (e.g., progress bars for step count, calories burned)
    - Notifications and reminders (e.g., to drink water or achieve fitness goals)
3. **Number of user inquiries**: This refers to the queries where a user submits input and immediately receives a response.
    - Checking current step count or calorie expenditure.
    - Retrieving past fitness history.
4. **Number of files**: These are logical groupings of data stored in the system. For FitTrack, files might include:
    - User data file (profile, fitness goals)
    - Historical workout data file
    - Food and nutrition database
    - Water intake records
5. **Number of external interfaces**: These could include external systems or APIs interacting with the app, such as:
    - Integration with wearable devices (e.g., Fitbit, Apple Watch)
    - Third-party nutrition databases for food logging

Step 2: Calculate the Function Points (FP)

Each of these components (inputs, outputs, inquiries, files, interfaces) is assigned a complexity score: **Simple (1), Average (2), Complex (3)**.

Components	Cost	Complexity	Function Points (FP)
User Inputs	5	2 (Average)	10
User Outputs	4	2 (Average)	8
User Inquiries	3	2 (Average)	6
Logical Files	4	2 (Average)	8
External Interfaces	2	3 (Complex)	6
Total Function Points (FP)			**38**

Step 3: Determine the Value Adjustment Factor (VAF)

Now, for the **Value Adjustment Factor (VAF)**, need to assess the general system characteristics (GSCs) for the FitTrack app. Each of these characteristics is rated on a scale of 0 to 5 (no influence to strong influence).

Table 4

GSC	Rating (0-5)
1. Data communications	3
2. Distributed data processing	2
3. Performance	4
4. Heavily used configuration	3
5. Transaction rate	3
6. On-Line data entry	4
7. End-user efficiency	3
8. On-Line update	2
9. Complex processing	4
10. Reusability	2
11. Installation ease	3
12. Operational ease	3
13. Multiple sites	1
14. Facilitate change	2

The **VAF** is calculated as:

$VAF=0.65+0.01\times\sum(\text{Ratings})$

$VAF=0.65+0.01\times(3+2+4+3+3+4+3+2+4+2+3+3+1+2)=0.65+0.01\times35=1.00$

Step 4: Estimate Function Points (FP)

Now, we calculate the estimated function points (FP):

$FP\ est.=\text{Total Count of Function Points}\times(0.65+0.01\times VAF)$

FP est.=2 ×1.00=2

Step 5: Calculate Project Cost and Effort

Next, calculate the **Total Project Cost** and **Total Estimated Effort** based on **FP**.

Cost per Function Point (FP)

Average **cost per FP** is $2 and the **productivity parameter** is 20 function points per person per month.

Total Project Cost:

Total Project Cost=FP est.×Cost per FP

Total Project Cost=2×100=200 USD

Total Estimated Effort:

Total Estimated Effort=1.9 person-months

Final Estimate:

- **Total Project Cost**: $200
- **Total Estimated Effort**: 1.9 person-months

The formulae are given as follows:

Cost / FP = labor rate / productivity parameter

Total Project Cost = FP est. * (cost / FP)

Total Estimated Effort = FP est. / productivity parameter

2.4.2. Project Cost Estimation by using COCOMO'81 (Constructive Cost Model):

Using the COCOMO model, the project cost is estimated based on the app's development size, effort, and time. The project is categorized under the **Semi-Detached Mode**, given its moderate complexity and integration with external APIs.

- Person= Effort/ Schedule

Assumptions:

- **Size of the app (KLOC)**: 20 KLOC (an estimate for a medium-sized app like FitTrack).
- **Development Type**: Organic, based on small software teams developing familiar types of software.

Three levels:

Basic: Is used mostly for rough, early estimates.

Intermediate: Is the most commonly used version, includes 15 different factors to account for the influence of various project attributes such as personnel capability, use of modern tools, hardware constraints, and so forth.

Detailed: Accounts for the influence of the different factors on individual project phases: design, coding/testing, and integration/testing. Detailed COCOMO is not used very often.

Each level includes three software development types:

1. **Organic:** Relatively small software teams develop familiar types of software in an in-house environment. Most of the personnel have experience working with related systems.

2. **Embedded:** The project may require new technology, unfamiliar algorithms, or an innovative new method
3. **Semi-detached:** Is an intermediate stage between organic and embedded types.

Basic COCOMO

Type	Effort	Schedule
Organic	PM= 2.4 (KLOC)1.05	TD= 2.5(PM)0.38
Semi-Detached	PM= 3.0 (KLOC)1.12	TD= 2.5(PM)0.35
Embedded	PM= 2.4 (KLOC)1.20	T.D2.5(PM)0.32

PM= person-month (effort)
KLOC= lines of code, in thousands
TD= number of months estimated for software development (duration)

Intermediate COCOMO

Type	Effort
Organic	PM= 2.4 (KLOC)1.05 x M
Semi-Detached	PM= 3.0 (KLOC)1.12 x M
Embedded	PM= 2.4 (KLOC)1.20 x M

PM= person-month
KLOC= lines of code, in thousands
M.- reflects 15 predictor variables, called cost drivers

The schedule is determined using the Basic COCOMO schedule equations.

People Required = Effort / Duration

2.4.3. Activity Based Costing

Activity-based costing (ABC) is a methodology that measures the cost and performance of activities, resources, and cost objects. Resources are assigned to activities, then activities are assigned to cost objects based on their use. Activity-based costing recognizes the causal relationships of cost drivers to activities.
Activity-based costing is about:

- Measuring business process performance, activity by activity.
- Estimating the cost of business process outputs based on the cost of the resources used in producing the product.
- Identifying opportunities to improve process efficiency and effectiveness. Activity costs are used as the quantitative measurement. If activities have unusually high costs or vice versa, they become targets for re-engineering.

The basic steps would use for **FitTrack** development:

Basic Cost Drivers:

For each activity state in an activity diagram, the basic cost drivers are:

- Resources: determine what business workers and business entities are participating, and how many instances of each. The allocation of a resource to a workflow implies a certain cost.
- Cost rate: each business worker or business entity instance may have a cost per time in use.
- Duration: an activity occurs for a certain time, therefore a resource can either be allocated for the duration of the activity, or for a fixed amount of time.
- Overhead: any fixed costs that the invocation of a workflow or an activity would incur.

Activities:

- Requirements Gathering
- App Design
- App Development (Coding)
- Testing
- Deployment
- Maintenance

Resources:

- Project Manager
- Developers **(Mobile app development)**
- UI/UX Designers
- Testers
- System Admins
- QA Engineers

Cost Rates: We would assign cost rates to each resource. For instance, hourly rates might look like this:

- Project Manager: $10
- Developer: $0
- Designer: $0
- Tester: $2

Duration: Duration can be estimated based on effort (from the previous COCOMO model). For instance, if we assume the total effort of 54.72 person-months translates into **average resources working 160 hours/month**, this would give us an estimate for each resource's workload.

Overhead: Overhead costs (e.g., office space, software licenses, administrative support) might account for a fixed percentage (e.g., 20%) of the total costs.

2.5. Task Dependency Table

A dependency table outlines tasks and their relationships, ensuring smooth project execution.

Table 5: Task Dependency Table

Task no #	Activity	Dependencies
T1	Interest Discussion	-
T2	Field Study	-
T3	Selection	T1, T2
T4	Platform Investigation	T3
T5	Technology Analysis	T4
T6	Resource Evaluation	T5
T7	Literature Research	T6
T8	Existing product research	T2
T9	Potential user's question	T7
T10	Requirements	T8
T11	SRS	T9
T12	Design	T10
T13	GUI Diagram	T11
T14	Front End Implementation	T12
T15	Back End Implementation	T13
T16	Testing	T14
T17	Deployment	T15

2.6. CPM - Critical Path Method

In 1957, DuPont developed a project management method designed to address the challenge of shutting down chemical plants for maintenance and then restarting the plants once the maintenance had been completed. Given the complexity of the process, they developed the Critical Path Method (CPM) for managing such projects. CPM provides the following benefits:

- Provides a graphical view of the project.
- Predicts the time required to complete the project.
- Shows which activities are critical to maintaining the schedule and which are not.

CPM models the activities and events of a project as a network. Activities are depicted as nodes on the network and events that signify the beginning or ending of activities are depicted as arcs or lines between the nodes. The following is an example of a CPM network diagram:

Steps in CPM Project Planning :

1. Specify the individual activities.
2. Determine the sequence of those activities.
3. Draw a network diagram.
4. Estimate the completion time for each activity.

5. Identify the critical path (longest path through the network)
6. Update the CPM diagram as the project progresses.

1. Specify the Individual Activities

Based on the FitTrack app's requirements, the following are some essential activities in the project development:

Table 6

Task NO #	Activity	Dependencies
T1	Interest Discussion	-
T2	Field Study	-
T3	Selection	T1, T2
T4	Technology Analysis	T3
T5	Existing product research	T2
T6	Potential user's question	T4
T7	Requirements	T5
T8	SRS	T6,T7
T9	Design	T8
T10	Use Case Diagram	T8
T11	ERD	T9
T12	GUI Diagram	T10, T9
T13	Object Modeling	T11
T14	Front End Implementation	T12
T15	Back End Implementation	T12
T16	Testing	T13

2. Determine the Sequence of the Activities

- Activity T1 must be completed before starting T2.
- Activities T14and T15 can proceed in parallel, but T9 depends on both.
- Activity T16 can begin once T14 and T15 is completed. So on.

3. Draw the Network Diagram

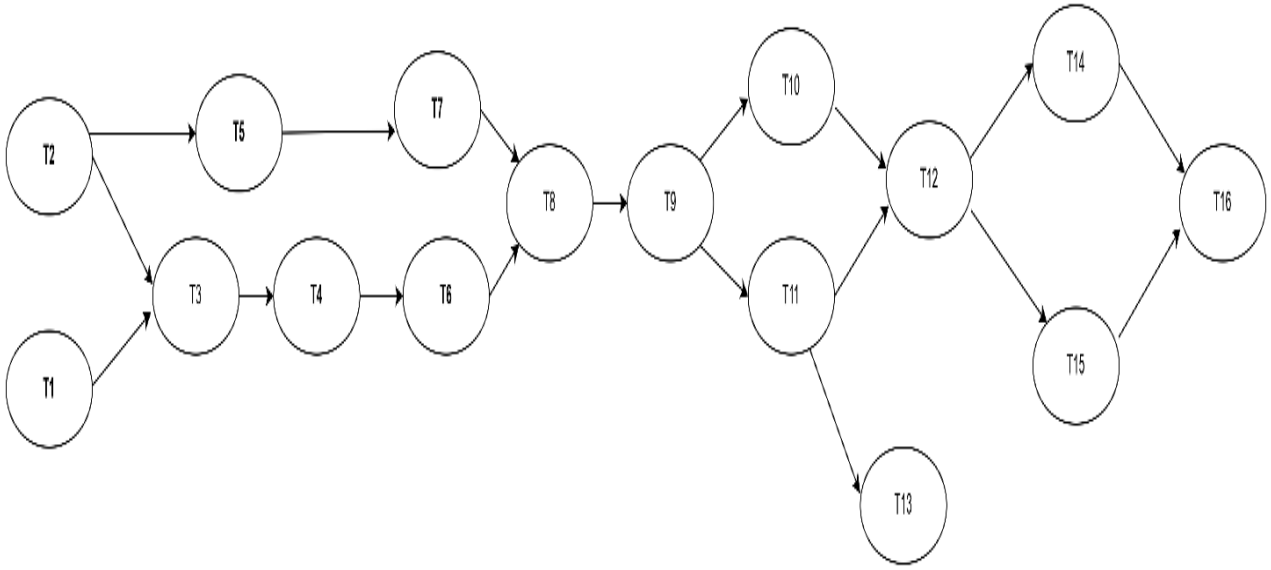

Figure 2

4. Estimate Activity Completion Time

T1: 1 weeks
T2: 1 weeks
T3: 1 weeks
T4: 2 weeks
T5: 2 weeks
T6: 2 weeks
T7: 1 weeks
T8: 2 weeks
T9: 2 weeks
T10:1weeks
T11: 1 weeks
T12: 2 weeks
T13: 6weeks
T15: 6 weeks
T16: 2 weeks

5. Identify the Critical Path

Table 7: Identify the Critical Path

Task NO #	Activity	Duration	ES	EF	LS	LF	TS	FS
T1	Interest Discussion	1W	0	1W	0	0	0	0
T2	Field Study	1W	0	1W	0	0	0	0
T3	Selection	2W	1W	3W	1W	1W	0	0
T4	Platform Investigation	1W	3W	4W	3W	3W	0	0
T5	Technology Analysis	2W	4W	6W	4W	4W	0	0
T6	Resource Evaluation	2W	6W	8W	6W	6W	0	0
T7	Literature Research	2W	8W	10W	8W	8W	0	0
T8	Existing product research	1W	3W	12W	11W	10W	8	0
T9	Potential user's question	2W	10W	16W	10W	12W	0	0
T10	Requirements	4W	4W	17W	12W	12W	0	0
T11	SRS	1W	12W	19W	16W	16W	0	0
T12	Design	2W	15W	23W	17W	17W	0	0
T13	GUI Diagram	2W	18W	24W	21W	21W	0	0
T14	Front End Implementation	6W	24W	30W	24W	24W	0	0
T15	Back End Implementation	6W	24W	30W	24W	24W	0	0
T16	Testing	4W	30W	34W	30W	30W	0	0

- ES: earliest start time
- EF: earliest finish time
- LF: latest start time
- LS: latest finish time
- TS: Total Slack
- FS: Free Slack

6. Update CPM Diagram

The parameters and slacks are calculated as follows:

The critical path is:

T1 → T2 → T3 → T4 → T5 → T6 → T7 → T9 → T10 → T11 → T12 → T13 → T14 → T15 → T16, as these tasks have **Slack Time (TS) = 0**, meaning any delay in these tasks will delay the project.

2.7. Introduction to Team member and their skill set

No team member single person.

2.8. Vision Document

The vision of the fitness tracker app is to provide a holistic and user-friendly platform for monitoring and improving physical and mental health. By integrating features like step counting, nutrition tracking, and water reminders, the app encourages healthier habits and provides valuable insights to users.

2.9. Risk List

Identified risks and mitigation strategies:

Table 8: Risk Identification

Risk ID	Risk Description
R1	Delays in development due to scope creep or unforeseen technical challenges.
R2	Dependency on third-party APIs (e.g., Firebase, IBM Watson, Google Cloud) failing.
R3	Lack of hardware resources for testing across multiple devices (e.g., Android)
R4	Performance issues with the app, especially on older devices or low bandwidth.
R5	Security vulnerabilities in handling user data (login, nutrition, workout data).
R6	Failure to meet user expectations for UX/UI design and app responsiveness.
R7	High server or cloud service costs beyond the planned budget.

Table 9: Risk Drivers

Risk Driver	Risk Description	Percentage Impact
RD1	Extended development time due to complexity of features.	25%
RD2	Unstable third-party services (API outages, rate limits, etc.).	20%
RD3	Insufficient testing across different hardware platforms.	15%
RD4	Poor app performance leading to crashes or slow responses.	15%
RD5	Data security breaches causing legal and financial risks.	10%

RD6	Budget overruns due to unplanned cloud infrastructure or development costs.	10%
RD7	Lack of interest or user adoption due to insufficient marketing and app promotion.	5%

Table 10: Risk Mitigation Plan

Risk ID	Mitigation Strategy	Risk Impact
R1	Conduct regular sprint reviews to ensure the project scope is adhered to. Break down large tasks into manageable parts.	Moderate
R2	Use reliable third-party APIs and ensure fallbacks or alternative solutions are in place. Keep API dependencies to a minimum where possible.	High
R3	Invest in cloud-based testing platforms like AWS Device Farm or Firebase Test Lab to ensure coverage across multiple devices.	Moderate
R4	Optimize the app's code for performance (e.g., memory management, data processing) and perform stress tests to identify bottlenecks.	High
R5	Implement strong encryption and follow best practices for user data security. Perform regular security audits and stay up to date with latest standards.	High
R6	Regularly monitor project expenses against the budget and establish clear guidelines for handling scope changes or unexpected costs.	Moderate
R7	Allocate a portion of the project budget for marketing and user acquisition. Use ASO (App Store Optimization) techniques to boost app visibility.	Low

2.10. Product Features/ Product Decomposition

Functional requirements capture the intended behavior of the system. This behavior may be expressed as services, tasks or functions the system is required to perform. The functions of system are:

- Exercises
- Water Reminder
- BMI Calculator
- Run Tracker
- Nutrition Tracking
- AI Chatbot
- Workout Plan

Chapter 3: Second Deliverable For Object Oriented Approach

3.1 Introduction:

Requirements engineering process provides the appropriate mechanism for understanding what the customer wants, analyzing need, assessing feasibility, negotiating a reasonable solution, specifying the solution unambiguously, validating the specification and managing the requirements as they are transformed into an operational system. The task of capturing, structuring, and accurately representing the user's requirements so that they can be correctly embodied in systems which meet those requirements (i.e. are of good quality).

- Requirements elicitation
- Requirements analysis and negotiation
- Requirements specification
- System modeling
- Requirements validation
- Requirements management

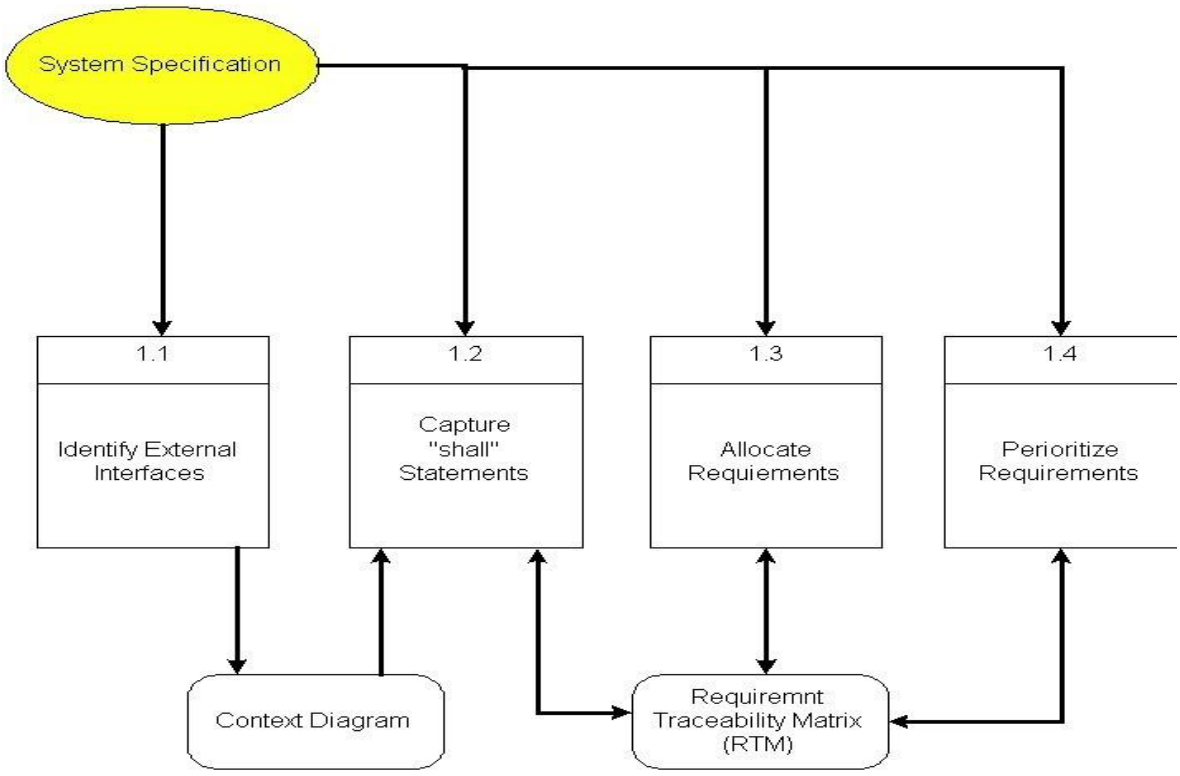

Figure 3

Here, requirements specification is to be discussed. Requirements specification would lead to the following four steps:

- Identify external interfaces
- Development of context diagram
- Capture “shall statements
- Allocate requirements
- Prioritize requirements
- Development of requirements traceability matrix

3.1.1 Systems Specifications

Introduction

This clause should contain brief “Introduction” of the system under discussion domain knowledge. It can also contain company, its location, its historical background and its current status in the market. The most important part of this clause is to give an overview of the major business areas of the company. This overview must be very brief so that one can get a bird’s eye view of the organization under study.

Existing System

Currently, users rely on fragmented tools such as standalone fitness trackers, separate nutrition apps, and manual tracking methods to monitor their health. These solutions lack integration and do not provide a holistic view of an individual’s fitness progress. Common issues in the existing systems include:

- **Inaccuracy in Step Counting**: Traditional step counters often count random hand motions as steps.
- **Disjointed Workflows**: Separate apps for workouts, nutrition, and hydration tracking complicate the user experience.
- **Limited Personalization**: Existing solutions fail to account for individual differences like weight, height, or specific fitness goals.
- **Poor Engagement**: Lack of reminders, motivational tools, or progress tracking reduces user engagement.

Organizational Chart

The organizational chart is given as:

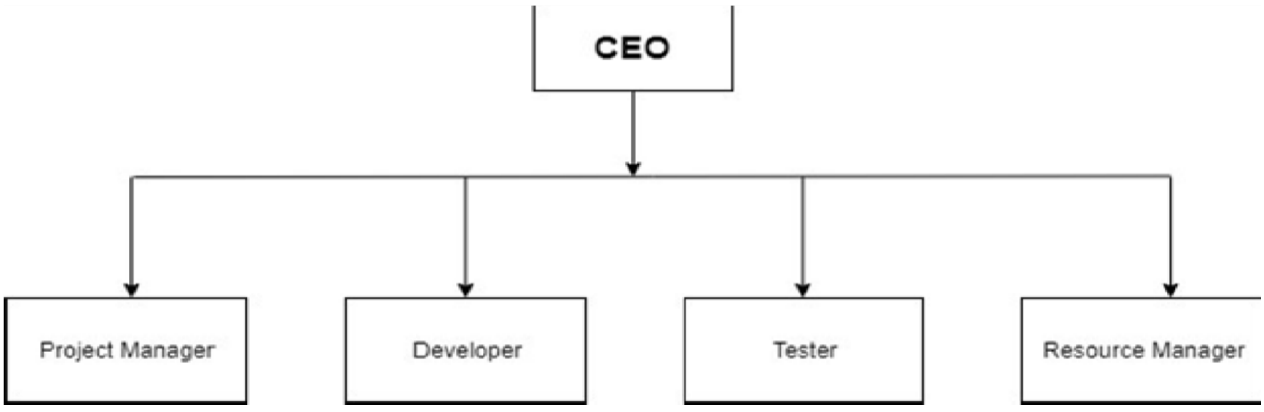

Figure 4

Summary of Requirements: (Initial Requirements)

The FitTrack app must:

- Provide accurate and reliable step tracking.
- Enable users to set and achieve fitness goals through personalized plans.
- Support data logging for nutrition and hydration.
- Offer reminders and notifications to promote user engagement.
- Ensure data security and privacy.

3.1.2. Identifying External Entities:

From the abstract, the following entities are identified:

The external entities interacting with the app are:

- **Users**: Interact with the app to track fitness, set goals, and monitor progress.
- **Sensors**: Provide data for step counting and geolocation for run tracking.
- **Notifications Service**: Handles reminders for water intake and workout schedules.
- **Database**: Stores user data, fitness records, and goals.
- **Third-Party APIs**: Services used for nutrition analysis or integration with fitness devices.

Perform Refinement

After refining, the relevant entities include:

- **User**: Interacts with all app features.
- **Wearables**: Optional integration for advanced tracking.
- **Notifications System**: Sends hydration and activity reminders.
- **Database**: Manages and stores user-specific data.

3.1.3. Context Level Data Flow Diagram:

A graphical tool for communication with users, managers and personnel. Focus on the movement of data between external entities and processes, and between processes and data stores. The context-level DFD represents the interaction between the user, external systems, and the app. Key components include:

- Input from sensors and user.
- Data processed and stored in the database.
- Output displayed as insights, reminders, and progress reports.

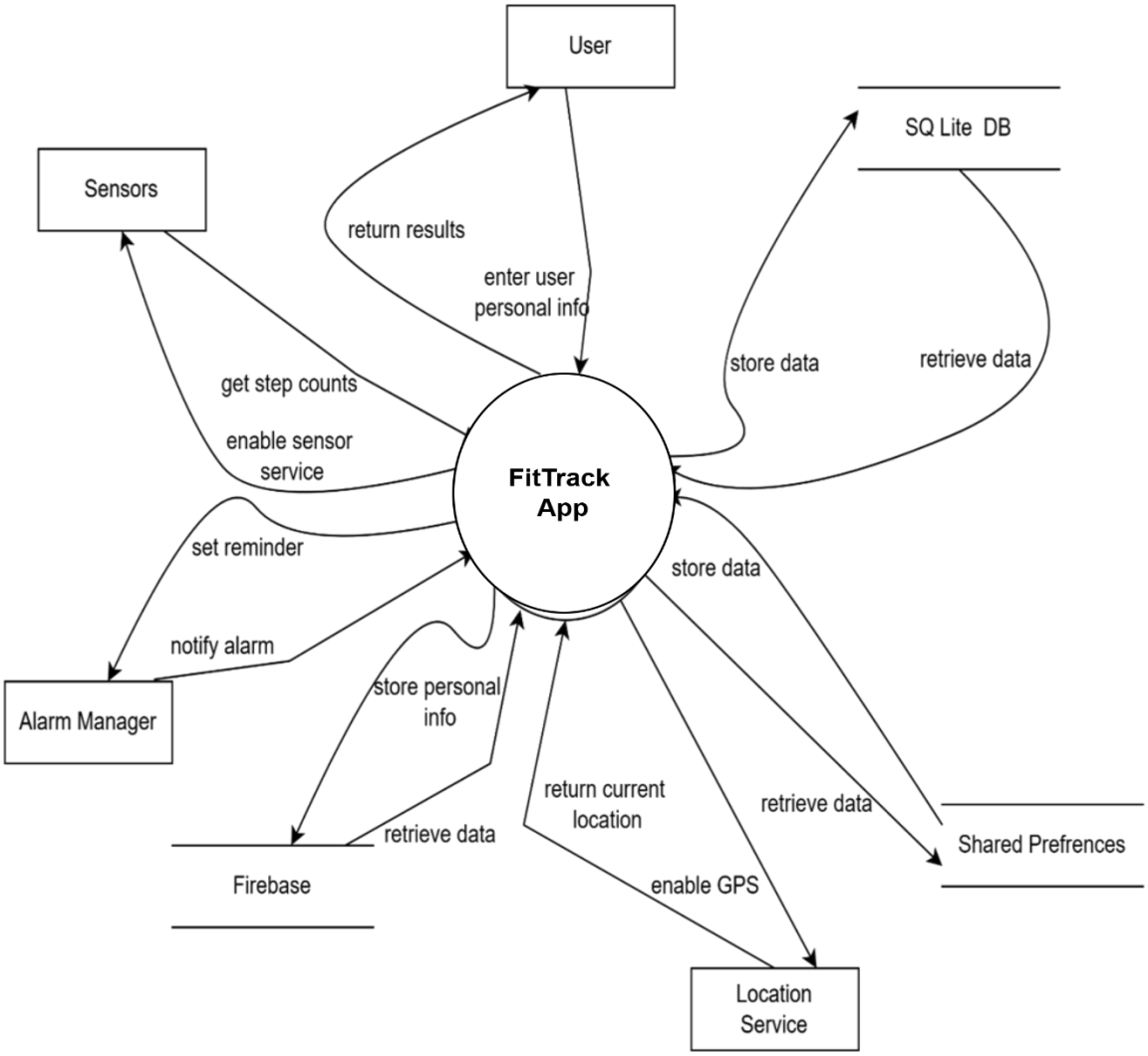

Figure 5

Entities

1. **User**: Interacts with the app to track exercises, log water intake, view reminders, and check nutrition data.
2. **Database (SQLite/Firebase)**: Stores user data, exercise plans, step counts, water reminders, and nutrition info.
3. **Sensors**: Provides step data for step counting and physical activity tracking.

Process

- **Fitness Tracker App**: Central system that processes user inputs, logs activity data, calculates stats, and sends reminders.

Data Flow:

- User interacts with the Fitness Tracker App.
- Device Sensors send data to the app.
- App interacts with Third-Party Services for additional functionalities.

2.1.4. Capture "shall" Statements:

The use of "Shall" statements is prevalent in the software systems development process, as they offer a precise and concise means of specifying requirements and guaranteeing that the system satisfies the stakeholders' needs. Typically, they form a crucial part of the requirements gathering and analysis stage, during which the development team and stakeholders collaborate to outline the functional and non-functional requirements of the system.

Table 11: "shall" Statements

Para #	Initial Requirements
1.1	*Step Tracking* 1. The system **shall** count steps accurately, minimizing false positives caused by hand movements. 2. The system **shall** display real-time step count updates to the user. 3. The system **shall** allow users to set daily step goals. 4. The system **shall** track and log steps taken over a specified time period for historical data.
1.2	*Workout Plans* 5. The system **shall** provide personalized workout recommendations based on user preferences and fitness levels. 6. The system **shall** include pre-designed workout routines for different fitness goals (e.g., weight loss, muscle gain). 7. The system **shall** allow users to schedule workouts and receive reminders. 8. The system **shall** display progress toward workout goals.
1.3	*Nutrition Tracking* 9. The system **shall** enable users to log daily food intake, including calorie and macronutrient details. 10. The system **shall** analyze and provide feedback on the user's nutrition habits. 11. The system **shall** allow users to set nutrition goals (e.g., daily calorie intake).

	12. The system **shall** store nutrition data for historical analysis
1.4	*Hydration Tracking* 13. The system **shall** send reminders to users to drink water at regular intervals. 14. The system **shall** allow users to log water intake throughout the day. 15. The system **shall** calculate hydration goals based on user-specific data (e.g., weight, activity level). 16. The system **shall** track water intake and display progress toward hydration goals.
1.5	*User Interface* 17. The system **shall** display progress using graphs and visual representations. 18. The system **shall** allow users to customize settings for reminders, goals, and notifications.
1.6	*Data Storage and Synchronization* 20. The system **shall** store user data securely in a local database. 21. The system **shall** back up user data to prevent loss in case of system failure.
2.0	*Notifications and Reminders* 23. The system **shall** send notifications for goal achievements. 24. The system **shall** provide motivational prompts to encourage user engagement. 25. The system **shall** allow users to customize the timing and type of notifications.

3.1.5. Functional Requirements:

The app's functional requirements include:

- **Step Counter**: Tracks and displays steps, distance, and calories burned.
- **Nutrition Tracking**: Allows users to log meals and track calories.
- **Water Reminder**: Sends notifications for timely water intake.
- **BMI Calculator**: Calculates BMI based on user-provided data.
- **Workout Plan**: Provides personalized workout schedules.
- **Run Tracker**: Tracks distance and pace using GPS data.

Table 12

Identifier	**REQ-001**
Title	Signup
Requirement	The system shall allow users to sign up with an email address and password.
Source	App Owner
Rationale	To enable new users to create an account and access the application
Restrictions and Risk	Email validation, strong password requirements, potential spam users
Dependencies	REQ-002
Priority	High

Table 13

Identifier	**REQ-002**
Title	Login
Requirement	The system shall allow users to log in using their email and password.
Source	App Owner
Rationale	To provide access to registered users.
Restrictions and Risk	Account lockout after multiple failed attempts, secure password storage.
Dependencies	REQ-001
Priority	High

Table 14

Identifier	**REQ-003**
Title	Workout plan
Requirement	The system shall allow users to create, edit and view a custom workout plan
Source	App Owner
Rationale	To enable users to tailor their fitness routines according to their goals and preferences
Restrictions and Risk	Incorrect user input, overwhelming number of options.
Dependencies	None
Priority	High

Table 15

Identifier	**REQ-004**
Title	Step Counter
Requirement	The system shall count user steps in real-time using device sensors.
Source	App Owner
Rationale	To provide users with immediate feedback on their physical activity levels.
Restrictions and Risk	Sensor accuracy, battery consumption, background app permissions.
Dependencies	None
Priority	High

Table 16

Identifier	**REQ-005**
Title	BMI Calculator
Requirement	The system shall calculate BMI using the formula: weight (kg) / height (m)^2.And also take weight and height from user as Input.
Source	App Owner
Rationale	To provide users with their BMI, which is an indicator of body fatness
Restrictions and Risk	Calculation accuracy, handling edge cases (e.g., extreme values).
Dependencies	Input Taken form user
Priority	High

Table 17

Identifier	**REQ-006**
Title	Exercises
Requirement	The system shall maintain a comprehensive database of exercises, categorize exercises based on muscle groups and fitness goals, and also provide detailed descriptions and instructions for each exercise.
Source	Fitness Expert

Rationale	To ensure users perform exercises correctly and safely.
Restrictions and Risk	Insufficient or unclear instructions, potential for user injury.
Dependencies	None
Priority	High

Table 18

Identifier	**REQ-009**
Title	Nutrition Tracking
Requirement	The system shall allow users to log and track their daily nutritional intake (e.g., calories, macronutrients).
Source	App Owner
Rationale	To help users monitor their diet and ensure they are meeting their nutritional goals.
Restrictions and Risk	Users may enter incorrect data, lack of real-time food database updates, or inconsistent tracking.
Dependencies	REQ-001 (User Signup), REQ-002 (User Login)
Priority	Medium

Table 19

Identifier	**REQ-007**
Title	Run Tracker
Requirement	The system shall track the user's running route and distance using GPS.
Source	App Owner
Rationale	To accurately record the user's running activity and monitor their performance and adjust their running speed accordingly provide real-time feedback users monitor their performance.
Restrictions and Risk	GPS signal accuracy, battery consumption, location privacy concerns, background app behavior, Inaccurate pace calculations, user frustration with inconsistent data.
Dependencies	None
Priority	High

Table 20

Identifier	REQ-008
Title	Water Reminder
Requirement	The system shall allow users to set daily water intake reminders and track their water consumption.
Source	App Owner
Rationale	To help users maintain proper hydration throughout the day by reminding them to drink water at regular intervals.
Restrictions and Risk	User may disable notifications, leading to missed reminders; potential inaccurate tracking if the user doesn't log water intake consistently.
Dependencies	REQ-001 (User Signup), REQ-002 (User Login)
Priority	High

3.1.6. Non-Functional Requirements:

Non-functional requirements are like the quality standards for a product or system. They tell us how well it should work, not just what it should do.

Table 21

Req No	Description	Type
NR-101	The app should keep running even if the user switches to another app or locks their phone. It will only stop running when the user intentionally closes the app.The app should process and update data within 2 seconds.	**Performance**
NR-102	The app should be accessible to the user at all times, without interruption or delay, regardless of time or location Notifications must fire accurately; no data loss during app crashes.	**Reliability & Availability**

	.	
NR-103	The application should be user friendly and simple, engaging UI, easy to navigate. The layout should simple and make use of a concurrent theme with a good color scheme, in order to grab the attention of users.	**Usability**
NR-104	The application should secure any confidential data The FitTrack store users health and personal sensitive information that should be kept secret by using two-factor authentication and password.	**Security**
NR-105	The app stores user profiles, workout histories, step counts, health metrics (e.g., BMI, blood pressure), and other fitness-related data. Currently, the storage requirements are moderate, as each user's data is relatively small in size. SQLite efficiently handles this data on the user's device, while Firebase manages user authentication and synchronization of key data across devices. It will efficiently manage increasing data in future by optimizing local storage, utilizing scalable cloud storage, archiving older data, ensuring flexible system architecture, and closely monitoring performance.	**Capacity**

NR-106	The application should be compatible with a wide range of devices and operating systems. Should function correctly and seamlessly across different platforms, screen sizes, and hardware configurations. This includes compatibility with various mobile devices, tablets, and desktop computers. The app must be compatible with Android 8.0+ devices.	**Compatibility**
NR-107	The application should be able to handle increasing numbers of users and data without compromising performance or functionality, by expanding its resources	**Scalability**
NR-108	The app should be built with well-organized code that is easy to understand. Different parts of the app, like tracking steps, workouts, food, and logins, should be separate and easy to change without causing problem the whole app. This makes it simple to fix problems, add new things, or improve the app over time.	**Maintainability**

3.1.7. Requirements Trace-ability Matrix:

The aim of RTM is to guarantee that all requirements are adequately traced and tracked throughout the development life cycle, and any changes or modifications to the requirements are properly documented and comprehended. The creation of an RTM is a vital stage in system development, as it ensures that all requirements are thoroughly traced and tracked throughout the development life cycle.

Table 22

Sr#	Feature	Use case ID	UI ID	Priority	Build Number	Use Case Cross reference
1	**User Login**	UC_01	UI_01	High	Build_001	UC_02 (Sign-Up)
2	**User Registration (Sign-Up)**	UC_02	UI_02	High	Build_001	UC_01 (Login)
3	**Step Counter Tracking**	UC_03	UI_03	High	Build_002	UC_04 (Water Reminder)
4	**Water Intake Reminder**	UC_04	UI_04	Medium	Build_003	UC_03 (Step Counter), UC_05 (Nutrition)
5	**Nutrition Logging**	UC_05	UI_05	Medium	Build_003	UC_04 (Water Reminder), UC_06 (Workout Plan)
6	**Workout Plan Management**	UC_06	UI_06	High	Build_004	UC_05 (Nutrition), UC_07 (Run Tracker)
7	**Run Tracker**	UC_07	UI_07	Medium	Build_004	UC_06 (Workout Plan)
8	**View Exercise History**	UC_08	UI_08	Low	Build_005	UC_06 (Workout Plan)
9	**View Water Consumption History**	UC_09	UI_09	Low	Build_005	UC_04 (Water Reminder)
10	**View Nutrition History**	UC_10	UI_10	Low	Build_005	UC_05 (Nutrition)

3.1.10. High Level Usecase Diagram:

The high-level use case diagram represents the overall interaction between the user and system modules, including step counting, nutrition tracking, and goal setting and others.

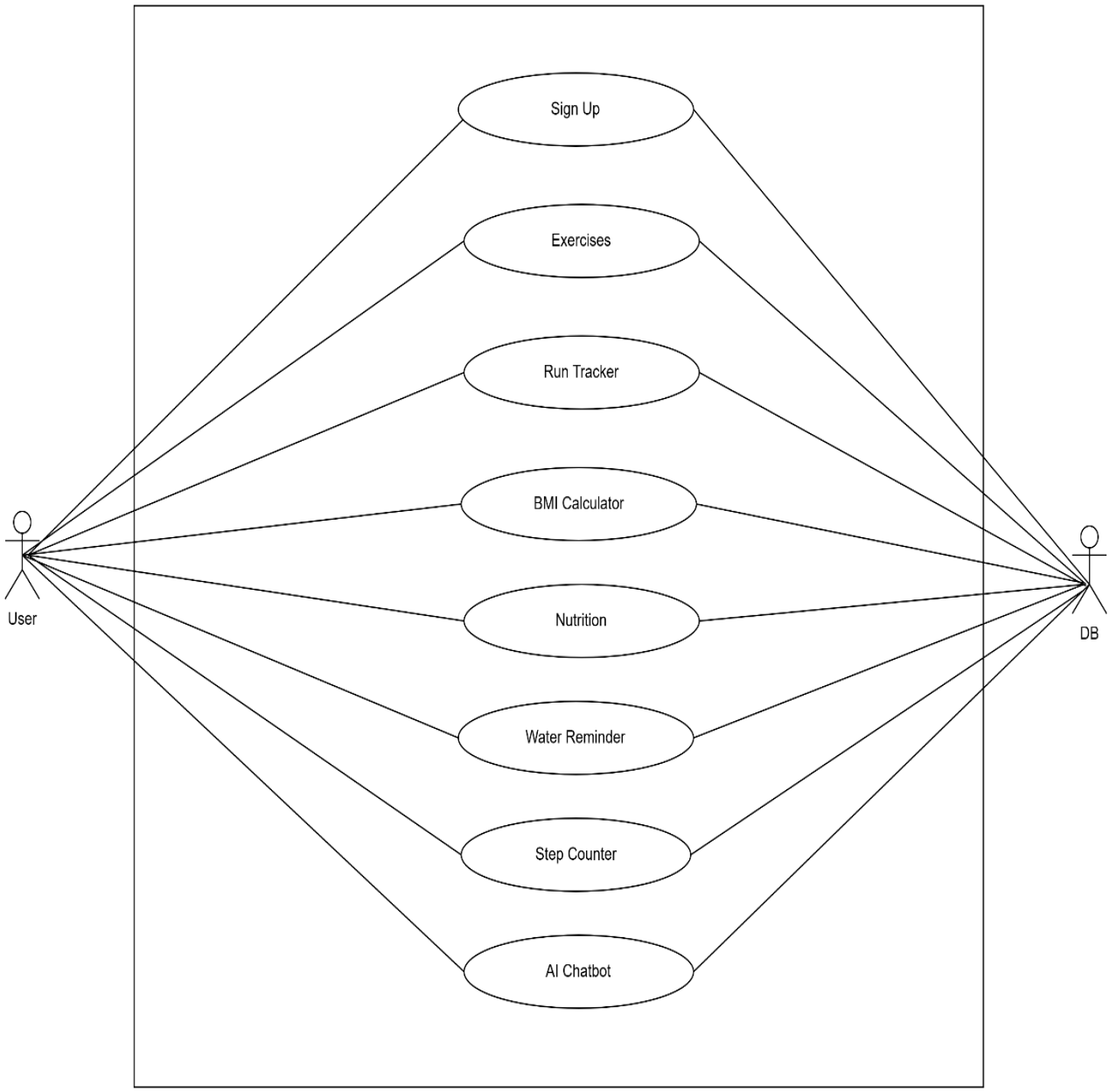

Figure 6

3.2.11. Analysis Level Usecase Diagram:

The analysis-level diagram provides a detailed view of each module's use cases.

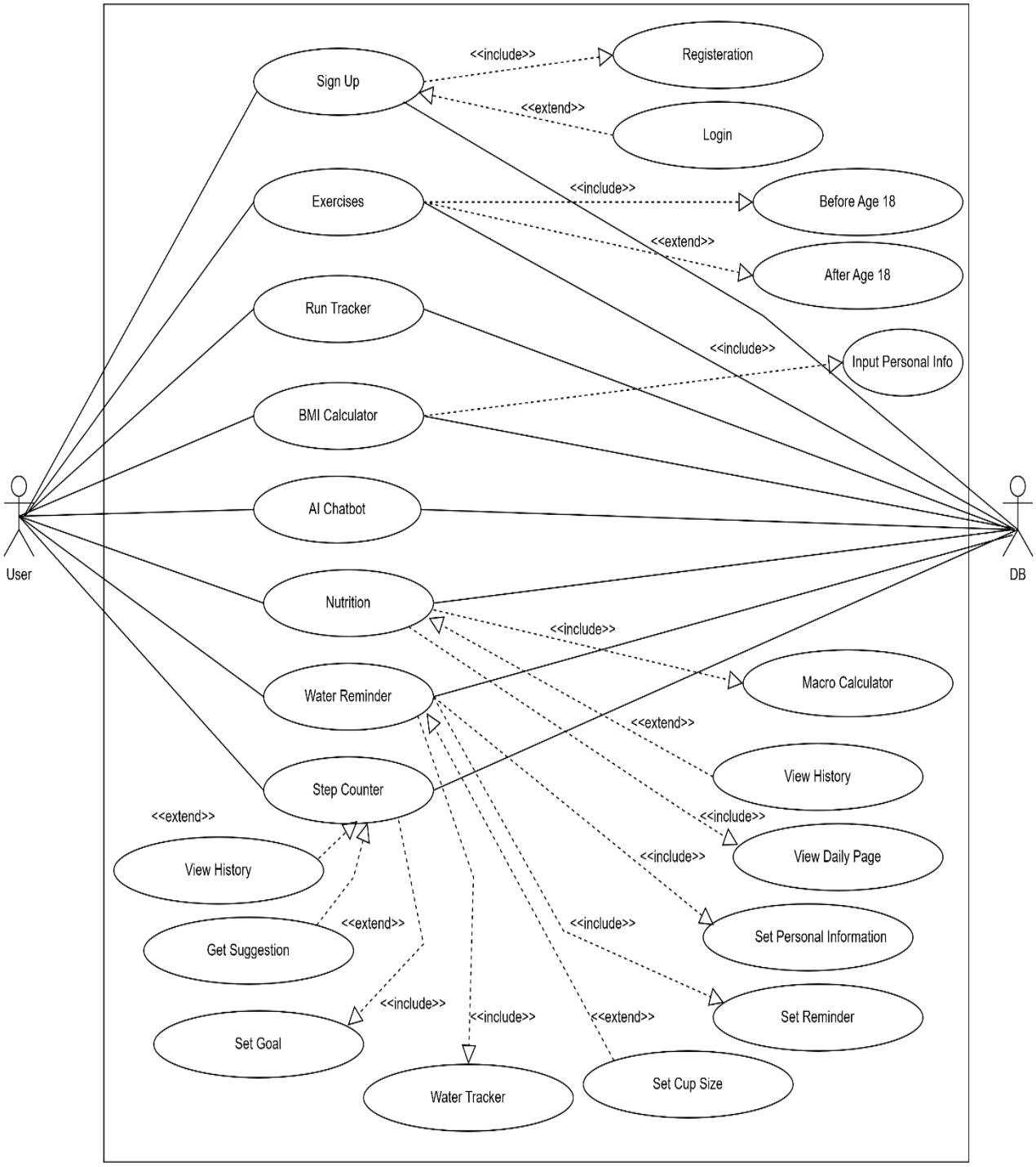

Figure 7

3.2.12. Usecase Description

- **Actors**: Users, sensors, or system services.
- **Preconditions**: Steps required before use.
- **Main Flow**: Primary interaction flow.
- **Alternate Flow**: Deviations from the main flow.
- **Post Conditions:** Expected outcome.

Table 23

Use Case ID:	UC-01
Use Case Name:	Signup
Actors:	**Primary Actor:** User **Secondary Actor:** System
Description:	This use case describes the process by which a user signs up for an account on the system.
Trigger:	The user initiates the signup process by accessing the signup page.
Preconditions:	• The user must have access to the internet. • User has opened the system
Postconditions:	• The user is successfully registered in the system. • The user receives a confirmation email or SMS (if applicable)
Normal Flow:	1. The user navigates to the signup page. 2. The system displays the signup form. 3. The user fills in the required details (e.g., name, email, password, etc.). 4. The user submits the signup form. 5. The system validates the provided information. 6. The system creates a new user account. 7. The system sends a confirmation email or SMS to the user. 8. The system displays a confirmation message to the user.
Alternative Flows:	**AF-1: User Attempts to Sign Up with Existing Email** 1. Steps 1-4 of the Normal Flow. 2. The system checks if the email is already in use. 3. The system displays an error message indicating the email is already registered. 4. The user is prompted to log in or reset their password. **AF-2: User Enters Weak Password** 1. Steps 1-4 of the Normal Flow. 2. The system checks the password strength. 3. The system displays an error message indicating the password does not meet security requirements.

	4. The user is prompted to enter a stronger password and resubmit **AF-3: User Cancels Signup** 1. The user navigates to the signup page. 2. The user decides not to proceed and cancels the signup process. 3. The system redirects the user to the homepage or previous page.
Exceptions:	A system error occurs (e.g., database connection issue). The system displays an error message to the user. The system detects an invalid email format.
Includes:	Data Verification (if email verification is part of the signup process).
Special Requirements:	The password must adhere to security standards (e.g., minimum length, complexity requirements). The system should be able to handle high volumes of signup requests simultaneously
Assumptions:	The user has a basic understanding of how to fill out online forms.
Notes and Issues:	Ensure that error messages do not reveal sensitive information about the system.

Table 24

Use Case ID:	UC-02
Use Case Name:	Login
Actors:	**Primary Actor:** User **Secondary Actor**: System
Description:	This use case describes the process by which a user logs into the system.
Trigger:	The user initiates the login process by accessing the login page or section of the application.
Preconditions:	• The user must have an existing account in the system. • The user must have access valid username and password.
Postconditions:	• The user is successfully authenticated and granted access to their account. • The user's session is started, allowing them to access protected areas of the application.
Normal Flow:	1. The user navigates to the login page. 2. The system displays the login form, prompting the user to enter their username/email and password. 3. The user enters their credentials and submits the form. 4. The system validates the entered credentials against 5. the stored user data.

	6. If the credentials are valid, the system creates a new session for the user. 7. system redirects the user to the homepage or dashboard. 8. The user gains access to the protected areas of the application.
Alternative Flows:	**AF-1: User Forgets Password** 1. Steps 1-2 of the Normal Flow. 2. The user selects the "Forgot Password" option. 3. The system prompts the user to enter their registered email address. 4. The user submits the email address. 5. The system sends a password reset link to the user's email. 6. The user follows the link and resets their password. 7. The user returns to the login page, enters their new password, and logs in successfully. **AF-2: User Chooses to Use Two-Factor Authentication (2FA)** 1. Steps 1-4 of the Normal Flow. 2. The system detects that the user has 2FA enabled. 3. The system prompts the user to enter a verification code sent to their registered device (e.g., via SMS or email). 4. The user enters the verification code and submits it. 5. The system validates the verification code. 6. Steps 5-7 of the Normal Flow
Exceptions:	The user has a basic understanding of how to enter credentials and navigate the login process.
Includes:	• Password Reset (for handling forgotten passwords) • Two-Factor Authentication (for additional security during login)
Special Requirements:	The system must securely store and handle user credentials. The system should provide a user-friendly interface for the login process. The system must validate credentials efficiently and accurately.
Assumptions:	The user has a basic understanding of how to enter credentials and navigate the login process
Notes and Issues:	Ensure compatibility with various devices and browsers. Plan for regular updates to enhance security and address any bugs.

Table 25

Use Case ID:	UC-03
Use Case Name:	Step Counter
Actors:	**Primary Actor:** User **Secondary Actor:** System
Description:	Process for a user to track steps and physical activity using a step counter feature on their device first by giving permission of body sensors.
Trigger:	User accesses the step counter feature on their device.
Preconditions:	• User has a device with step counter functionality (e.g., smartphone, smartwatch). • Step counter application/feature is installed and accessible. • Device has necessary sensors (e.g., accelerometer).
Postconditions:	• Steps are accurately tracked and recorded. • User can view step count and related activity data.
Normal Flow:	1. User opens the step counter application/feature. 2. System initializes the step counter and accesses necessary sensors. 3. User begins physical activity. 4. System continuously tracks steps in real-time. 5. User views current step count in the step counter feature. 6. System displays step count and other related data (e.g., distance traveled, calories burned). 7. And after completion of steps will be stored in database.
Alternative Flows:	**AF-1: User Resets Step Counter** • Steps 1-5 of Normal Flow. • User opts to reset step counter to zero. • System confirms reset action and resets step count to zero, then continues tracking new steps. **AF-2: User Pauses Step Counting** • Steps 1-5 of Normal Flow. • User opts to pause step counting. • System pauses step counting. • User resumes step counting. • System resumes tracking steps from where it paused.
Exceptions:	Sensor Malfunction System Error During Tracking
Includes:	NULL
Special Requirements:	Accurate step tracking using device sensors. Minimized battery consumption during tracking. Background tracking capability when the application is not actively open.

	Clear and intuitive user interfaces for viewing step counts and related data.
Assumptions:	User understands how to use mobile applications and features. Device sensors are functioning correctly and properly calibrated.
Notes and Issues:	Ensure compatibility with various devices and operating systems. Plan for regular updates to improve accuracy, fix bugs, and add new features.

Table 26

Use Case ID:	UC-04
Use Case Name:	BMI Calculator
Actors:	**Primary Actor:** User **Secondary Actor:** System
Description:	Process for a user to calculate their Body Mass Index (BMI) using the system.
Trigger:	User accesses the BMI calculator page
Preconditions:	• User must have access to the internet. • BMI calculator page must be accessible. • User must know their height and weight.
Postconditions:	• User receives their BMI value and health category.
Normal Flow:	1. User navigates to the BMI calculator page. 2. System displays the BMI calculator form. 3. User enters their height and weight into the form. 4. User submits the form. 5. System validates the entered data. 6. System calculates the user's BMI. 7. System displays the BMI value and corresponding health category.
Alternative Flows:	**AF-1: User Enters Invalid Data** • Steps 1-4 of Normal Flow. • System detects invalid data (e.g., non-numeric values, negative numbers). • System displays an error message. • User corrects the data and resubmits the form. **AF-2: User Cancels Calculation** • User navigates to the BMI calculator page. • User cancels the calculation process. • System redirects user to the homepage or previous page.
Exceptions:	System Error During Calculation
Includes:	Input weight and height

Special Requirements:	System should provide accurate BMI calculations. BMI categories (e.g., underweight, normal weight, overweight, obesity) must be clearly defined and displayed.
Assumptions:	User understands height and weight measurements
Notes and Issues:	Print BMI results User-friendliness

Table 27

Use Case ID:	UC-05
Use Case Name:	Exercises
Actors:	**Primary Actor:** User **Secondary Actor:** System
Description:	Process for a user to track their exercises using the system.
Trigger:	User starts exercise tracking on their device.
Preconditions:	• User has a device with exercise tracking (e.g., smartphone, smartwatch). • Exercise tracking app/feature is installed and accessible. • Device has necessary sensors (e.g., GPS, accelerometer).
Postconditions:	• Exercises are accurately tracked and recorded • User can view their exercise data
Normal Flow:	1. User opens exercise tracking app/feature. 2. System shows exercise tracking interface. 3. User selects type of exercise (e.g., running, cycling). 4. User starts exercise session. 5. System tracks exercise using sensors. 6. User finishes exercise and stops tracking. 7. System saves exercise data. 8. System displays exercise summary (duration, distance, pace, calories).
Alternative Flows:	**AF-1: User Pauses and Resumes Tracking** • Steps 1-5 of Normal Flow. • User pauses tracking. • System pauses tracking. • User resumes tracking. • System resumes tracking from where it paused. **AF-2: User Cancels Tracking** • Steps 1-5 of Normal Flow. • User cancels exercise session. • System asks for confirmation to discard data.

	• System discards current exercise data and resets tracking.
Exceptions:	System malfunction
Includes:	NULL
Special Requirements:	Accurate tracking with device sensors. Minimize battery use during tracking.
Assumptions:	User understands basic app usage. Device sensors work correctly and are calibrated.
Notes and Issues:	Add motivational features (goals, reminders, achievements). Ensure compatibility with various devices and systems.

Table 28

Use Case ID:	UC-06
Use Case Name:	Workout plans
Actors:	**Primary Actor:** User **Secondary Actor:** System
Description:	Process for a user to create, view, and follow workout plans using the system.
Trigger:	User opens the workout plans feature on their device.
Preconditions:	• User has a device with workout plan functionality (e.g smartphone, tablet). • Workout plans app/feature is installed and accessible. • User is logged into their account (if needed).
Postconditions:	• Workout plans are created, saved, and accessible. • User can view and follow their workout plans.
Normal Flow:	1. User opens workout plans app/feature. 2. System shows workout plans interface. 3. User selects option to create a new workout plan. 4. User enters workout plan details (e.g., exercises, duration, repetitions). 5. User saves the workout plan. 6. System saves and makes the workout plan available. 7. User selects a saved workout plan to view. 8. System displays the selected workout plan. 9. User follows the workout plan during exercise. 10. System tracks user's progress and updates the plan as needed.
Alternative Flows:	**AF-1: User Edits a Workout Plan** • Steps 1-2 of Normal Flow. • User selects an existing workout plan to edit. • System displays workout plan details. • User makes changes to the plan. • User saves the updated plan.

	• System updates and saves the changes. **AF-2: User Deletes a Workout Plan** • Steps 1-2 of Normal Flow. • User selects an existing workout plan to delete. • System asks for confirmation to delete. • User confirms deletion. • System deletes the workout plan.
Exceptions:	System Error During Plan Creation/Update
Includes:	NULL
Special Requirements:	Supports various exercises and customization options. Intuitive and easy-to-use interface.
Assumptions:	User understands basic workout routines and mobile app usage. System is reliable and accurately tracks data.
Notes and Issues:	Plan regular updates for improved functionality and new features

Table 29

Use Case ID:	UC-07
Use Case Name:	Run Tracker
Actors:	**Primary Actor:** User **Secondary Actor:** System
Description:	Process for a user to track their running activity using the system.
Trigger:	User starts the run tracker feature on their device.
Preconditions:	• User has a device with run tracker functionality (e.g., smartphone, smartwatch). • Run tracker app/feature is installed and accessible. • Device has necessary sensors (e.g., GPS, accelerometer).
Postconditions:	• Running activity is accurately tracked and recorded. • User can view their run data
Normal Flow:	1. User opens the run tracker app/feature. 2. System shows the run tracker interface. 3. User starts the run tracking session. 4. System tracks the run using device sensors (e.g., GPS for distance, accelerometer for steps). 5. User completes the run and stops the tracking. 6. System saves the run data. 7. System displays a summary of the run (e.g., distance, duration, pace, calories burned).
Alternative Flows:	**AF-1: User Pauses and Resumes Run Tracking** • Steps 1-4 of Normal Flow.

	• User pauses the run tracking. • System pauses the tracking. • User resumes the run tracking. • System resumes tracking from where it paused. AF-2: **User Cancels Run Tracking** • Steps 1-4 of Normal Flow. • User cancels the run session. • System asks for confirmation to discard the data. • User confirms cancellation. • System discards current run data and resets tracking.
Exceptions:	Sensor malfunction
Includes:	NULL
Special Requirements:	Accurate tracking using device sensors. Minimize battery consumption during tracking.
Assumptions:	User understands how to use mobile applications. Device sensors are functioning correctly
Notes and Issues:	Ensure compatibility with various devices and operating systems. Plan regular updates for improved accuracy, bug fixes, and new features

Table 30

Use Case ID:	UC-08
Use Case Name:	Water Reminder
Actors:	**Primary Actor:** User **Secondary Actor:** System
Description:	Process for a user to set up reminders to drink water and track their water intake.
Trigger:	User starts the water reminder feature on their device.
Preconditions:	• User has a device with water reminder functionality (e.g., smartphone, smartwatch). • Water reminder app/feature is installed and accessible. • User enter its inputs like weight, working hour, bedtime and wake time. • User allows notification access. • Device has working notification services.
Postconditions:	• Water intake is accurately tracked and recorded. • User receives timely reminders to drink water.
Normal Flow:	• User opens the water reminder app/feature. • System displays the water reminder interface.

	• User sets their daily water intake goal. • User schedules reminders for drinking water. • System sends timely reminders based on the user's schedule. • User logs water intake after each reminder. • System tracks and updates the water intake progress.
Alternative Flows:	• **AF-1: User Adjusts Reminders** • Steps 1-4 of Normal Flow. • User changes the bed time and wake time of water reminders. • System updates the reminder schedule accordingly.
Exceptions:	**Notification delivery issues**: • System fails to send reminders due to device or app notification problems.
Includes:	NULL
Special Requirements:	Minimize battery consumption while running in the background.
Assumptions:	User understands how to interact with mobile applications.
Notes and Issues:	Regular updates for feature improvements, bug fixes, and compatibility with new device models.

Table 31

Use Case ID:	UC-09
Use Case Name:	Nutrition Tracker
Actors:	**Primary Actor:** User **Secondary Actor:** System
Description:	Process for a user to track daily food intake, manage nutrition goals, and monitor calories and nutrients using the system.
Trigger:	User accesses the nutrition tracker feature on their device.
Preconditions:	• User has a device with the nutrition tracking functionality. • Nutrition tracking app/feature is installed and accessible. • User provides relevant information (e.g., age, weight, dietary preferences). • Device has an internet connection for accessing a food database (if required).
Postconditions:	• User opens the nutrition tracker app/feature. • System displays the nutrition tracking interface.

	• User sets a daily calorie or nutrition goal (e.g., calorie limit, macro targets like protein, carbs, fats). • User logs food intake by selecting items from the database or manually entering details. • System calculates the nutritional information of the logged food (e.g., calories, macros, vitamins). • System updates the user's daily nutrition progress in real time. • User views their current progress toward daily nutrition goals (e.g., calories consumed, remaining, balance of macros). • System provides daily/weekly/monthly reports on nutrition intake.
Normal Flow:	• User opens the water reminder app/feature. • System displays the water reminder interface. • User sets their daily water intake goal. • User schedules reminders for drinking water. • System sends timely reminders based on the user's schedule. • User logs water intake after each reminder. • System tracks and updates the water intake progress.

Alternative Flows:	• **AF-1: User Modifies Nutrition Goals**
	  • Steps 1-4 of Normal Flow.
	  • User adjusts their daily or long-term nutrition goals.
	  • System recalculates future progress based on the new goal.
	• **AF-2: User Scans Food Label for Quick Entry**
	  • Steps 1-4 of Normal Flow.
	  • User uses the system to scan a food item's barcode.
	  • System pulls data from the food database and logs it for the user.
	• **AF-3: User Deletes Food Entry**
	  • Steps 1-4 of Normal Flow.
	  • User deletes a previously entered food item from their daily log.
	  • System recalculates the user's daily nutritional progress.

Exceptions:	**Database connection issues**: • System fails to retrieve nutritional information for specific foods if an internet connection is unavailable
Includes:	NULL
Special Requirements:	• User-friendly interface for quick entry and review of food items.

	• Personalized nutrition recommendations based on the user's health goals (e.g., weight loss, muscle gain).
Assumptions:	• User understands basic nutrition concepts and goals. • Internet connection is available for access to food databases.
Notes and Issues:	• Ensure that the food database is regularly updated with new and relevant items. • Allow users to input custom recipes and foods for more accurate tracking. • Regular updates to improve accuracy and user experience.

Chapter 4: Third Deliverable For Object Oriented Approach

4.1. Introduction:

This chapter focuses on the detailed design aspects of the fit track app using object-oriented principles. It includes diagrams and models to represent the app's dynamic and static behaviors, inter-component interactions, and data organization.

Following artifacts must be included in the 3rd deliverable.

1. Domain Model
2. System Sequence Diagram
3. Sequence Diagram
4. Collaboration Diagram
5. Operation Contracts
6. Design Class Diagram
7. State Transition Diagram
8. Data Model

4.2. Domain Model

The domain model represents the conceptual structure of the system, identifying key entities and their relationships.

Entities and Relationships:

- **User**: Tracks personal data such as name, age, gender, and goals.
- **Step Counter**: Logs steps, distance, and calories.
- **Nutrition Tracker**: Records meals and macronutrients.
- **Water Reminder**: Manages water intake schedules.
- **BMI Calculator**: Calculates BMI based on user input.
- **Run Tracker**: Tracks running activities using GPS.
- Relationships are established using associations (e.g., a User "logs" steps or "tracks" nutrition).

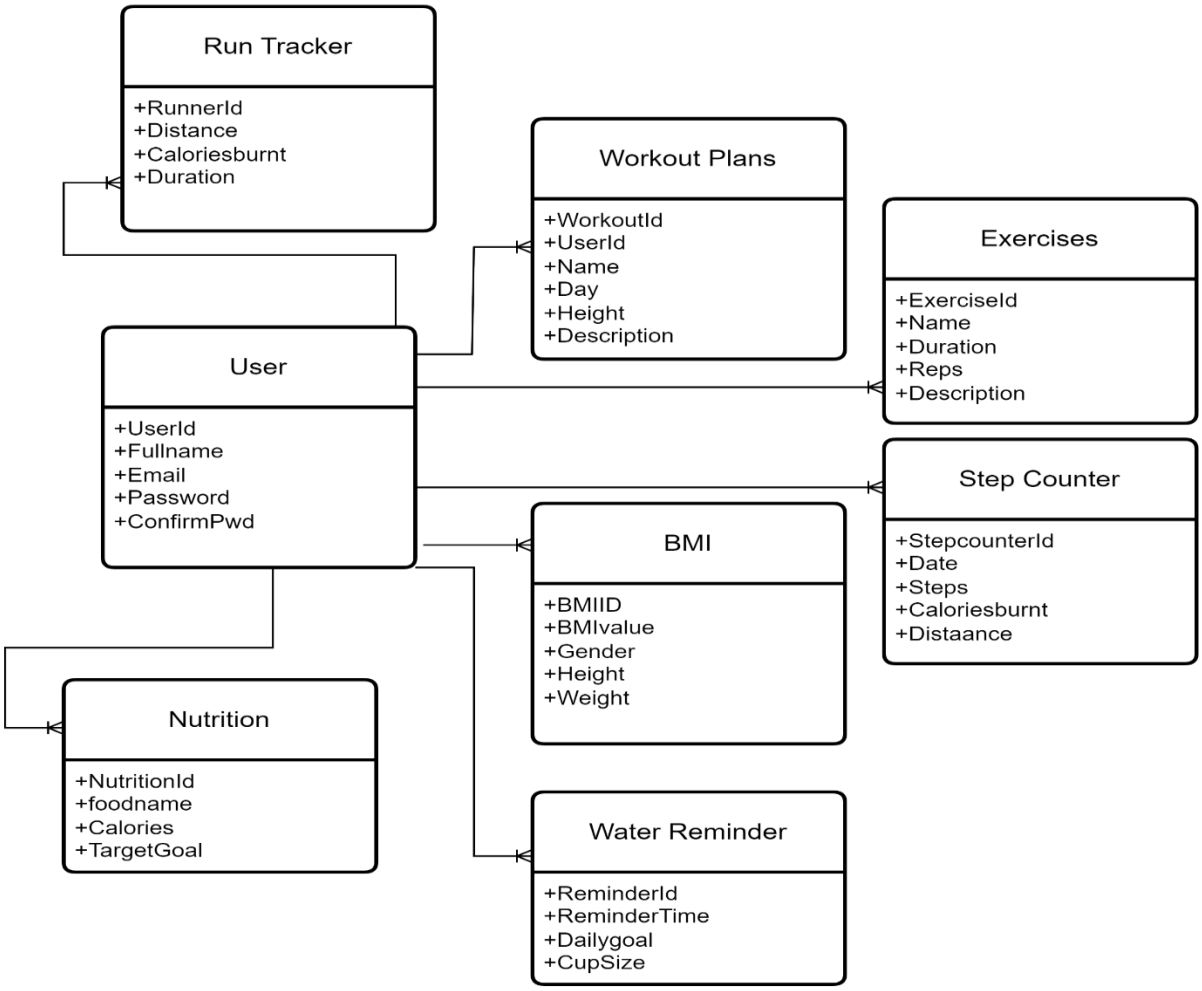

Figure 8

4.3. System Sequence Diagram

The system sequence diagram (SSD) highlights interactions between the system and external actors. It illustrates the sequence of messages exchanged between the actor and the system to accomplish a particular usecase and scenario.System sequence diagrams are useful for understanding the external interactions of a system , identifying the inputs and outputs involved , modeling the flow of control during the execution of usecase.

Example SSD for Step Counting:

1. **Actor**: User initiates the "Start Step Counter" action.
2. **System**: Activates sensor and logs steps.
3. **Actor**: Requests progress.
4. **System**: Displays steps, distance, and calories burned.

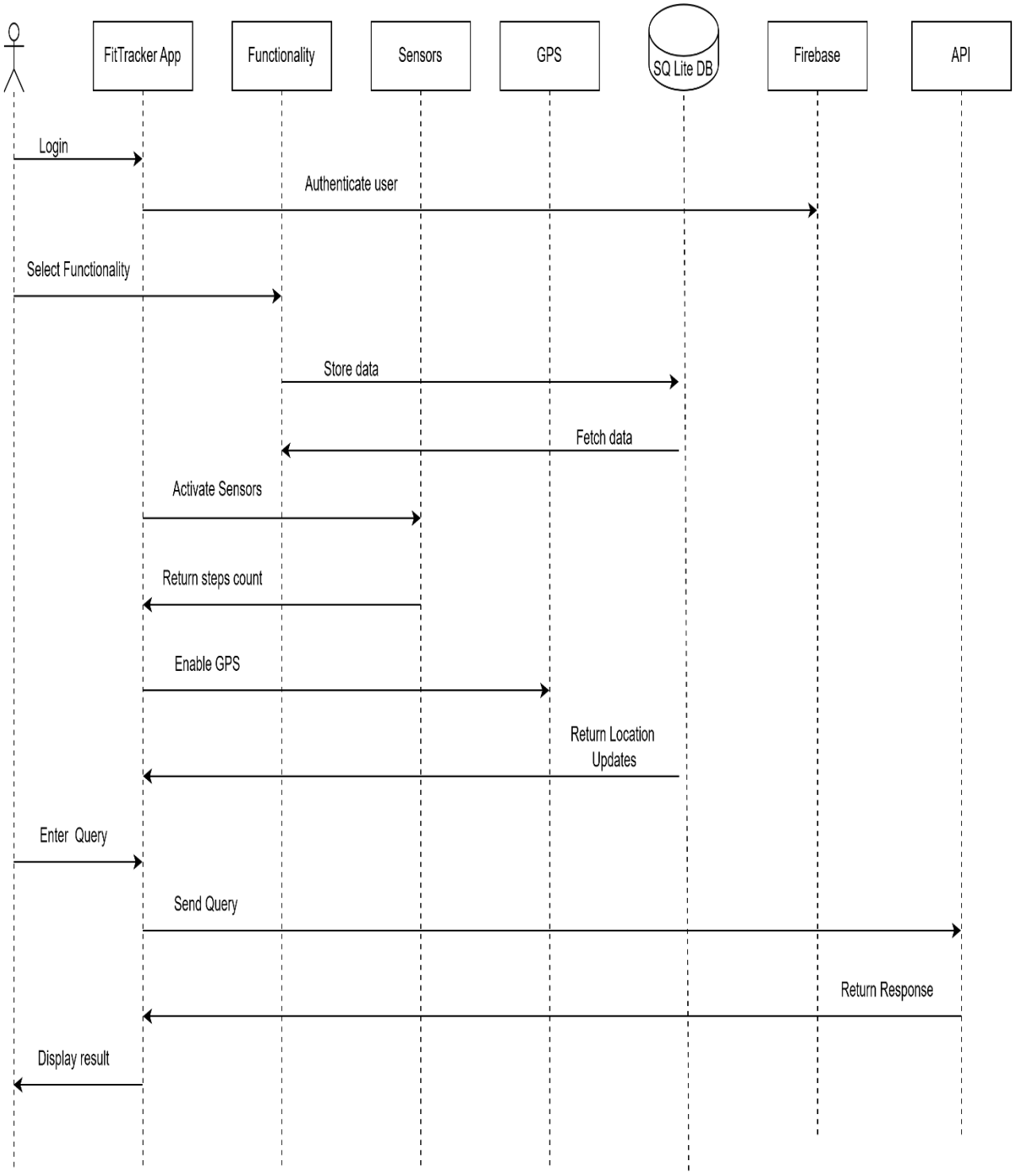

Figure 9

4.4. Sequence Diagram

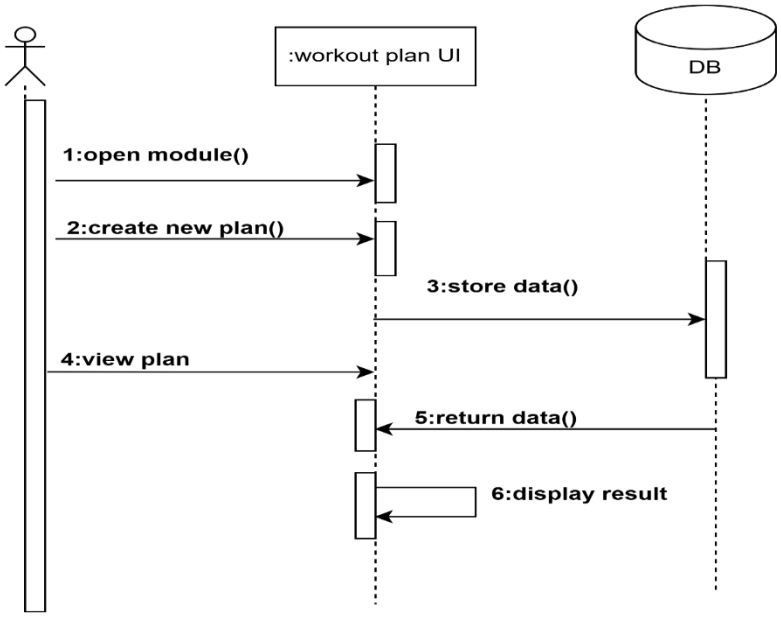

Figure 10 : Workout Plans

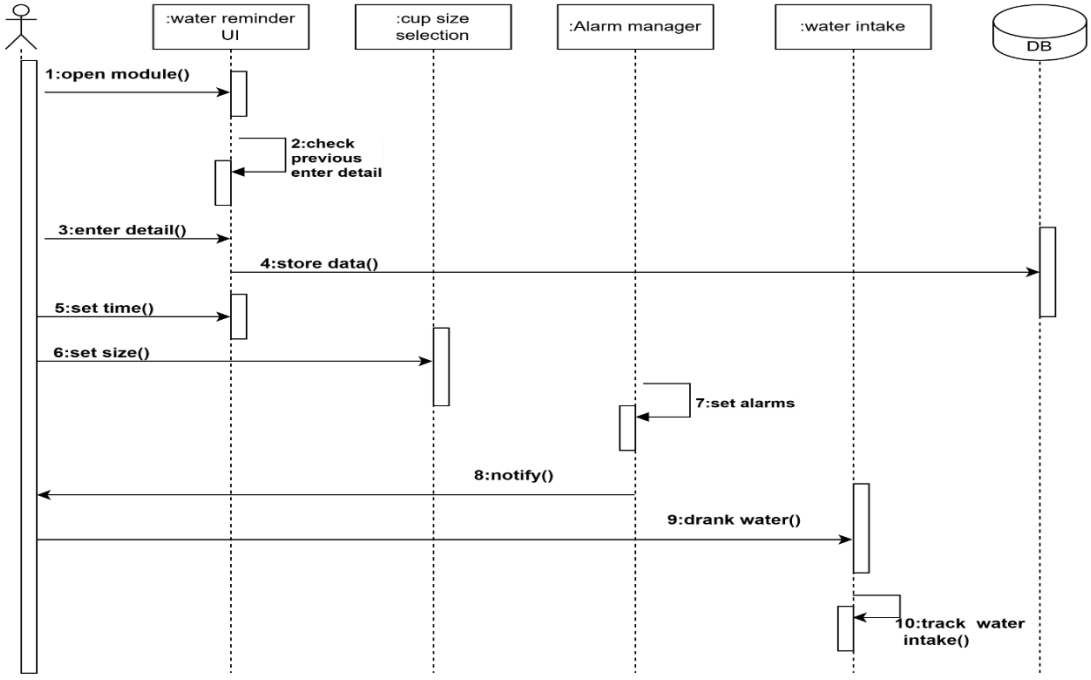

Figure 11 : Water Reminder

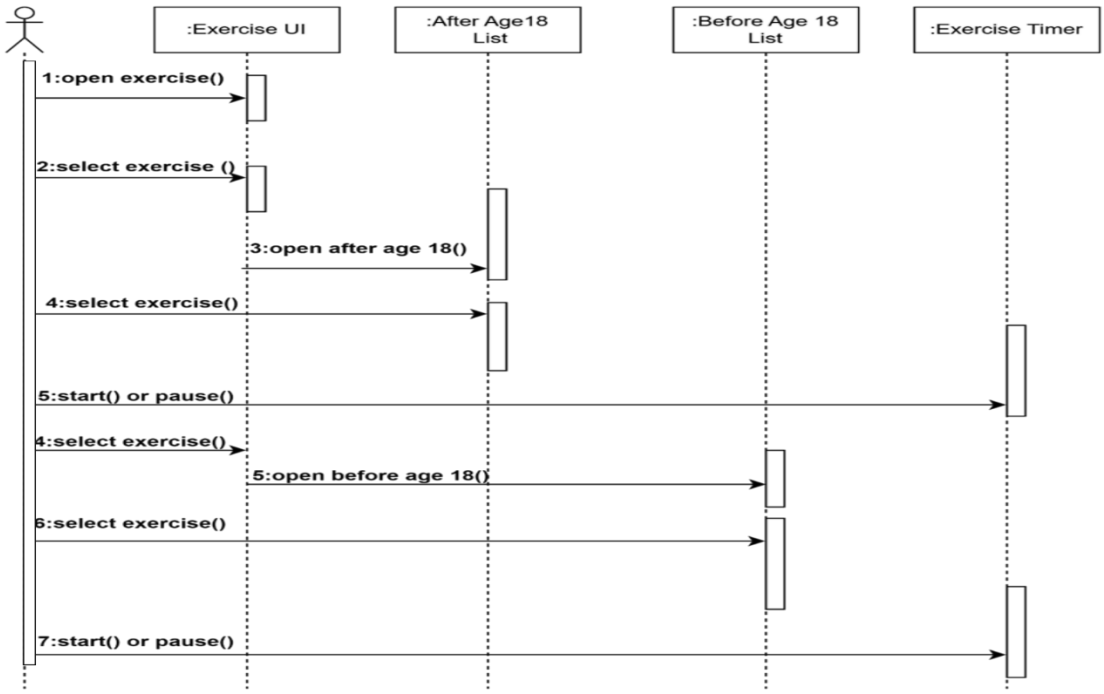

Figure 12: Exercises

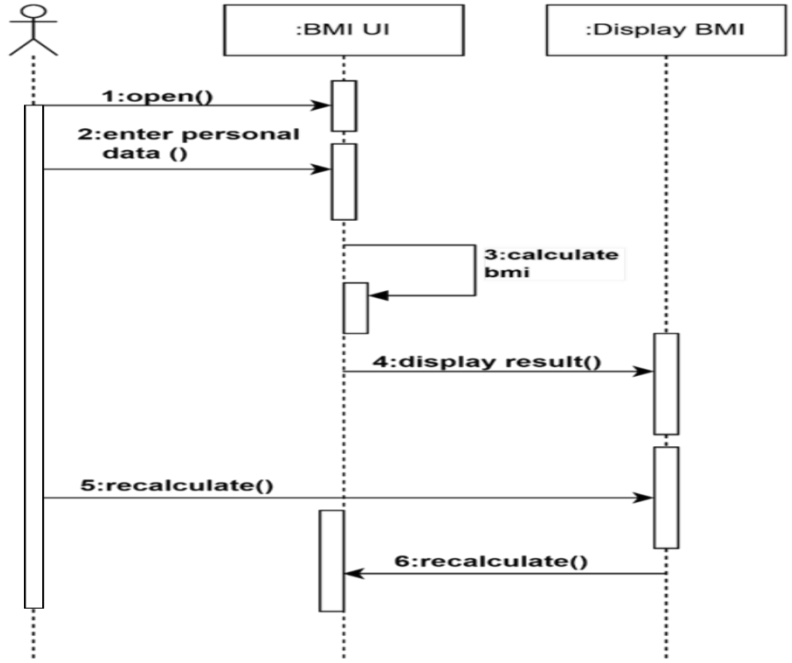

Figure 13 : BMI Calculator

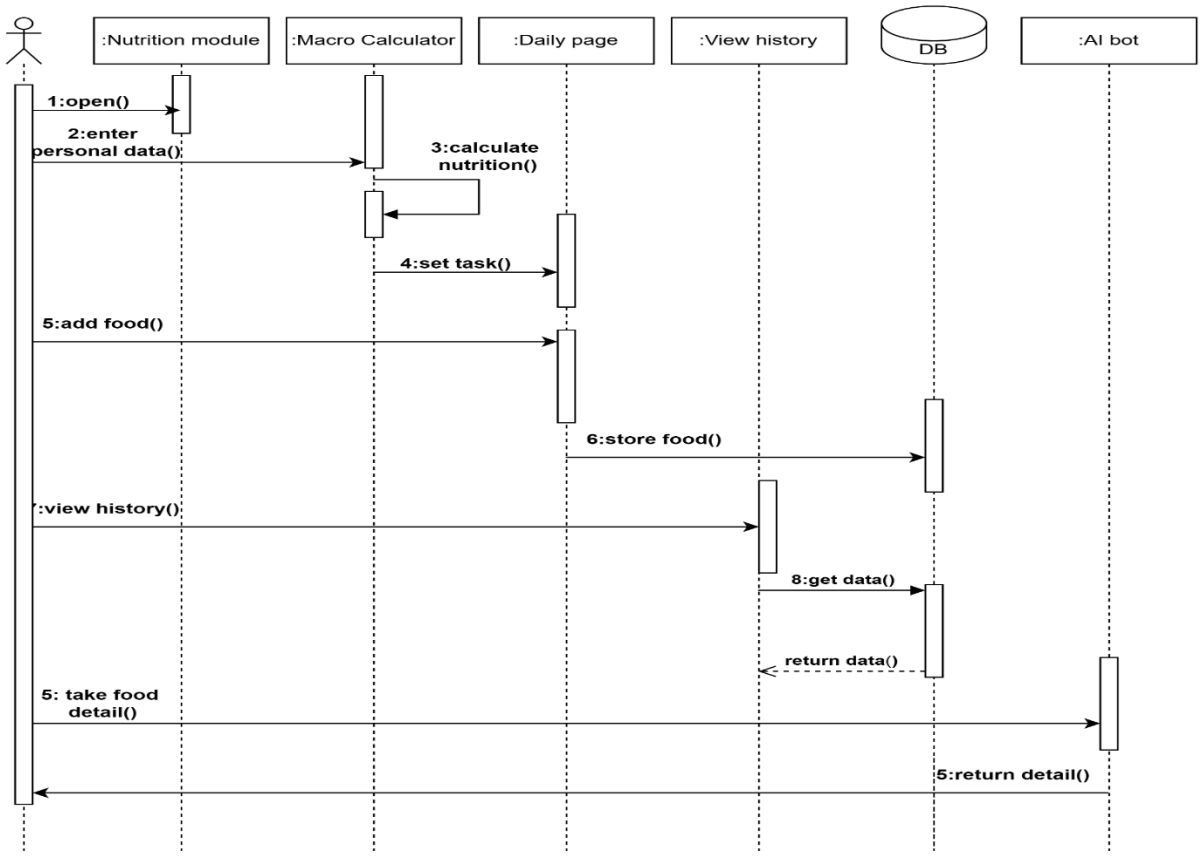

Figure 14 : Nutrition Tracking

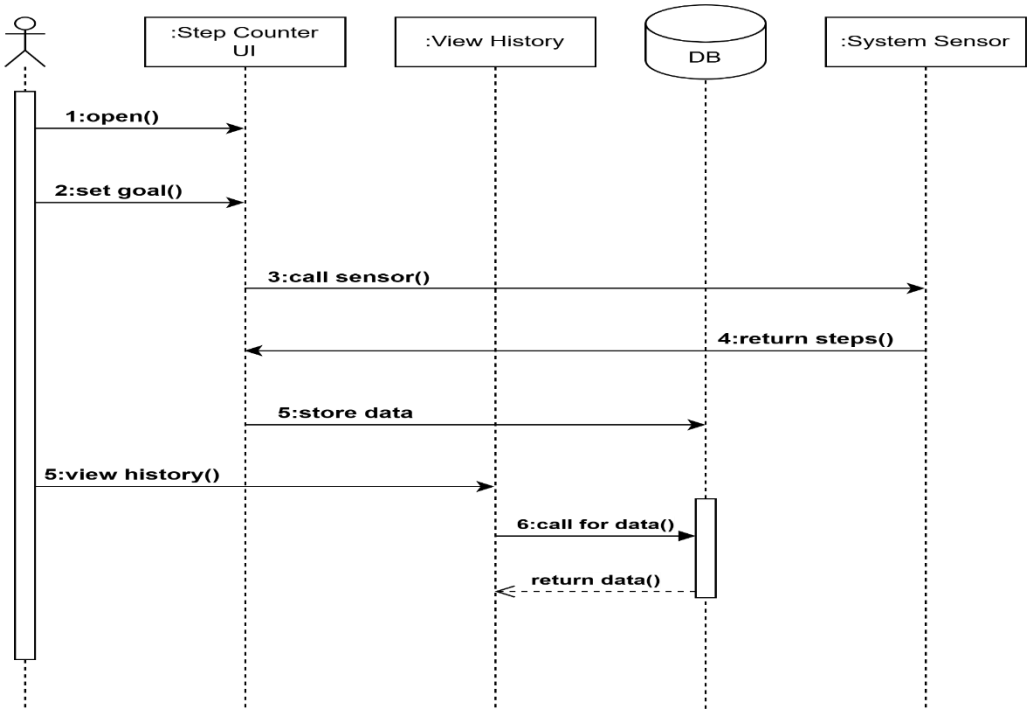

Figure 15 : Step Counter

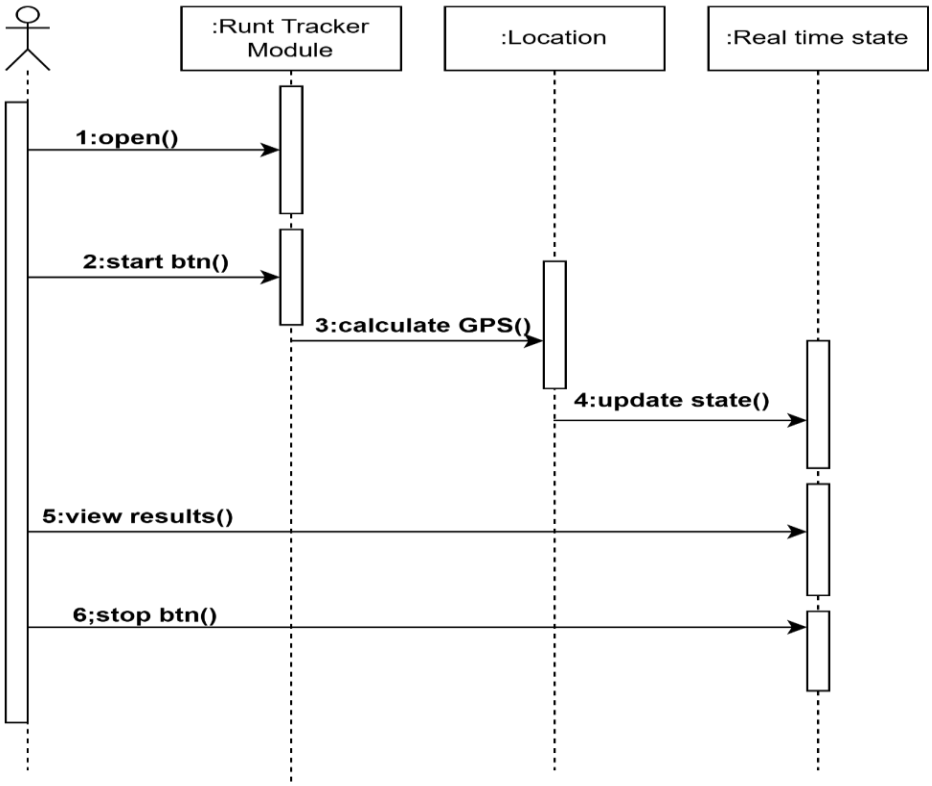

Figure 16 : Run Tracker

4.5. Collaboration Diagram

4.5.1. Contents of Collaboration Diagrams

A collaboration diagram represents interactions among objects with a focus on structural organization.

Key Components:

- Objects (e.g., User, Database).
- Links between objects (e.g., User interacts with StepCounterService).
- Messages exchanged (e.g., "Log Steps", "Fetch Summary").

A collaboration diagram that describes part of the flow of events of the use case Receive Deposit Item in the Recycling-Machine System.

4.5.2. Constructs of Collaboration Diagram:

- **Objects**: Represent entities participating in interactions.
- **Links**: Show connections between objects.
- **Messages**: Indicate the order and type of communication.

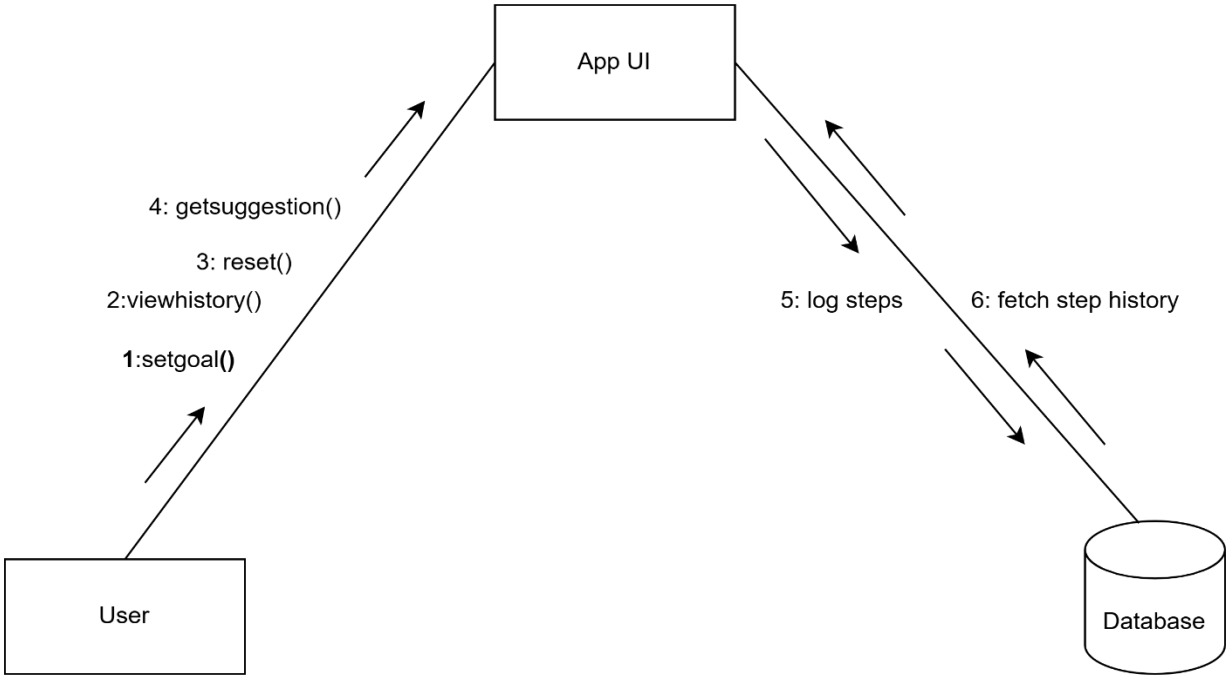

Figure 17

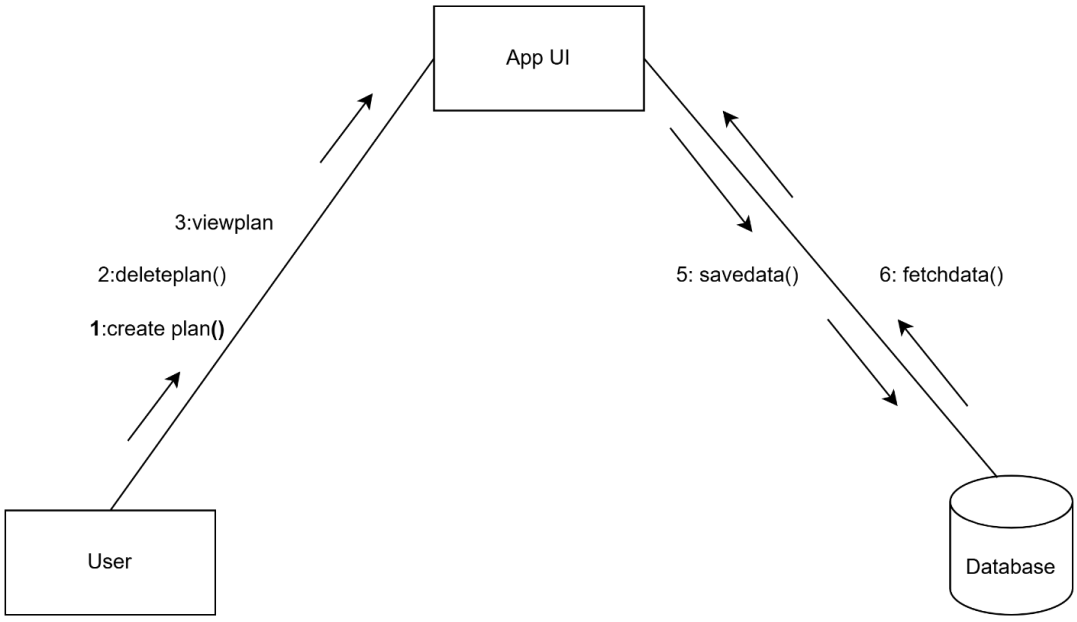

Figure 18

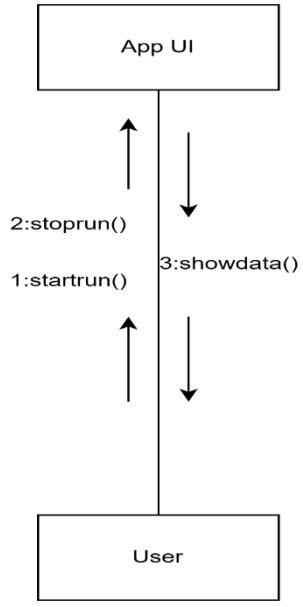

Figure 19

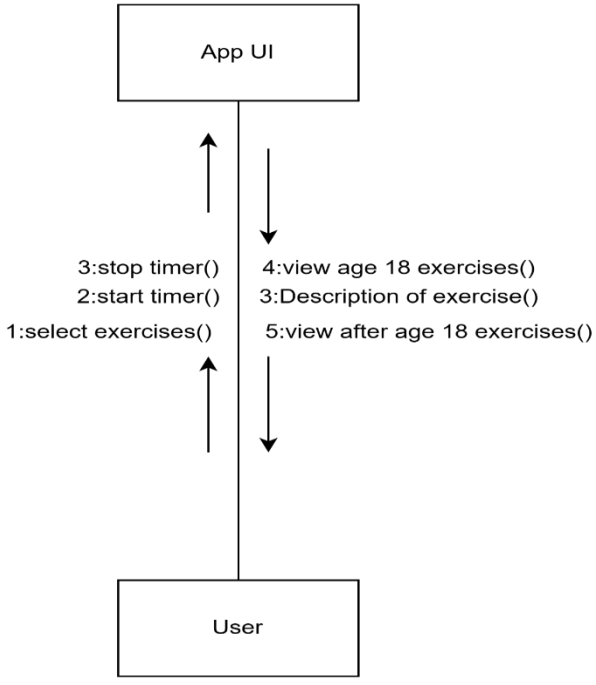

Figure 20

4.6. Operation Contracts

A UML Operation contract identifies system state changes when an operation happens. Operational contracts specify the behavior of system operations, focusing on their preconditions, postconditions, and state changes. They help ensure that each operation behaves as expected and modifies the system state appropriately.

Operational Contract for Step Counter

Operation Name: `startStepCounter()`

- **Preconditions**:
  - The user is logged in.
  - Device sensors are enabled.
- **Postconditions**:
  - The step counter is activated.
  - Steps, distance, and calories are initialized to zero for the session.

Operation Name: `getStepProgress()`

- **Preconditions**:
  - Step counting is active.
- **Postconditions**:
  - Returns the current step count, distance covered, and calories burned.

Operational Contract for Water Reminder

Operation Name: `setWaterReminder(time)`

- **Preconditions**:
  - The user is logged in.
  - The provided time is valid.
- **Postconditions**:
  - A reminder is scheduled for the specified time.

Operation Name: `sendReminderNotification()`

- **Preconditions**:
  - A reminder is scheduled.
  - The current time matches the reminder time.
- **Postconditions**:
  - A notification is sent to the user.

Operational Contract for Nutrition Tracker

Operation Name: `addMeal(foodName, calories, macros)`

- **Preconditions**:
  - The user is logged in.
  - Food name and calorie values are valid.

- **Postconditions**:
    - The meal is added to the database.
    - The user's daily nutrition summary is updated.

Operation Name: `getDailySummary()`

- **Preconditions**:
    - At least one meal has been logged for the day.
- **Postconditions**:
    - Returns the list of logged meals and totals for calories and macros.

Operational Contract for Run Tracker

Operation Name: `startRunTracker()`

- **Preconditions**:
    - GPS permissions are granted.
    - The user is logged in.
- **Postconditions**:
    - The GPS tracker is activated.
    - Distance and speed are initialized to zero.

Operation Name: `getRunStats()`

- **Preconditions**:
    - The run tracker is active.
- **Postconditions**:
    - Returns current distance, duration, and average speed.

Operational Contract for BMI Calculator

Operation Name: `calculateBMI(height, weight)`

- **Preconditions**:
    - Height and weight values are valid and non-zero.
- **Postconditions**:
    - The BMI value is calculated and returned.
    - The BMI category (e.g., underweight, normal, overweight) is determined.

Operational Contract for Profile Management

Operation Name: `updateUserProfile(name, age, gender)`

- **Preconditions**:
    - The user is logged in.
    - Name, age, and gender values are valid.
- **Postconditions**:
    - User profile data is updated in the database.

Operation Name: `getUserProfile()`

- **Preconditions**:
    - The user is logged in.
- **Postconditions**:
    - Returns the user’s profile data, including name, age, gender, and goals.

4.7. Design Class Diagram

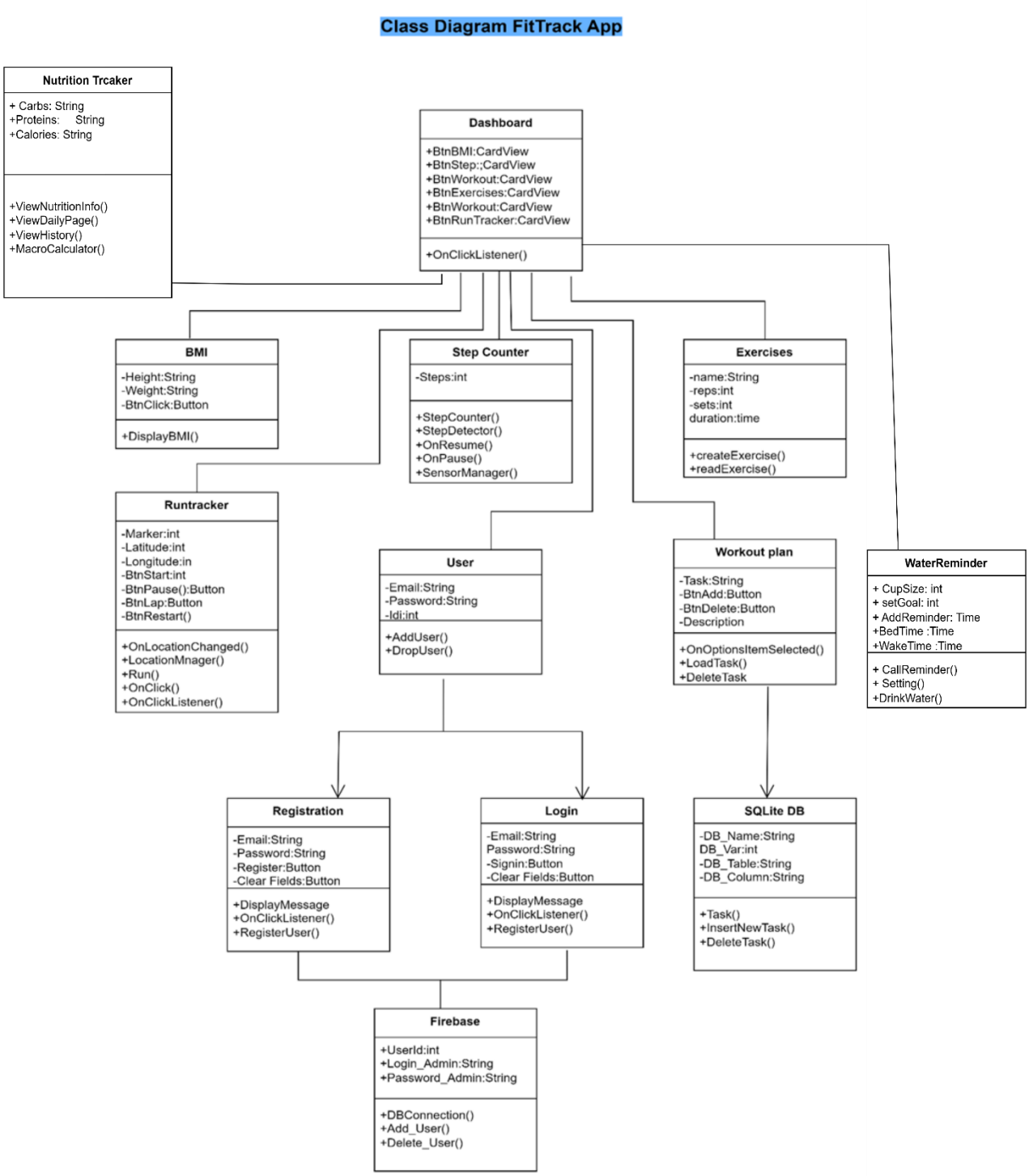

Figure 21: Class Diagram

4.8. State chart diagram

- The state chart diagram represents the states of a system or component and transitions triggered by events.

  A state transition diagram, also known as a state machine diagram or state chart diagram, is a type of behavioral diagram in UML (Unified Modeling Language) that depicts the various states an object or system can be in and the transitions between those states.
- State transition diagrams are useful for modeling the dynamic behavior of systems and objects that exhibit different states and respond to events or conditions by transitioning from one state to another.
- They provide a visual representation of the states, events, and transitions that occur within a system or object, helping to understand the system's behavior over time.

Key elements of a state transition diagram include:

1. **States:**
   - Represented by rectangles, states represent the different conditions or modes in which an object or system can exist. Each state has a name that describes the condition or behavior associated with it.
2. **Transitions:**
   - Represented by arrows, transitions depict the movement from one state to another in response to an event or condition. Transitions are triggered by events and can have associated actions or conditions that need to be met for the transition to occur.
3. **Events:**
   - Represented by labeled arrows or small circles, events trigger transitions between states. They are external stimuli or triggers that cause the system to change its state.

4.Actions:

- Represented by labeled actions associated with transitions, actions describe the activities or behaviors that are performed when a transition occurs.

5. Initial and Final States:

- The initial state indicates the starting point of the system or object's behavior, while the final state represents the end or termination of the system's behavior.

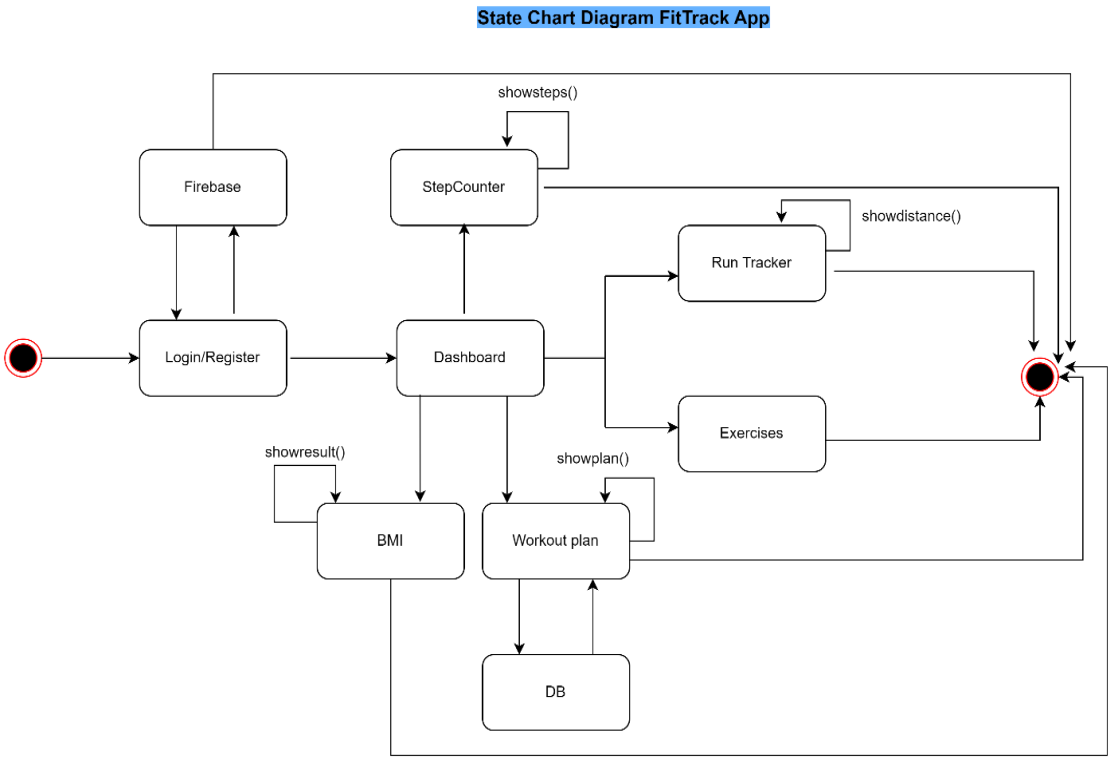

Figure 22

4.9. Data Model

The data model describes how information is stored and organized within the system. Data model for FitTrack app is:

1. **User**
   - **Attributes**:
     - UserID (PK)
     - Username
     - Email
     - Password
     - ConfirmPwd
2. **BMI**
   - **Attributes**:
     - BMIID (PK
     - UserID (FK)
     - Gender
     - Height
     - Weight
     - BMIValue

3. **StepCounter**
    - **Attributes**:
        - StepCounterID (PK)
        - UserID (FK)
        - Caloriesburnt
        - Distance
        - Steps
4. **RunTracker**
    - **Attributes**:
        - RunTrackerID (PK)
        - UserID (FK)
        - Date
        - Distance
        - Duration
        - CaloriesBurned
5. **WorkoutPlan**
    - **Attributes**:
        - WorkoutPlanID (PK)
        - UserID (FK)
        - Name
        - Description
6. **Exercise**
    - **Attributes**:
        - ExerciseID (PK)
        - WorkoutPlanID (FK)
        - Name
        - Reps
        - Set
7. **Water Reminder**
    - **Attributes**:
        - ReminderID(PK)
        - UserID(FK)
        - ReminderTime
        - DailyGoal
        - Cupsize
8. **Nutrition**
    - **Attributes**:
        - NutritionID(PK)
        - UserID(FK)
        - FoodName
        - DailyGoal
        - Calories

Table 32: **User**

Column	Data Type	Description
UserID	INT	Unique identifier for each user (PK)
Username	VARCHAR	The username of the user
Email	VARCHAR	The email address of the user
Password	VARCHAR	The password for the user account
Height	FLOAT	The height of the user
Weight	FLOAT	The weight of the user

Table 33: BMI Calculator

Column	Data Type	Description
BMIID	INT	Unique identifier for each BMI record (PK)
UserID	INT	Foreign key linking to the User entity (FK)
Date	DATE	The date when the BMI was recorded
BMIvalue	FLOAT	The BMI value calculated

Table 34: Step Counter

Column	Data Type	Description
StepCounterID	INT	Unique identifier for each step count record (PK)
UserID	INT	Foreign key linking to the User entity (FK)
Date	DATE	The date when steps were counted
Steps	FLOAT	The number of steps counted

Table 35: Run Tracker

Column	Data Type	Description
RunTrackerID	INT	Unique identifier for each run tracking record (PK)
UserID	INT	Foreign key linking to the User entity (FK)
Date	DATE	The date when the run was recorded
Distance	FLOAT	The distance covered during the run
Duration	TIME	The duration of the run
CaloriesBurned	FLOAT	The number of calories burned during the run

Table 36: Workout Plan

Column	Data Type	Description
WorkoutPlanID	INT	Unique identifier for each workout plan (PK)
UserID	INT	Foreign key linking to the User entity (FK)
Name	VARCHAR	The name of the workout plan
Description	TEXT	A description of the workout plan

Table 37: Exercises

Column	Data Type	Description
ExerciseID	INT	Unique identifier for each exercise (PK)
WorkoutPlanID	INT	Foreign key linking to the Workout Plan entity (FK)
Name	VARCHAR	The name of the exercise
Reps	INT	The number of repetitions for the exercise
Sets	INT	The number of sets for the exercise
Duration	TIME	The duration of the exercise

Table 38: Water Reminder

Column	Data Type	Description
ReminderID	INT	Unique identifier for each water reminder (PK)
UserID	INT	Foreign key linking to the Workout Plan entity (FK)
ReminderTime	TIME	The time when the water reminder is triggered
DailyGoal	INT	The daily water intake goal in milliliters (ml)
Cupsize	VARCHAR	The size of the cup used (e.g., Small, Medium, Large)

Table 39: Nutrition

Column	Data Type	Description
NutritionID	INT	Unique identifier for each nutrition record (PK)
UserID	INT	Foreign key linking to the User entity (FK)
FoodName	VARCHAR	The name of the food item
DailyGoal	INT	The daily calorie intake goal in kilocalories(kcal)
Calories	INT	The calories contained in the food item(kcal)

Relationships:

1. **User and BMI**:
    - **Relationship**: One-to-Many
    - **Explanation**: A user can have multiple BMI records, but each BMI record is associated with only one user.
    - **Example**: User A can have BMI records for different dates (e.g., January 1, February 1).
2. **User and StepCounter**:
    - **Relationship**: One-to-Many
    - **Explanation**: A user can have multiple step count records, but each step count record is associated with only one user.

- **Example**: User A can have step counts recorded for multiple days.

3. **User and RunTracker**:
   - **Relationship**: One-to-Many
   - **Explanation**: A user can have multiple run tracking records, but each run tracking record is associated with only one user.
   - **Example**: User A can have multiple runs recorded with different distances and durations.
4. **User and WorkoutPlan**:
   - **Relationship**: One-to-Many
   - **Explanation**: A user can create multiple workout plans, but each workout plan is associated with only one user.
   - **Example**: User A can create different workout plans for different fitness goals.
5. **WorkoutPlan and Exercise**:
   - **Relationship**: One-to-Many
   - **Explanation**: A workout plan can include multiple exercises, but each exercise is associated with only one workout plan.
   - **Example**: A workout plan named "Full Body Workout" can include exercises like "Push-ups", "Squats", and "Plank".
6. **Water Reminder and User**:
   - **Relationship**: One-to-Many
   - **Explanation**: A single use**r** can set multiple Water Reminders, but each Water Reminder is associated with only one user. This reflects the fact that users can have different reminders throughout the day to drink water, but each reminder belongs to a specific user.
   - **Example**: A user named **John** can set multiple reminders such as **8 AM Water Reminder"**, **"1 PM Water Reminder"**, and **"6 PM Water Reminder"**. Each of these reminders is linked to John's profile, and no other user can access or manage them.
7. **Nutrition and User**:
   - **Relationship**: One-to-Many
   - **Explanation**: A single use**r** can set multiple Nutrition, but each Nutrition is associated with only one User. This indicates that users can track different meals or food items, but each food log is tied to the user who logged it.

Chapter 5: 2nd & 3rd Deliverable For structured Approach

5.1. Introduction:

Analysis & Design Model for structured approach must contain following artifacts:

1. Entity Relationship Diagram
2. Architecture Design
3. Component Level Design

5.2. Entity Relationship Diagram:

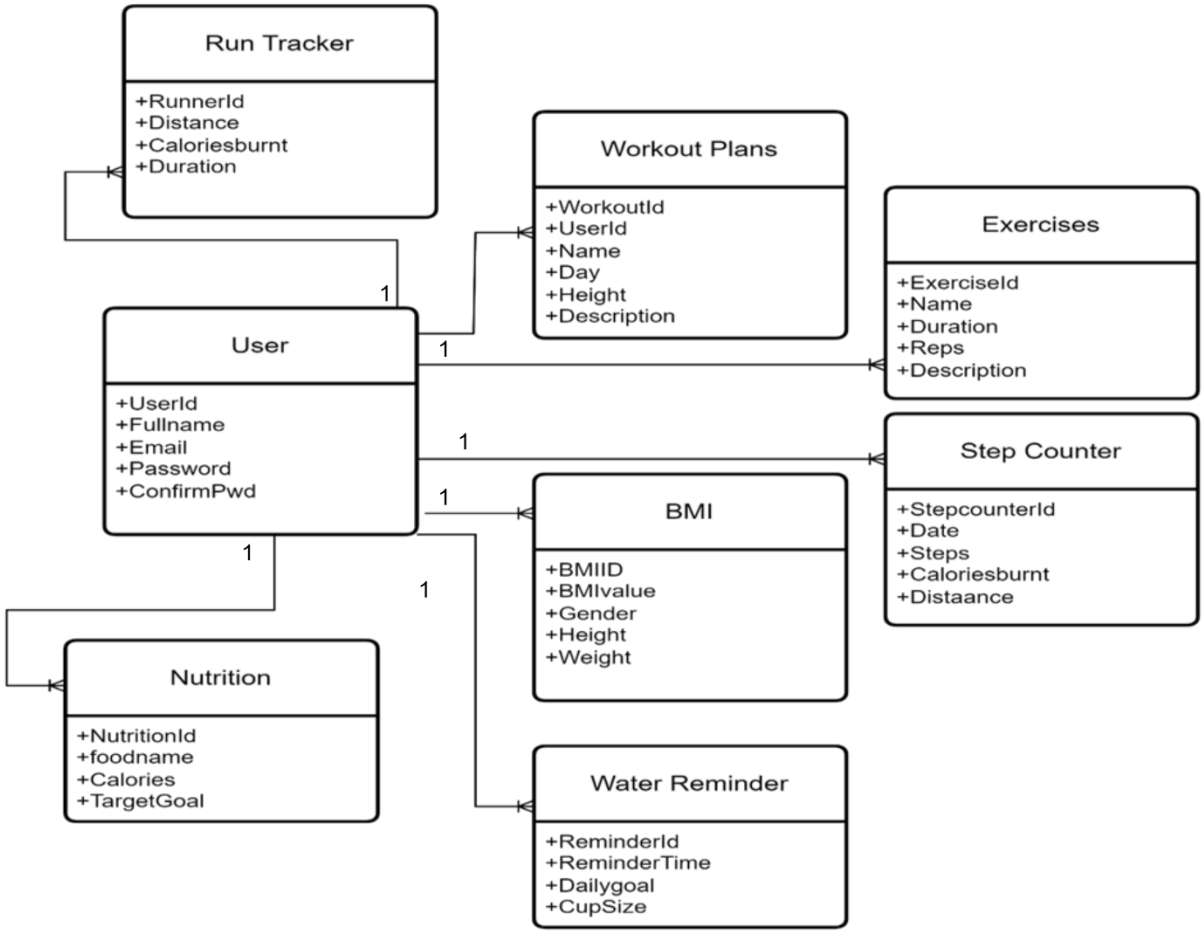

Figure 23

5.5. Architectural design

Data flow diagram (Functional Model)

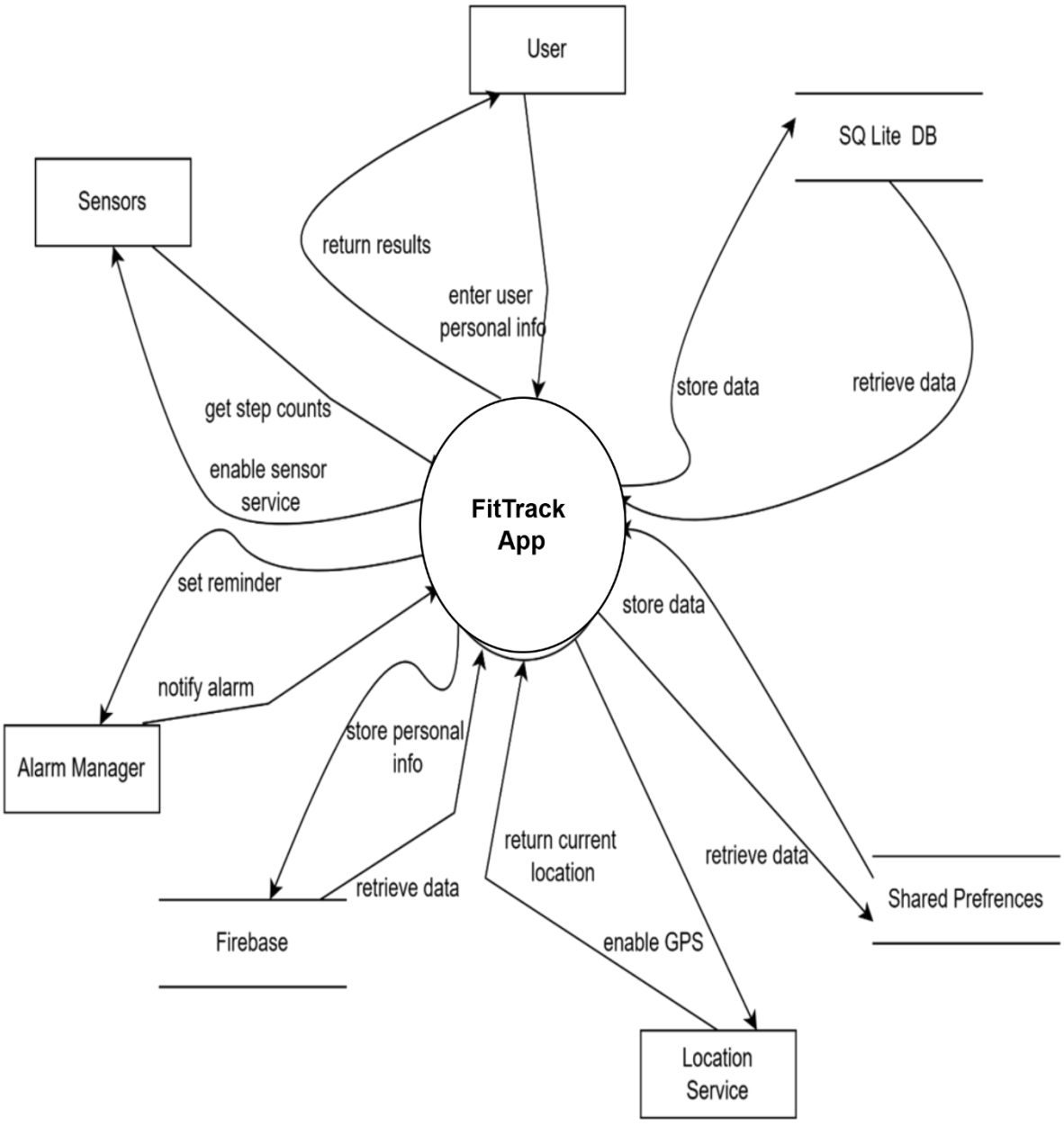

Figure 24 : Level 0 DFD

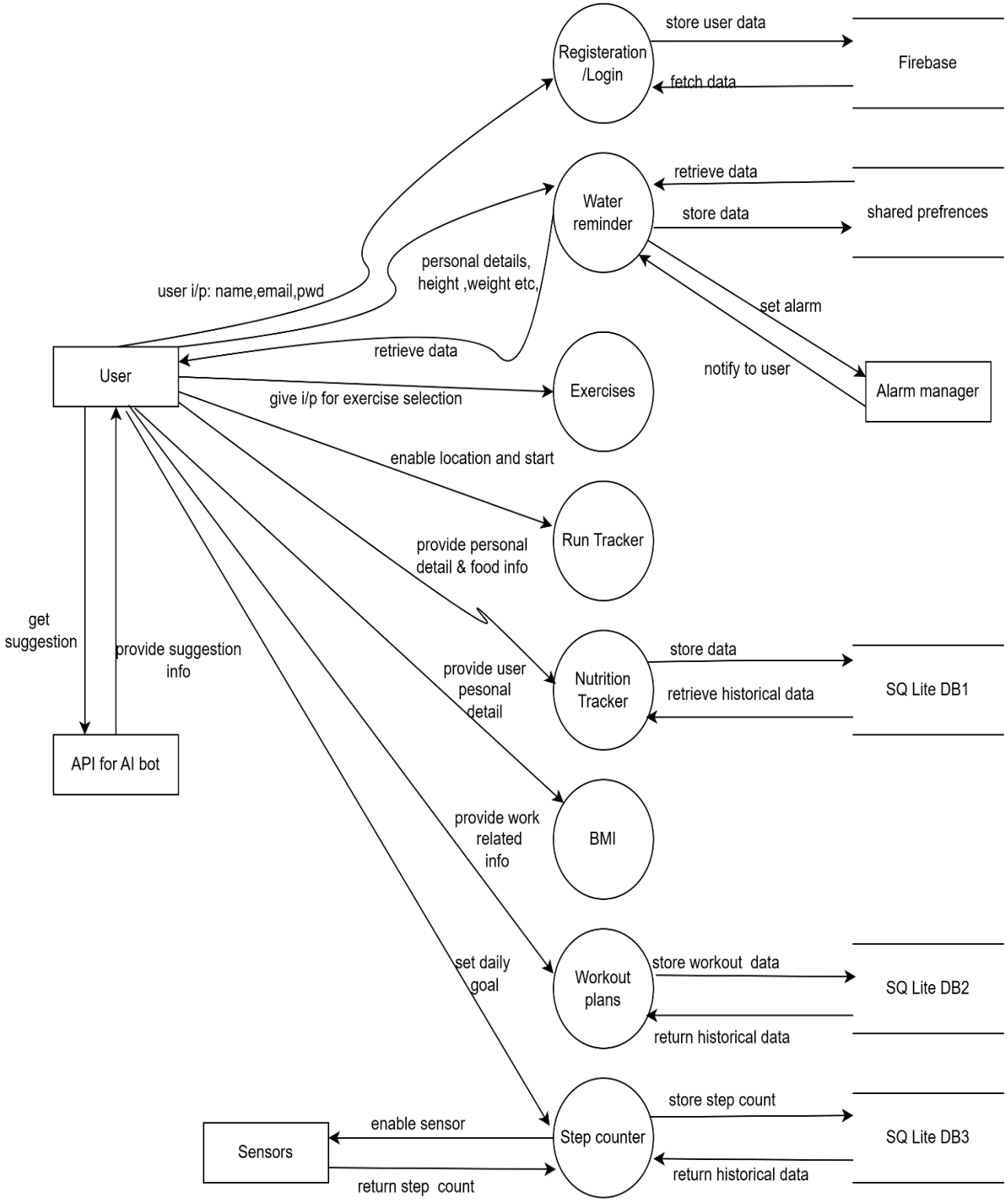

Figure 25 :Level 1 DFD

5.6. Component Level Design

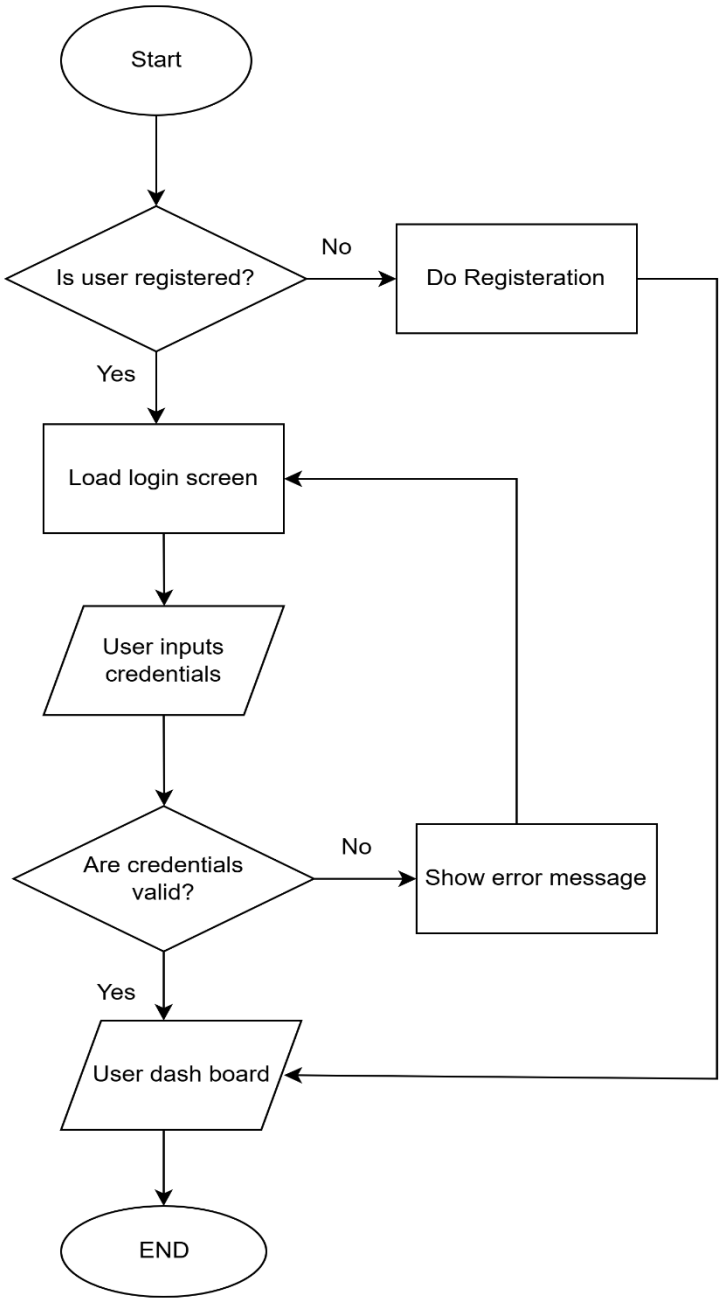

Figure 26: Registeration Algorithm

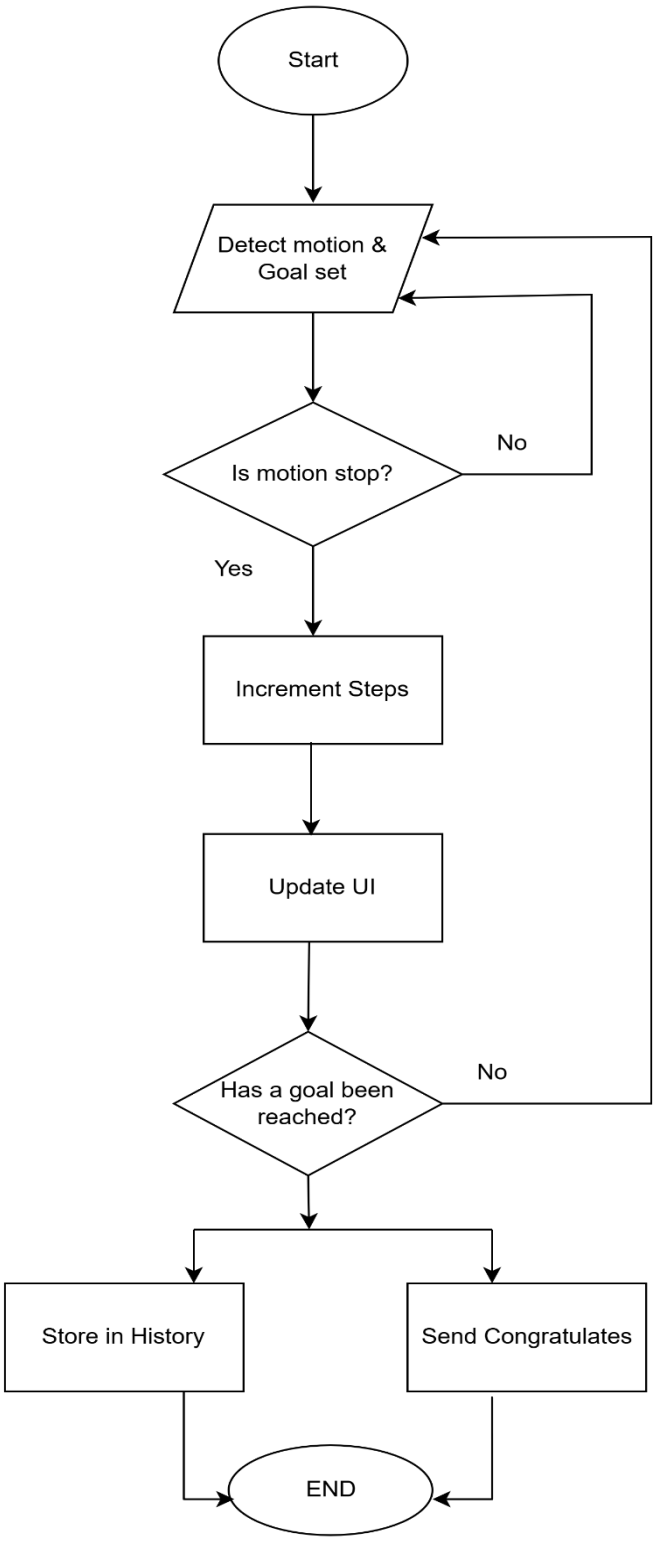

Figure 27: Step Counter Algorithm

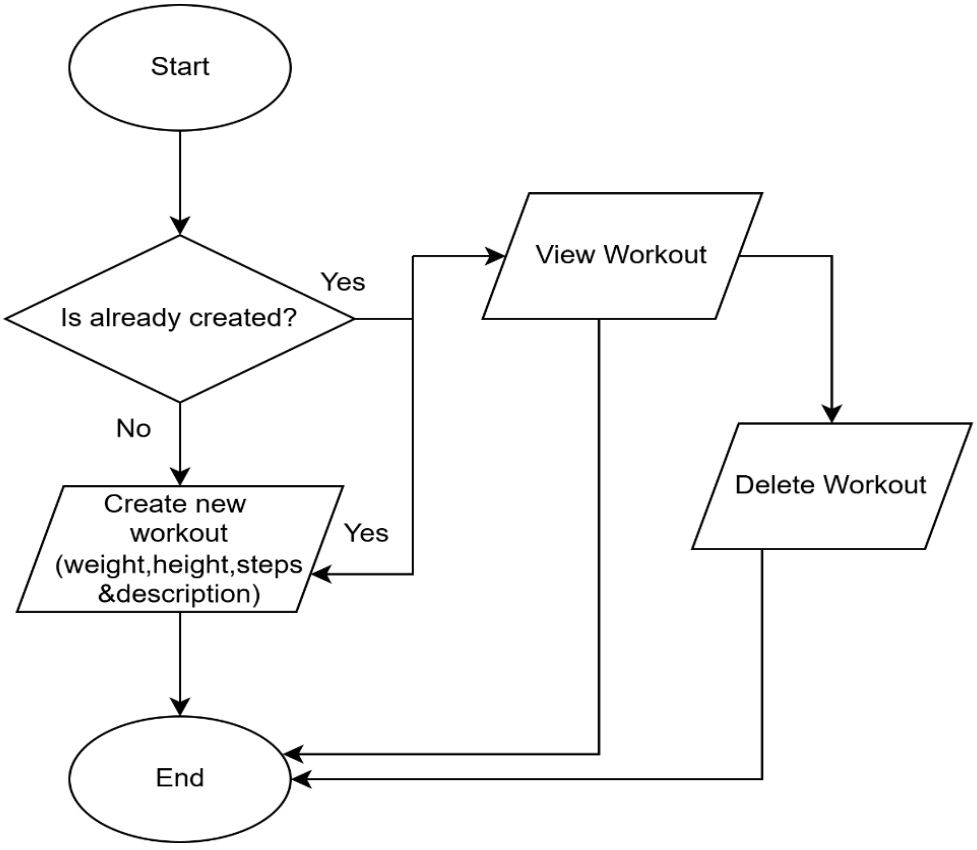

Figure 28: Workout Plan Algorithm

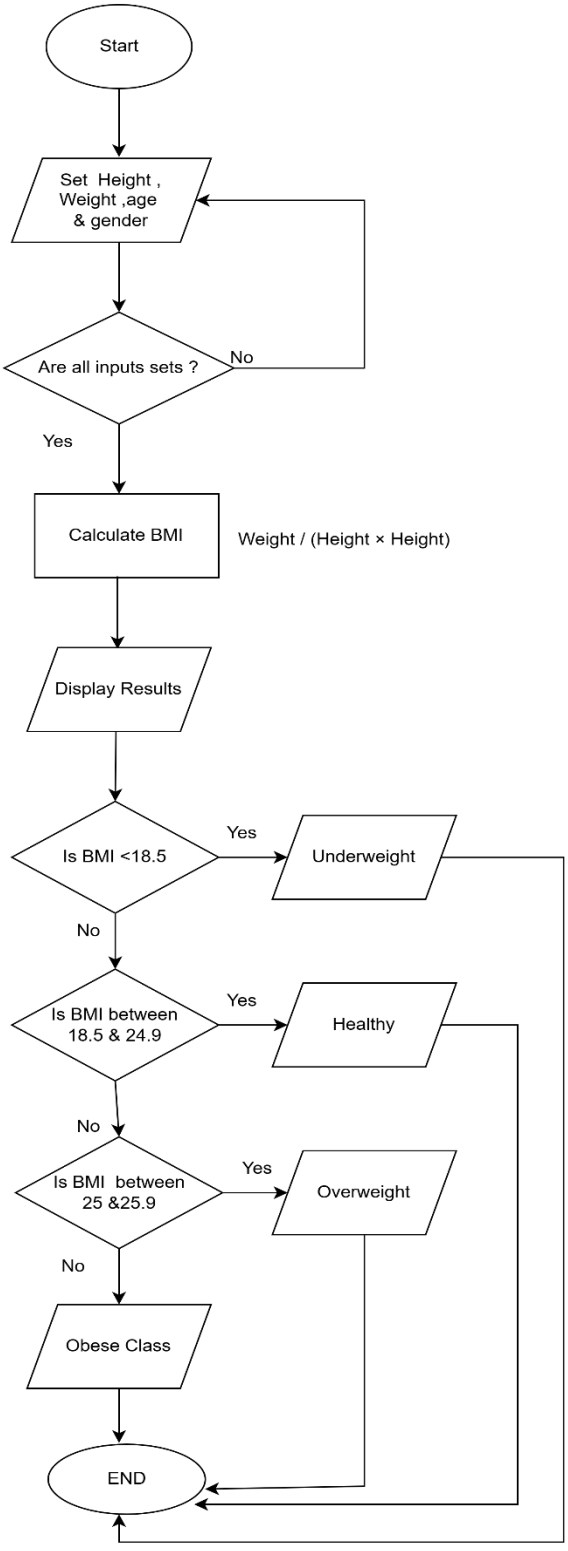

Figure 29: BMI Calculator Algorithm

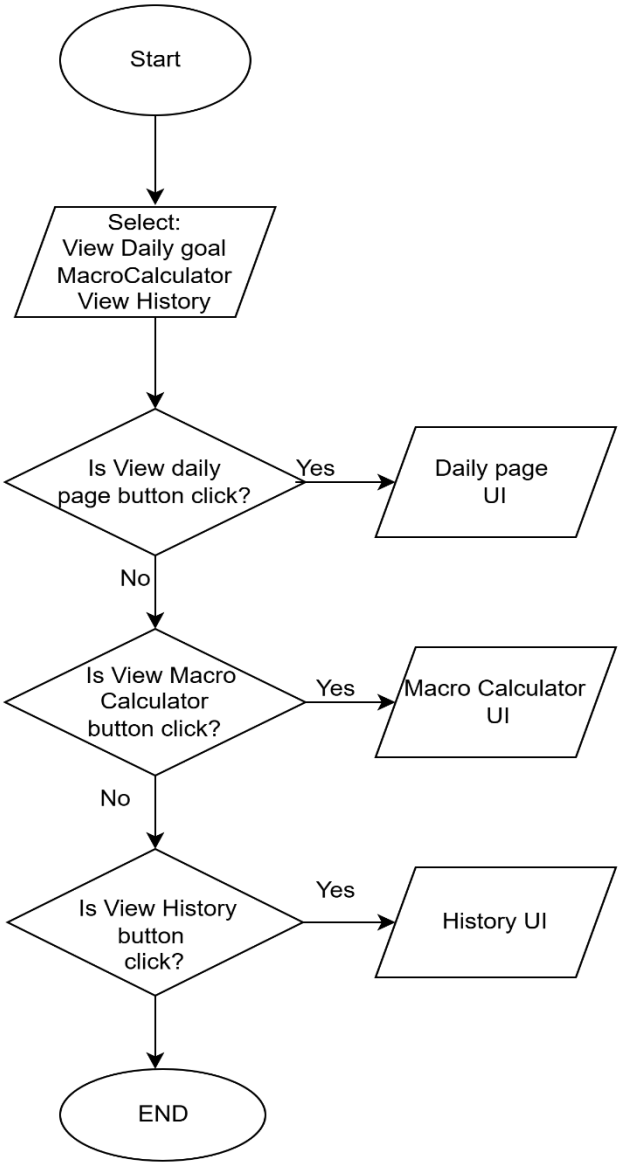

Figure 30: Nutrition Tracking Algorithm

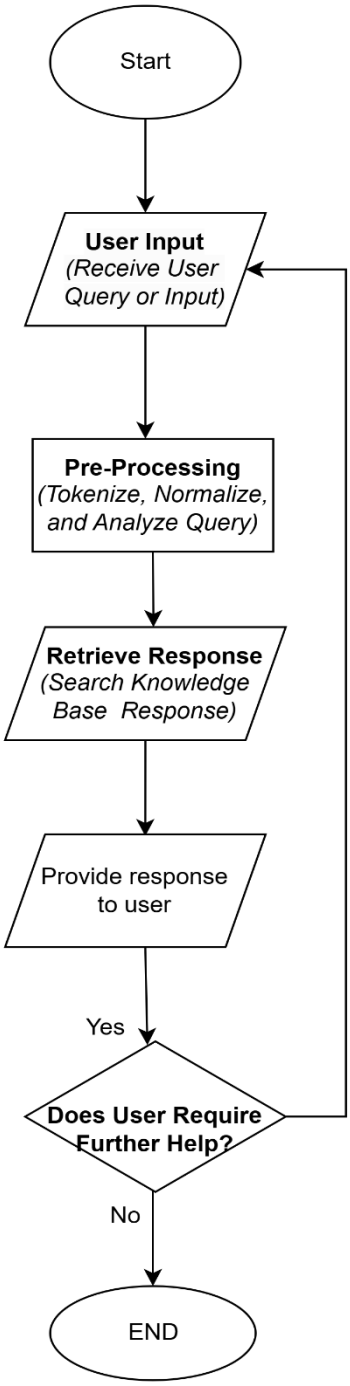

Figure 31: AI Chatbot Algorithm

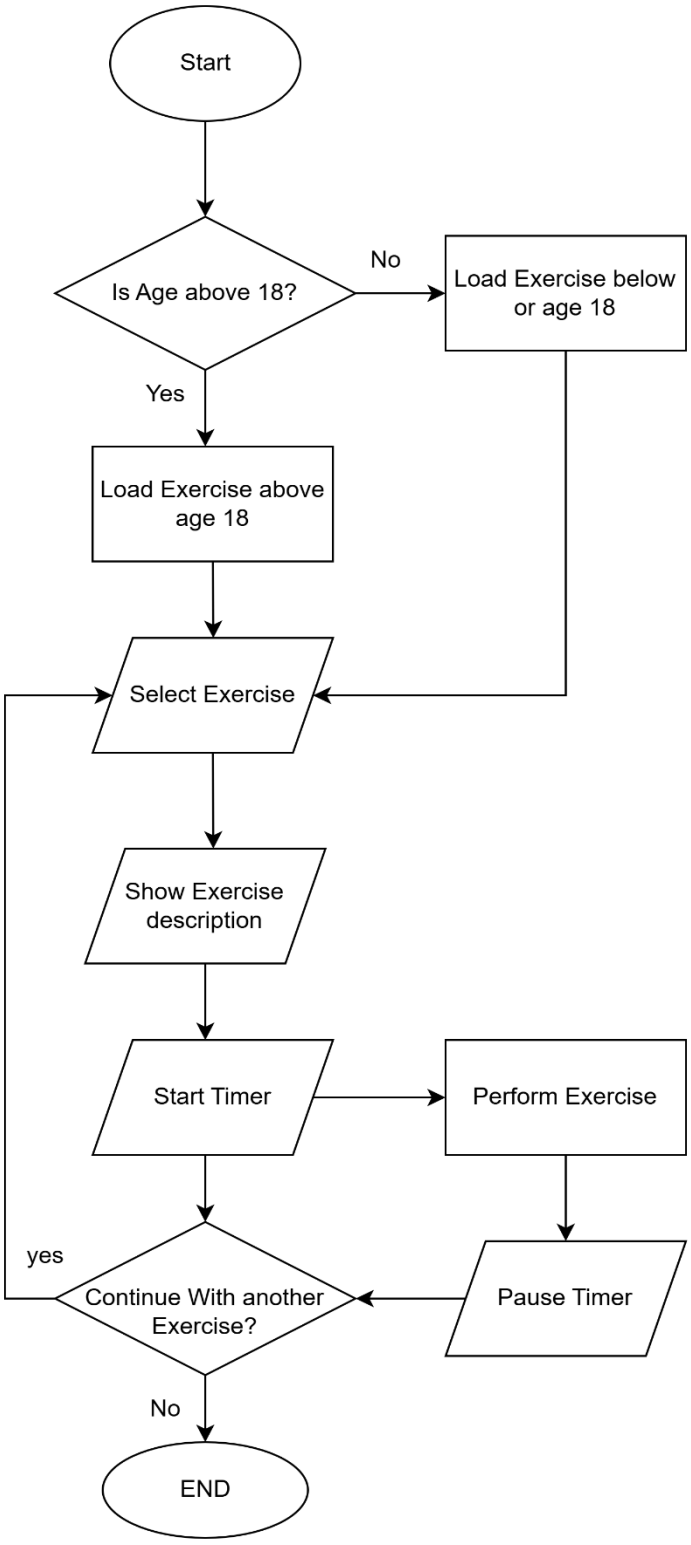

Figure 32: Exercises Algorithm

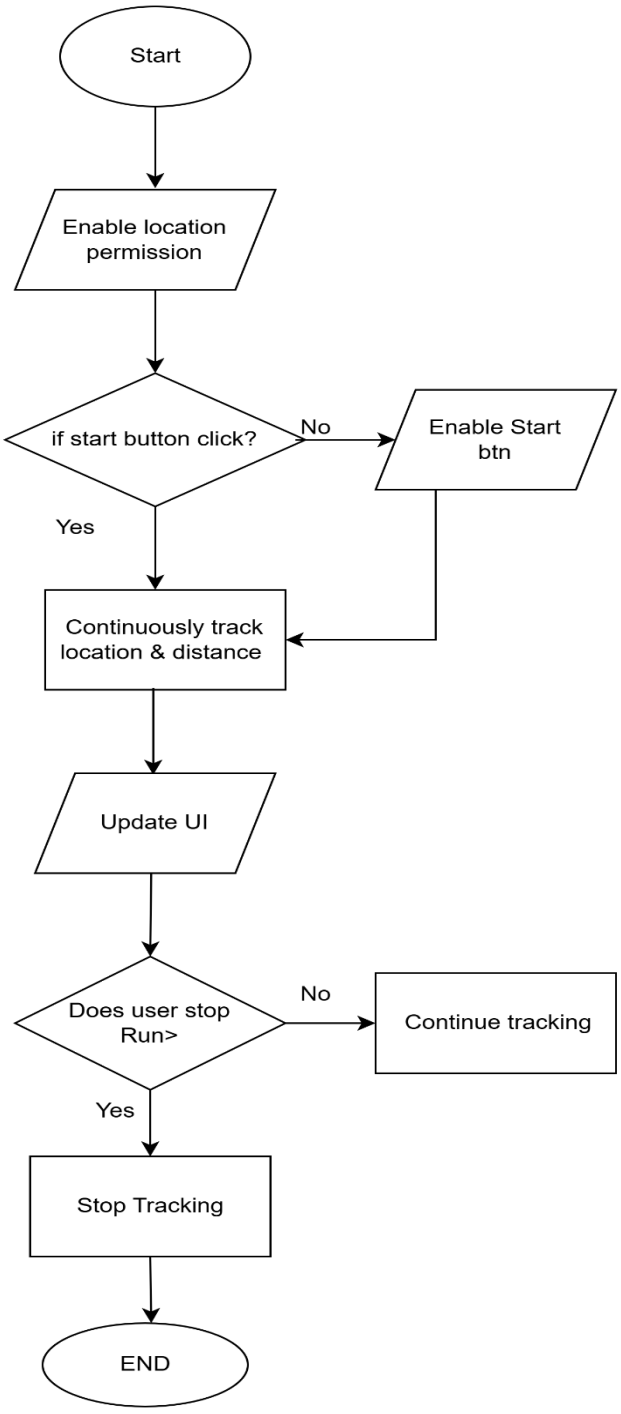

Figure 33: Run Tracker Algorithm

Chapter 6: 4th Deliverable (User Interface Design)

6.1. Introduction

This chapter focuses on the user interface (UI) design of the FitTrack application. The design aims to ensure an intuitive and user-friendly experience while maintaining consistency and functionality. It includes site maps, storyboards, navigational maps, and a GUI table to demonstrate the interface flow and component design.

6.2. Site Maps

The site map illustrates the hierarchical structure of the app, showing the relationship between different screens and modules:

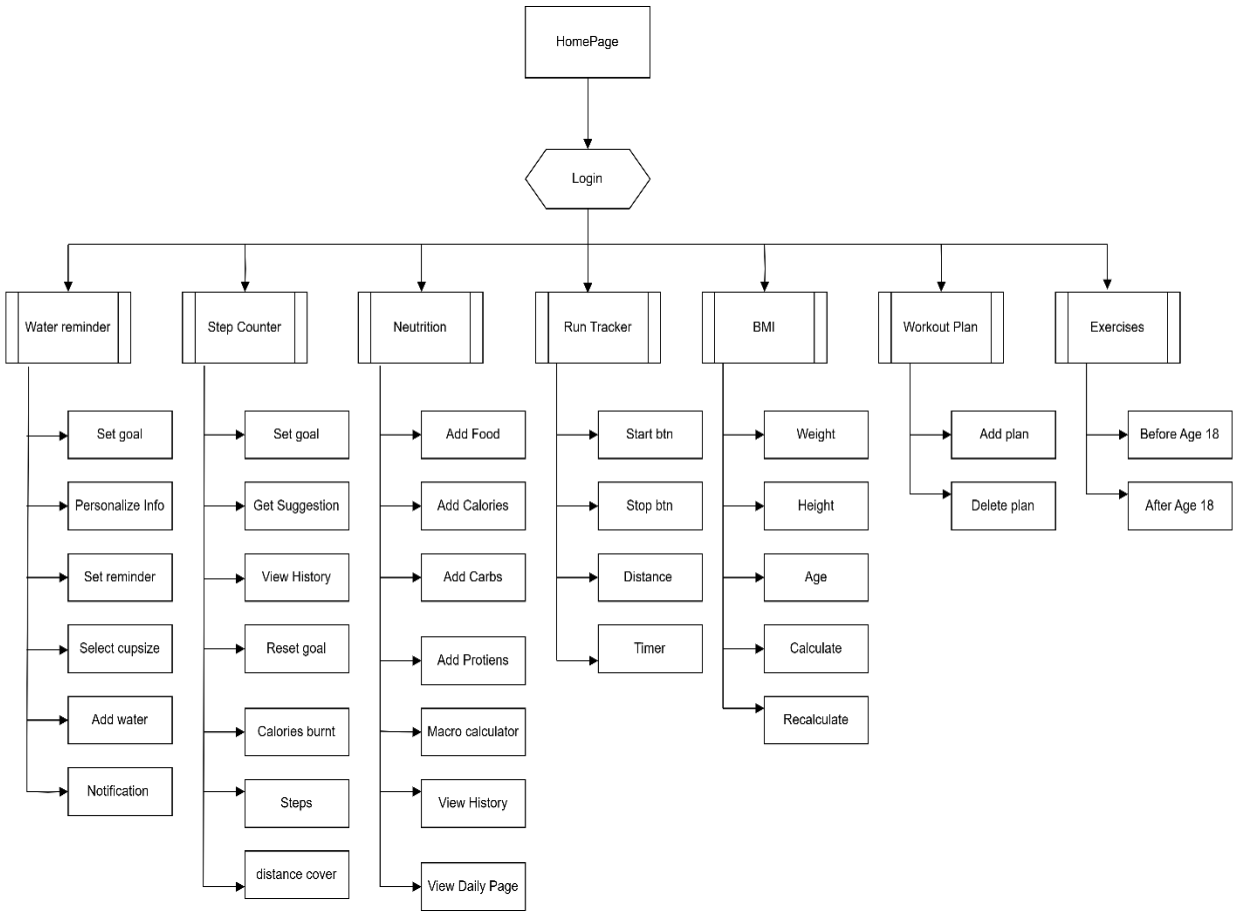

Figure 34

6.3. Story boards

A storyboard is a sequence of single images, each of which represents a distinct event or narrative. It is also a visual representation of the script illustrating the interaction between the user and the machine.

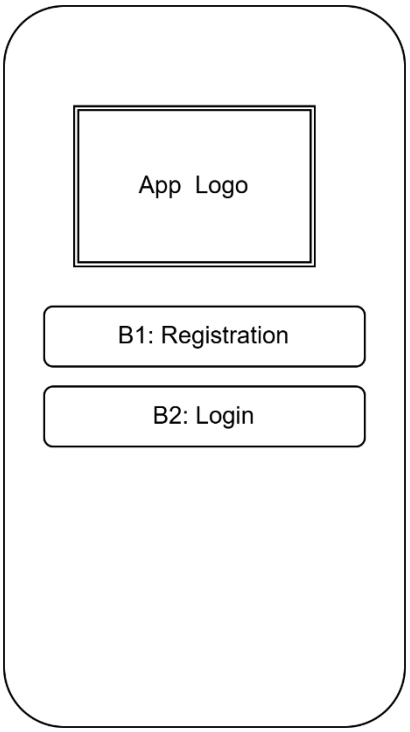

Figure 35

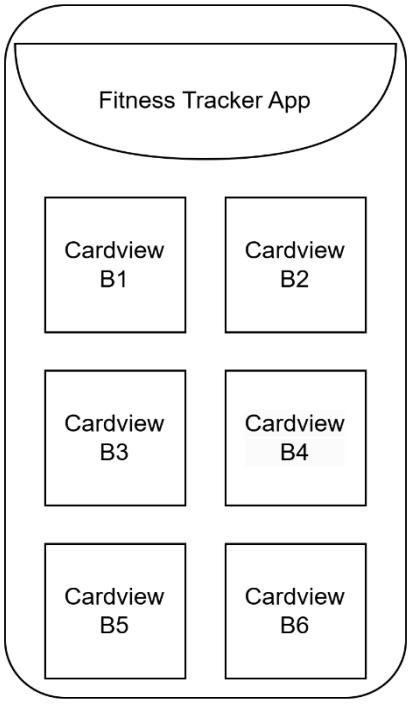

Figure 36

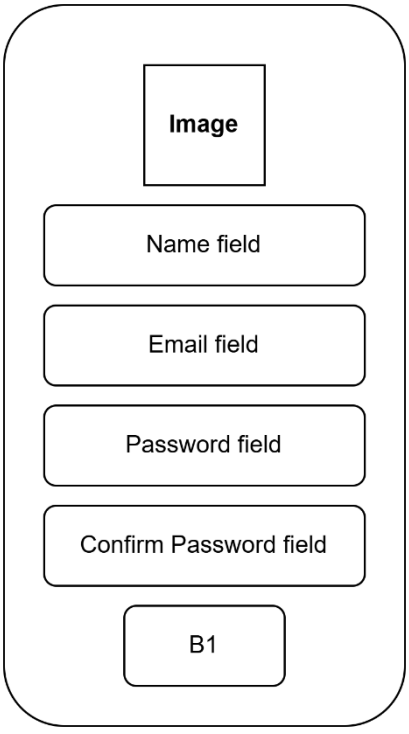

Figure 37

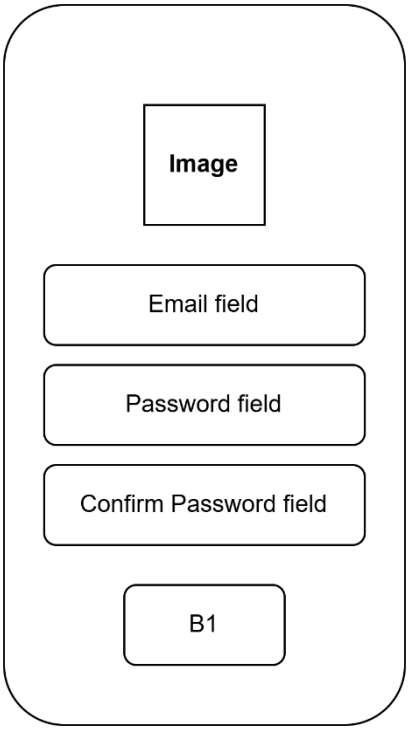

Figure 38

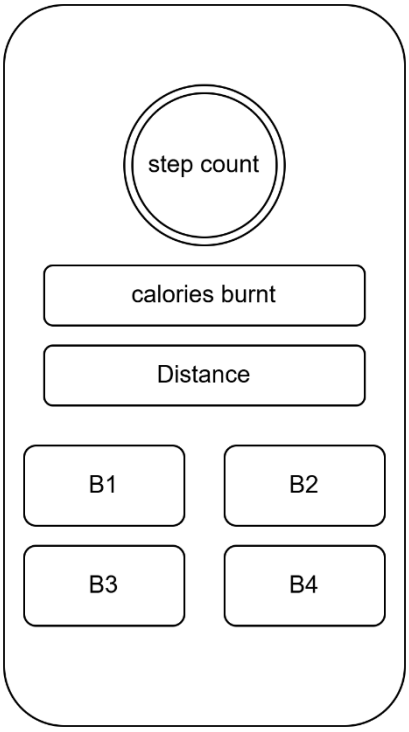

Figure 39

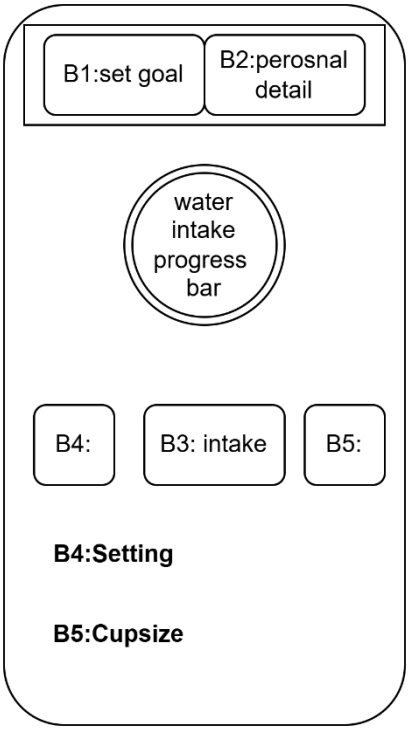

Figure 40

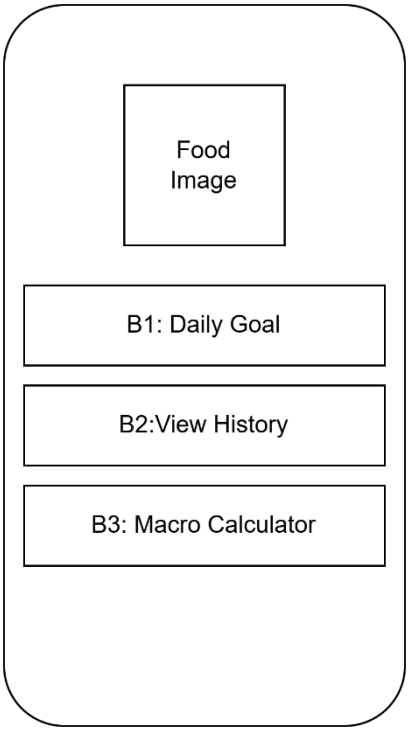

Figure 41

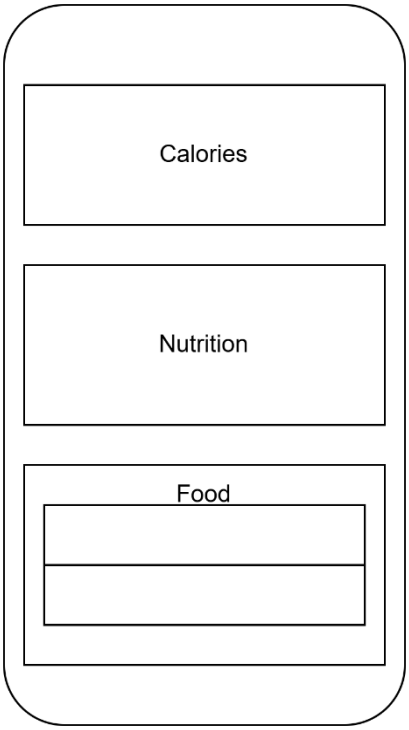

Figure 42

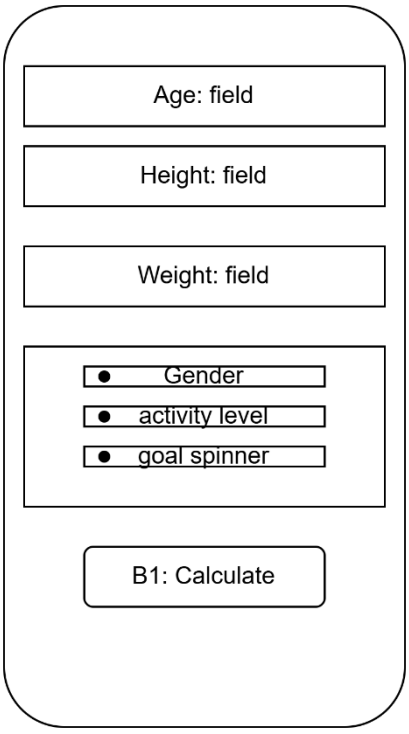

Figure 43

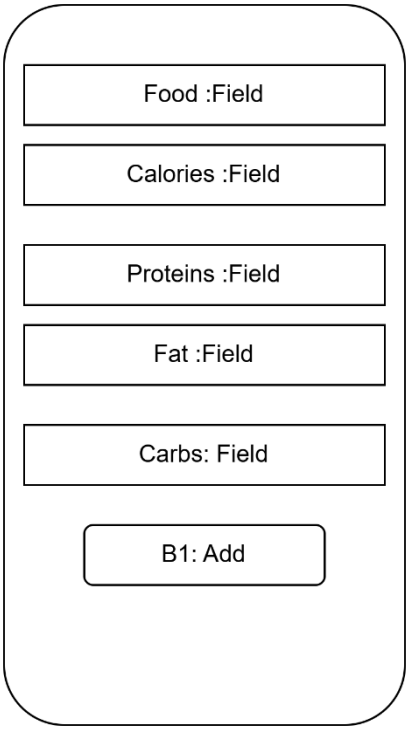

Figure 44

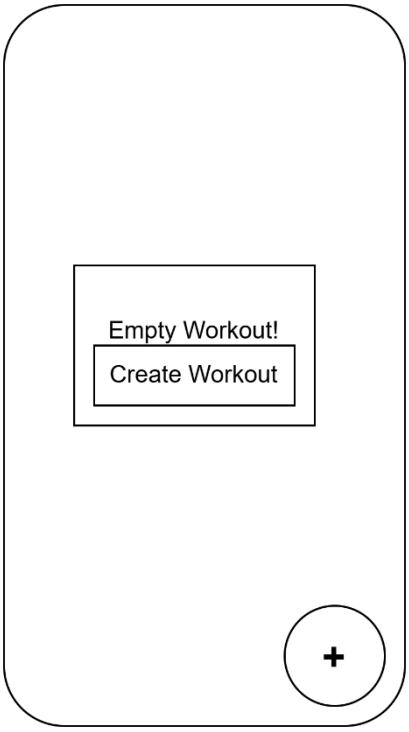

Figure 45

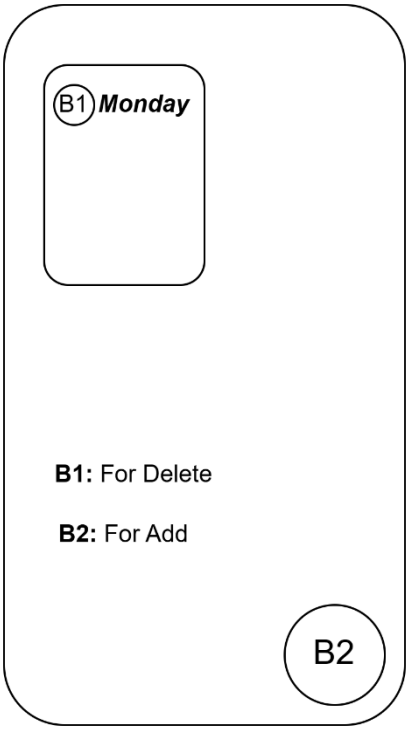

Figure 46

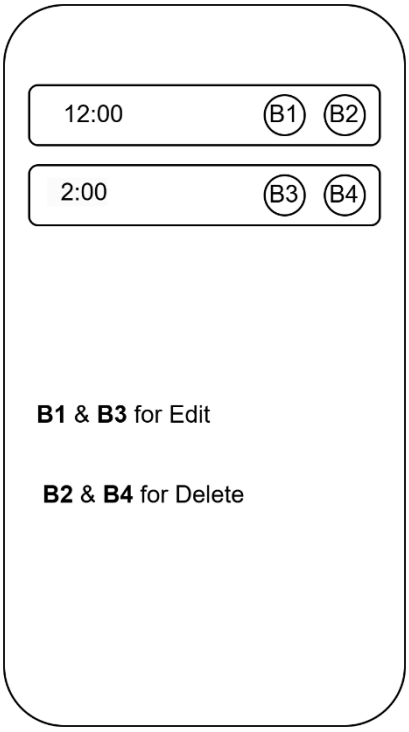

Figure 47

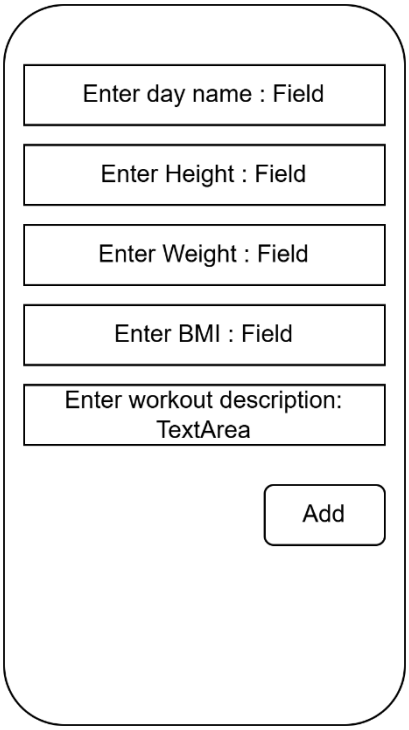

Figure 48

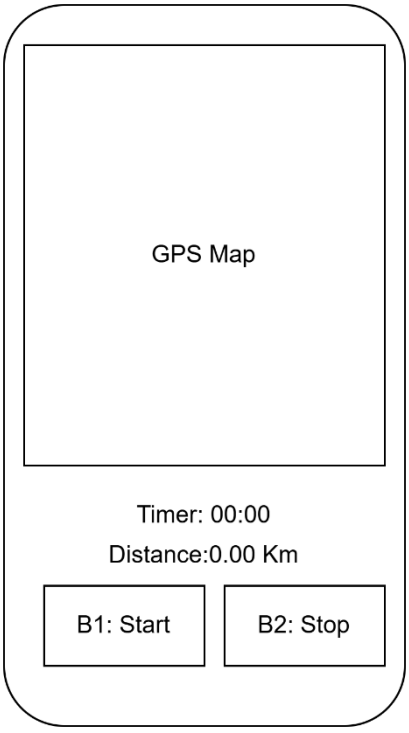

Figure 49

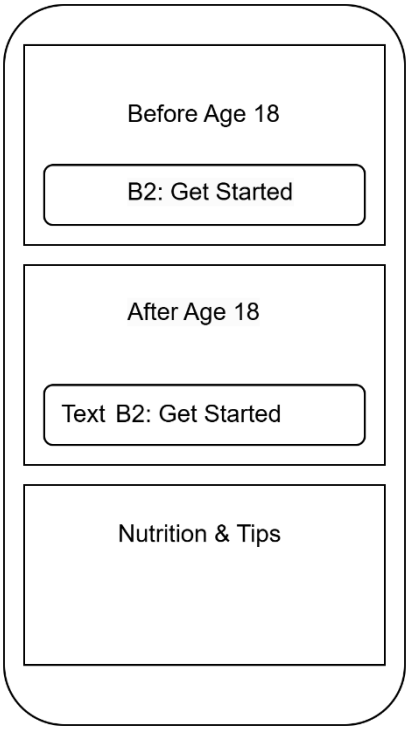

Figure 50

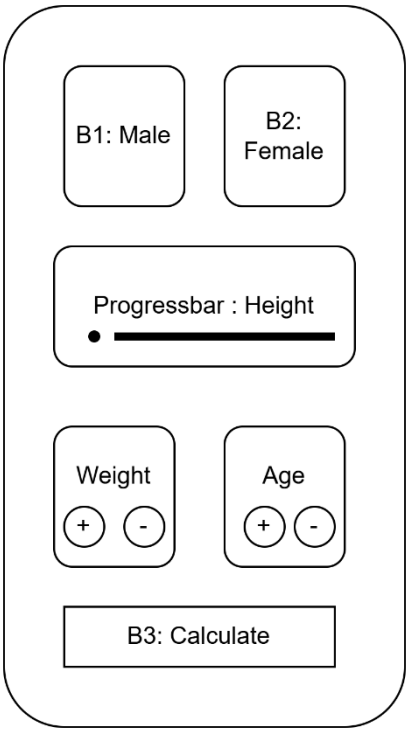

Figure 51

6.4. Navigational maps:

The navigational map ensures seamless user transitions between features:

1. **Home → Step Counter**
   - Direct access to progress and goal setting.
2. **Home → Menu→ Water Reminder → set reminders**
   - Opens the water log with reminder options.
3. **Home → Menu→ Nutrition Tracker**
   - Redirects to meal logging and summaries.
4. **Home → Run Tracker**
   - Launches the GPS tracking screen.
5. **Home → BMI Calculator**
   - Leads to the BMI input and results screen.
6. **Home → Menu→ Profile**
   - Opens the user info related.
7. **Home → Menu→ AI chatbot**
   - Redirects to chatbot.
8. **Home → Exercises→ Before Age 18 or After Age 18 → Select exercise → Start time →Stop time**
   - Opens the exercises and perform.
9. **Home → Menu→ Nutrition Tracker → View Daily page or Macro Calculator or View History**
   - Redirects to meal logging and summaries.
10. **Home → BMI**
    - Launches the BMI screen.
11. **Home → Workout**
    - Launches Workout screen

6.5 GUI Tables

The GUI table outlines the design elements for each screen.

Table 40

Login Page	
Interface Id.	UI_01
Use case Reference	UC_01 (User Login)
Snapshot	
The login page allows users to enter their email and password to authenticate themselves into the fitness tracker app	
Data dictionary reference	
Label	**Data dictionary identifier**
1	Email Field (Text field)
2	Password Field (Password field)
3	Login Button (Action button)
4	Sign Up Link (Navigational link)

Table 41

Sign Up Page	
Interface Id.	UI_02
Use case Reference	UC_02 (User Registration)
Snapshot	
The sign-up page allows new users to register by providing their email, password, and username.	
Data dictionary reference	
Label	**Data dictionary identifier**
1	Username Field (Text field)
2	Email Field (Text field)
3	Password Field (Password field)
4	Confirm Password Field (Password field)
5	Sign-Up Button (Action button)

Table 42

BMI	
Interface Id.	UI_04
Use case Reference	UC_04 (BMI)
Snapshot	
The BMI Page allows users to input their height, weight, age, and gender to calculate their Body Mass Index (BMI). The page will display the BMI result along with a classification (e.g., underweight, normal, overweight, etc.) based on the calculated value. Users can also recalculate their BMI by updating the input values.	
Data dictionary reference	
Label	**Data dictionary identifier**
1	Height (Numeric field)
2	Weight (Numeric field)
3	Age (Numeric field)
4	Gender (Dropdown menu)
5	Calculate BMI Button (Action button)
6	BMI Result (Text field)
7	BMI Classification (Text field)
8	Recalculate BMI Button (Action button)

Table 43

Water Reminder	
Interface Id.	UI_08
Use case Reference	UC_08 (Water Intake Tracking)
Snapshot	
The water reminder interface notifies users to drink water and allows them to log their water consumption.	
Data dictionary reference	
Label	**Data dictionary identifier**
1	Reminder Time (Text field)
2	Daily Goal (Text field)
3	Log Water Button (Action button)
4	Water Intake Progress (Visual progress bar)
5	Cup Size (Action button)
6	Description of Water Tips (Text field)

Table 44

Workout Plan	
Interface Id.	UI_06
Use case Reference	UC_06(Workout plan)
Snapshot	
The workout plan interface helps users create and manage their personalized workout routines with exercises, and can enter as record for weight, height, BMI, and description.	
Data dictionary reference	
Label	**Data dictionary identifier**
1	Workout Plan Name (Text field)
2	Add Exercise (Text field)
3	Add Height (Text field)
4	Add Weight (Text field)
5	Add BMI (Text field)
6	Add Description (Text field)
7	Add (Action button)

Table 45

Nutrition	
Interface Id.	UI_09
Use case Reference	UC_07 (Nutrition Tracking)
Snapshot	
The nutrition tracker interface allows users to log their meals and view the number of calories consumed.	
Data dictionary reference	
Label	**Data dictionary identifier**
1	Food Name (Text field)
2	Calories Field (Text field)
3	Add Meal Button (Action button)
4	Daily Goal Display (Text)
5	Meal Log (List view)

Table 46

Run Tracker	
Interface Id.	UI_07
Use case Reference	UC_07 (Run Tracking)
Snapshot	
The run tracker interface displays the user's running distance, duration, and calories burned, while integrating map tracking.	
Data dictionary reference	
Label	**Data dictionary identifier**
1	Distance Display (Text field)
2	Duration Display (Text field)
3	Calories Burned (Text field)
4	Start/Stop Button (Action button)
5	Map (Visual display)

Table 47

Exercises	
Interface Id.	UI_05
Use case Reference	UC_04 (Exercises)
Snapshot	
The Exercise Page allows users to view and perform exercises included in their workout plan. Each exercise displays its name, the number of sets and repetition, description of how to perform, and the duration of the exercise.	
Data dictionary reference	
Label	**Data dictionary identifier**
1	Exercise Name (Text field)
2	Before Age 18 Exercises (Action button)
3	After Age 18 Exercises (Action button)
4	Exercise duration (Action button)

Table 48

Step Counter	
Interface Id.	UI_03
Use case Reference	UC_03 (Step Counting)
Snapshot	
The step counter interface tracks the number of steps a user takes daily and shows a progress bar based on their step goal.	
Data dictionary reference	
Label	**Data dictionary identifier**
1	Steps Display (Text)
2	Progress Bar (Visual progress bar)
3	Set Goal Button (Action button)
4	History Button (Action button)

Chapter 7: Software Testing

7.1 Introduction:

This deliverable is based on the software testing. In this deliverable will make test cases and test the system according to those defined test cases. It includes the complete testing report with all test items and summary reports etc.
The standard artifacts, included in this deliverable are:

1. Test Plan
2. Test Design Specification
3. Test Case Specification
4. Test Procedure Specification
5. Test Item Transmittal Report
6. Test Log
7. Test Incident Report
8. Test Summary Report

7.2. Test plan

7.2.1. Purpose

The purpose of test plan is making sure that features this system provides are compatible with each other and how the system will react to the inputs provided by the user. The error occurring due to invalid inputs lets us know what went wrong. The testing will also check whether expected and actual outcomes are being met. If any case fails then it will be reviewed and required changes will be made to it.

7.2.2. Outline

A test plan shall have the following structure:

a. Test plan identifier
b. Introduction
c. Test items
d. Features to be tested
e. Features not to be tested
f. Approach
g. Item pass/fail criteria
h. Suspension criteria and resumption requirements
i. Test deliverables
j. Testing tasks
k. Environmental needs
l. Responsibilities
m. Staffing and training needs
n. Schedule
o. Risks and contingencies
p. Approvals

7.2.2.1. Test plan identifier
Test plan identifier is TP-001

7.2.2.2. Introduction
This test plan is a blueprint that aims at conducting the testing activities for the app "Fit Track App". The motivation behind this test plan is to make sure all required features are taken to test out their working. The purpose of test plan is making sure that features this system provides are compatible with each other and how the system will react to the inputs provided by the user. The testing will also check whether expected and actual outcomes are being met.

7.2.2.3. Test items
Items which were tested in App are as follows:

7.2.2.4. Features to be tested
- Registeration/Login
- Exercises
- Nutrition Tracking
- Water Reminder
- BMI Calculator
- Run Tracker
- AI Chatbot
- Step Counter
- Workout Plan

7.2.2.5. Features not to be tested
- View History
- View Workout
- View Reminder
- View Personal Information

7.2.2.6. Approach
We have used the traditional approach to testing, because firstly we had developed the whole system and then perform testing according the features of system.

7.2.2.7. Item pass/fail criteria
For the app to be categorized as complete and working, it means that it performs the work according to the specified requirements and satisfy them. If any of the feature stated is not working properly, meaning that it created unexpected error, it will be categorized as fail. If each feature is producing the right output which is expected and shows that it fulfils the requirement (expected output=actual output), it is categorized as pass

7.2.2.8. Suspension criteria and resumption requirements
The testing will be suspended when network issue occurs or 30% of the tests have failed. Once the above issues are resolved the testing will be resumed.

7.2.2.9. Test deliverables

The following documents should be included:

a. Test plan document
b. Test case document
c. Test design specifications
d. Test data
e. Test output
f. Test procedure specification

7.2.2.10. Testing tasks

- Features to be tested must be selected
- Specification of features stated
- Test plan prepared
- Test data collected
- Setting up testing environment
- Performing the test

7.2.2.11. Environmental needs

The hardware and software specifications of FitTrack App re as follows:

Hardware Specs:

- Mobile
- Tablet

Software Specs:

- APK install

7.2.2.12. Responsibilities

Responsibility for designing, preparing testing of features properly. The test manager is responsible executing, managing and resolving the issues of the project.

7.2.2.13 Staffing and training needs

No special skills are required; however, tester must know how to use the system from start to end and how to carry out the designed tests

7.2.2.14. Schedule

Table 49: Schedule

Activities	Members	Time Required
Test Planning and specification	Hira	7 days
Perform Testing	Hira	2 days
Test Report	Hira	2 days
Test Delivery	Hira	2 days

7.2.2.15. Risks and contingencies

- If the first component testing is not completed within a day, it could delay bug fixes and final testing.
- If the testers don't have the basic understanding of software development, testing could be delayed or not be conducted properly. If the testers don't have the basic understanding of software development, testing could be delayed or not be conducted properly.
- There is a possibility that internet access may be lost for a few days, this may lead to catching up, to follow the schedule and this results in hastiness, which consequently leads to creating errors

7.2.2.16 Approvals

The supervisor who is involved throughout the system development must approve this plan.

Table 50: Testing Approval

Name	Title	Signature
Sir Abdullah Sahi	Supervisor	

7.3. Test design specification

7.3.1. Purpose

The purpose of test design specification is to specify important details about the testing that is carried out. It highlights which items tested based on which features. Also discussed the important tasks that need to be carried out to test perform actual testing, the relevant scheduling of testing and the responsibilities associated with each.

7.3.2. Outline

A test design specification shall have the following:

a. Introduction;
b. Test items;
c. Features to be tested;
d. Approach;
e. Item pass/fail criteria;
f. Suspension criteria and resumption requirements;
g. Approvals.

7.4. Test Case Specification

7.4.1. Purpose

The test cases stated here are specified according to the test design specifications. They are used as a form of guidance for the tester, as they will know what is expected of tested item.

It is a systematic guide that cross-refers to the use cases involved against the feature being tested.

7.4.2. Outline

A test case specification shall have the following structure:

a. Test case name
b. Test case id
c. Tester
d. Date
e. Req-ID
f. UC-Name
g. Test Data
h. Expected Output
i. Actual Output
j. Status
k. Remarks

Table 51: Registeration

Test Case Name: Registeration				**Test Case Id:** TC-001		
Tester: Hira				**Date:** 12/10/2024		
Req-ID	**UC-Name**	**Test Data**	**Expected Output**	**Actual Output**	**Status**	**Remarks**
UC-01	UC_ Registeration	Null, Null, Null,	Please fill all fields	Please fill all fields	Pass	This test case is successfully executed
		Email:"invalid_email", Password:"password", Name: "John Doe"	Invalid email format	Invalid email format	Pass	Email format validation works.
		john.doe@gmail.com, Password@123, John Doe	Password must meet complexity requirement	Password must meet complexity requirement	Pass	Weak password validation works.

		john.doe@gmail.com, Password@123, John Doe	Name cannot be empty	Name cannot be empty	Pass	Name validation works.
		john.doe@gmail.com, Password@123, John Doe	Registration successful	Registration successful	Pass	Registration works with valid input.
		Duplicate Email: "john.doe@gmail.com"	Email already exists	Email already exists	Pass	Duplicate email validation works.
		"john.doe@gmail.com, Password”123", "John Doe", Profile Picture: "Null"	Registration successful with default profile picture	Registration successful with default profile picture	Pass	Default profile picture handled correctly.

Table 52: Login

Test Case Name: Login			**Test Case Id:** TC-002			
Tester: Hira			**Date:** 20/12/2024			
Req-ID	**UC-Name**	**Test Data**	**Expected Output**	**Actual Output**	**Status**	**Remarks**
UC-2	UC_Login	Null, Null,	Please choose for all the fields	Please choose for all the fields	Pass	Validation for empty fields works.
		Email:"invalid_email", Password:"password",	Invalid email format	Invalid email format	Pass	Email format validation works.

		Email:"invalid_email", Null,	Password cannot be empty	Password cannot be empty	Pass	Password field validation works.
		Email: "user@gmail.com", Password: "incorrect"	Incorrect email or password	Incorrect email or password	Pass	Invalid login details handled properly
		Email: "user@gmail.com", Password: "Password@123"	Firebase authentication verified	Firebase authentication verified	Pass	Firebase login integration verified.
		Email: "user@gmail.com", Password: "Password@123"	Redirect to home page	Redirect to home page	Pass	Post-login redirection works.

Table 53: Step Counter

Test Case Name: Step Counter			**Test Case Id:** TC-004			
Tester: Hira			**Date:** 22/12/2024			
Req-ID	**UC-Name**	**Test Data**	**Expected Output**	**Actual Output**	**Status**	**Remarks**
UC-3	UC_Step Counter	Null, Null,	Step count should remain zero	Step count remained zero	Pass	App handles no data gracefully.

		Simulated hand motion	Steps should not increase	Steps did not increase	Pass	Hand motion not detected as steps.
		Actual walking motion	Steps should increase accurately	Steps increased accurately	Pass	Step count matches physical activity.
		Walking for 5 minutes	Total steps and calories burned logged	Data logged successfully	Pass	Real-time updates observed.
		Walking while app is in the background	Steps should continue to count	Steps continued to count	Pass	Background service works correctly.
		App reopened after 10 minutes of activity	Total step count remains accurate	Total step count remains accurate	Pass	Data persistence verified.
		Exceeded daily step goal	Display congratulatory message	Congratulatory message displayed	Pass	Goal completion notification works.
		Steps reset manually	Step count reset to zero	Step count reset to zero	Pass	Manual reset works as expected.

		Sensors disabled during activity	No steps should be recorded	No steps recorded	Pass	Sensors disabled correctly.
		App uninstalled and reinstalled	Data should not persist	Data did not persist	Pass	Expected data clearing observed.
		Reached 1000 steps in 20 minutes (fast walk)	Steps and calories updated accordingly	Steps and calories updated correctly Pass	Pass	Fast walking activity tracked well

Table 54: BMI Calculator

Test Case Name: BMI Calculator				**Test Case Id:** TC-008		
Tester: Hira				**Date:** 23/12/2024		
Req-ID	**UC-Name**	**Test Data**	**Expected Output**	**Actual Output**	**Status**	**Remarks**
UC-4	UC_BMI_ Calculator	Null, Null, Null, Null, Null, Null, Null	Please fill all the fields	Please fill all the fields	Pass	This test case is successfully executed
		Height: "170 cm", Weight: "65 kg"	Display BMI: 22.5 (Normal Weight).	Display BMI: 22.5 (Normal Weight).	Pass	BMI calculation works correctly.

		Height: "0", Weight: "65 kg"	Display error: "Height cannot be zero or empty."	Display error: "Height cannot be zero or empty."	Pass	Validation for invalid height works.
		Height: "170 cm", Weight: "0 kg	Display error: "Weight cannot be zero or empty."	Display error: "Weight cannot be zero or empty."	Pass	Validation for invalid weight works.
	UC_BMI_ Result	BMI: 30.5	Display message: "Overweight. Consider consulting a doctor."	Display message: "Overweight. Consider consulting a doctor."	Pass	BMI result messages are appropriate
		BMI: 17.0	Display message: "Underweight. Consider a nutrition plan."	Display message: "Underweight. Consider a nutrition plan."	Pass	BMI result messages are appropriate.

Table 55: Exercises

Test Case Name: Exercises			**Test Case Id:** TC-007			
Tester: Hira			**Date:** 20/06/2023			
Req-ID	**UC-Name**	**Test Data**	**Expected Output**	**Actual Output**	**Status**	**Remarks**
UC-5	UC_Start_ Exercise	Null,	Display error: "No exercise selected."	Display error: "No exercise selected."	Pass	Validation for no exercise selection works.

		Selected: "Push-Ups"	Start the push-up timer and display instructions.	Start the push-up timer and display instructions.	Pass	Exercise starts with correct instructions.
	UC_Stop_ Exercise	Ongoing Exercise: "Push-Ups"	Stop exercise and save progress to SQLite database.	Stop exercise and save progress to SQLite database.	Pass	Exercise stops

Table 56: Workout Plan

Test Case Name: Nutrition Tracking				**Test Case Id:** TC-005		
Tester: Hira				**Date:** 26/12/2024		
Req-ID	**UC-Name**	**Test Data**	**Expected Output**	**Actual Output**	**Status**	**Remarks**
UC-6	UC_Open_ Workout_Dialog	User clicks "Create Plan" button	Dialog box appears with fields: Steps, Height, Weight, etc.	Dialog box appears with fields: Steps, Height, Weight, etc	Pass	Dialog opens as expected.
	UC_Validation	User leaves all fields empty	Display error: "All fields are required."	Display error: "All fields are required."	Pass	Validation for empty fields works correctly.

	UC_Input_Data	User enters Steps: 1000, Height: 5.7, Weight: 70 Exercise:Push-Up Description:text	Data is saved and displayed in the workout plan list.	Data is saved and displayed in the workout plan list.	Pass	Input data is saved successfully in SQ Lite Db.
	UC_Delete_ Workout_Plan	User clicks "Delete" button on a plan	Remove selected workout plan from the list.	Remove selected workout plan from the list	Pass	Plan deletion works as expected.
	UC_Close_Dialog	User clicks "Cancel" button in the dialog	Close dialog without saving any data.	Close dialog without saving any data.	Pass	Cancel button works as expected
	UC_Input_ Validation	User enters non-numeric data for Steps	Display error: "Invalid input for Steps. Please enter a number."	Display error: "Invalid input for Steps. Please enter a number."	Pass	Handles invalid data inputs correctly.

Table 57: Run Tracker

Test Case Name: Run Tracker			**Test Case Id:** TC-008			
Tester: Hira			**Date:** 28/12/2024			
Req-ID	**UC-Name**	**Test Data**	**Expected Output**	**Actual Output**	**Status**	**Remarks**

UC-7	UC_Start_ Run_Tracking	GPS enabled, User clicks "Start Run" location on the map.	Start tracking route and display live location on the map.	Start tracking route and display live location on the map.	Pass	Tracking starts correctly with GPS enabled.
	UC_Update_ RealTimeData	GPS signal strong	Update distance, timer	Update distance, timer	Pass	Real-time updates work with strong GPS signal.
	UC_Pause_ Tracking	User clicks "Pause"	Pause run tracking, stop updating distance and timer.	Pause run tracking, stop updating distance and timer.	Pass	Tracking pauses successfully.
	UC_Resume_ Tracking	User clicks "Resume"	Resume run tracking from the last recorded point.	Resume run tracking from the last recorded point.	Pass	Tracking resumes correctly.
	UC_Stop_ Run_Tracking	User clicks "Stop"	Stop tracking.	Stop tracking.	Pass	Run Stop Successfully.
	UC_Calculate_ Distance	GPS enabled, user runs 5 km	Calculate and display distance: 5 km.	Calculate and display distance: 5 km.	Pass	Distance calculation works correctly.

Table 58: Add Alarm for Water

Test Case Name: Add Alarm For Water			**Test Case Id:** TC-003			
Tester: Hira			**Date:** 20/06/2024			
Req-ID	**UC-Name**	**Test Data**	**Expected Output**	**Actual Output**	**Status**	**Remarks**
UC-8	UC_Set-Alarm	Null, Null, Null, Null,	Fields cannot be empty	Fields cannot be empty	Pass	This test case is successfully executed
		08:00 AM, Null, Null, Null	Reminder type must be selected	Reminder type must be selected	Pass	This test case is successfully executed
		08:00 AM, Water, Null, Null	Alarm set successfully	Alarm set successfully	Pass	This test case is successfully executed
		08:00 AM, Water, Repeat: Daily, Null	Alarm set successfully	Alarm set successfully	Pass	This test case is successfully executed
		Null, Water, Repeat: Daily, Sound Enable	Time must be provided	Time must be provided	Pass	Missing time issue identified.
		08:00 AM, Water, Repeat: Weekly, Notification sound not enabled	Notification sound must be enabled	Notification sound must be enabled	Pass	Notification sound issue identified.

Table 59: Nutrition Tracking

Test Case Name: Nutrition Tracking			**Test Case Id:** TC-005			
Tester: Hira			**Date:** 24/12/2024			
Req-ID	**UC-Name**	**Test Data**	**Expected Output**	**Actual Output**	**Status**	**Remarks**
UC-9	UC-Set_Daily_Goals	Null, Null	Please fill all the fields	Please fill all the fields	Pass	Validation for empty fields works.
		Calories: 2000, Protein: 70g, Fats: 50g, Carbs: 250g	Daily goal set successfully	Daily goal set successfully	Pass	Goals updated in database.
		Negative values for macros (e.g., Protein: -50g)	Input must be positive	Input must be positive	Pass	Validation for negative values works
		Calories: 2000, Missing macros (Protein: Null)	All fields must be filled	All fields must be filled	Pass	Partial input validation works.
	UC_Macro_Calculator	Weight: 70kg, Activity Level: Moderate	Accurate macro calculation displayed	Accurate macro calculation displayed	Pass	Accurate macro calculation displayed

		Weight: Null, Activity Level: High	Weight is required	Weight is required	Pass	Validation for missing weight works.
		Weight: 60kg, Activity Level: Null	Activity level must be selected	Activity level must be selected	Pass	Validation for missing activity works.

7.4.2.1. Test case specification identifier

The test case specification identifier is TCS_001.

7.5. Test procedure specification

7.5.1. Purpose

The purpose of the following test procedure specification is to specify the steps the tester will follow for executing a set of test cases. This also involves analyzing the system to evaluate the set of features that needed to be considered for testing.

7.5.2 Outline

A test procedure specification shall have the following structure:

a. Test procedure specification identifier
b. Purpose
c. Special requirements
d. Procedure steps

Details on the content of each section are contained in the following sub clauses.

7.5.2.1. Test procedure specification identifier

The unique identifier for test procedure specification is TPS – 1.

7.5.2.2. Purpose

The purpose of this module is to describe how the testing of the system is carried out. The procedure is carried out to execute the following test cases: TC-001 up-to TC-9. All of the 13 test cases are conducted manually within specified time constraint.

7.5.2.3. Special requirements

There are some special requirements in terms of skills; however, to carry out the testing, the following must be there:

- App installed in the smart phone

7.5.2.4. Procedure steps

Following are the procedure steps:

7.5.2.4.1. Log

There is no any special method or format for execution of test cases.

7.5.2.4.2. Set up

The following need to be considered before the testing can be carried out:

- App running in the mobile phone

7.5.2.4.3. Start

User Login to the system and continue further.

7.5.2.4.4. Proceed

Execute the tests in chronological order as specified from TC-001 that lead up-to TC-9.

7.5.2.4.5. Measure

The tests are conducted manually, therefore, the actual output will be measured based on the expected output and of course on human observation.

7.5.2.4.6. Shut down

Once all tests are carried out, the tester can logout of the system.

7.5.2.4.7. Restart

Login again to start the testing again.

7.5.2.4.8. Stop

Once all tests are carried out, the tester can logout of the system.

7.5.2.4.9. Wrap up

Make sure output of each test case is recorded in the test log.

7.5.2.4.10. Contingencies

Guidelines for inputting requirements can be changed.

7.6. Test item transmittal report

7.6.1. Purpose

To identify the test items being transmitted for testing. It includes the person responsible for each item, its physical location, and its status. Any variations from the current item requirements and designs are noted in this report.

7.6.2. Outline

A test item transmittal report shall have the following structure:

a. Transmittal report identifier
b. Transmitted items
c. Location
d. Status
e. Approvals

Details on the content of each section are contained in the following sub clauses.

7.6.2.1. Transmittal report identifier
The identifier for Transmittal report is TTR-001.

7.6.2.2. Transmitted items
A complete app is transmitted to the essential person for carrying out testing in a reasonable manner and each team member is responsible for it.

7.6.2.3. Location
To use our app tester must go to it and login only.

7.6.2.4. Status
The status of all test items transmitted is pass and all item are functioning properly as the actual output match the expected out meaning that the test plan has worked successfully.

7.6.2.5. Approvals

Table 60: Transmittal Approval

Name	Title	Signature
Sir Ahsan	Supervisor	

7.7. Test log
7.7.1. Purpose
The purpose of test log is to record all relevant details about the test cases executed including the output received against each test case. The log represents the observed results of the system for the tests conducted.

7.7.2. Outline
A test log shall have the following structure:

a. Test log identifier;
b. Description;
c. Activity and event entries.

Details on the content of each section are contained in the following sub clauses.

7.7.2.1. Test log identifier
Identifier for test log is TL-001.

7.7.2.2. Description

The items testes are already identified in the above section 5.2.2.3 Test Items. However, the testing is done manually and therefore the results are recorded manually by the tester. All of the environmental needs that helped in carrying out the test successfully are listed under the section 5.2.2.11 Environmental Needs.

7.7.2.3. Activity and event entries

The items testes are already identified in the above section 7.2.2.3 Test Items. However, the testing is done manually and therefore the results are recorded manually by the tester. All of the environmental needs that helped in carrying out the test successfully are listed under the section 5.2.2.11 Environmental Needs.

7.7.2.3.1. Execution description

The items testes are already identified in the above section 7.2.2.3 Test Items. However, the testing is done manually and therefore the results are recorded manually by the tester. All of the environmental needs that helped in carrying out the test successfully are listed under the section 5.2.2.11 Environmental Needs.

7.7.2.3.2. Procedure results

For gathering or write results the test case tables are already build up with their results of different test cases in section 5.4.3.

7.7.2.3.3. Environmental information

The hardware and software specifications of FitTrack App are as follows:

Hardware Specs:

- Mobile
- Tablet

Software Specs:

- APK install

7.8. Test incident report

7.8.1. Purpose

The purpose of this incident report is to record whether an unexpected incident occurred. If it did, then all necessary details about it are to be recorded.

7.8.2. Outline

A test incident report shall have the following structure:

a. Test incident report identifier
b. Summary
c. Incident description
d. Impact

Details on the content of each section are contained in the following sub clauses.

7.8.2.1. Test incident report identifier Test
incident report identifier is TIR-001.

7.8.2.2. Summary
No expected incident occurred as the errors are proactively handled beforehand through error messages.

7.8.2.3. Incident description No
incidents occurred.

7.8.2.4. Impact No
incidents occurred.

7.9. Test summary report

7.9.1. Purpose
The purpose of this test summary report is to summarize everything about the testing performed on system features. It wraps up everything regarding the results achieved against the features tested and describes various activities performed as part of testing.

7.9.2. Outline
A test summary report shall have the following structure:

a. Test summary report identifier
b. Summary
c. Variances
d. Comprehensive assessment
e. Summary of results
f. Evaluation
g. Summary of activities
h. Approvals

Details on the content of each section are contained in the following sub clauses.

7.9.2.1. Test summary report identifier Test
summary report identifier is TSR-001.

7.9.2.2. Summary
Test items identified against the features stated under 7.2.2.4. Features to be tested. Environment needed to carry out tests successfully is stated under 7.2.2.11 Environmental Needs. Sections 7.2, 7.3, 7.4, 7.5, 576 & 7.8 can be referred to for checking Test Plan, Test Design specification, Test Case Specification, Test Procedure Specification, Test Transmittal Report and Test logs.

7.9.2.3. Variances No
Variance.

7.9.2.4. Comprehensiveness assessment

The tests were carried out just as they were described in the test plan. All features the tests are conducted against are also described under the test plan.

7.9.2.5. Summary of results

Almost Eight test cases were designed to carry out during testing. All of them carried out successfully and the actual result received matched with the expected result, therefore, all of them declared as pass.

7.9.2.6. Evaluation

The process to carry out the testing was simple and we received the results as we expected them to be like. The thing we have concluded from testing of our system is that if the user follows the user manual provided, then they can work through the system without having to deal with issues..

7.9.2.7. Summary of activities

The items listed under 7.2.2.3 Test Items tested against the features listed under 7.2.2.4. Features to be tested. The duration spent on the testing activities are given below.

Table 61: Summary

Activities	Members	Time Required
Test Planning and specification	Hira	7 days
Perform Testing	Hira	2 days
Test Report	Hira	2 days
Test Delivery	Hira	2 days

7.9.2.8. Approvals

Table 62: Approvals

Name	Title	Signature
Sir Abdullah	Supervisor	

Chapter 8: User Technical Manual

A. GENERAL INFORMATION

8.1 System Overview

System Name: FitTrack
System Code: FT101
Category:

- **Major Application**: Tracks steps, water intake, nutrition, exercises, and BMI with secure data handling and AI-driven recommendations.
- **General Support System**: Syncs fitness data across devices and integrates with wearable fitness devices.

Operational Status:

- **Partially Operational**: Core features like step tracking, water intake, and BMI calculation are functional.
- **Under Development**: Advanced features like AI-based nutrition guidance and GPS tracking are in progress.

B. SYSTEM SUMMARY

FitTrack is designed to be user-friendly, enabling seamless interaction through an intuitive interface. Key features include:

- **Step Counter**: Tracks daily steps and allows goal setting.
- **Water Reminder**: Reminds users to stay hydrated and logs water intake.
- **Nutrition Tracker**: Logs meals and tracks macronutrients.
- **Run Tracker**: Monitors real-time running stats using GPS.
- **BMI Calculator**: Provides health insights based on user details.
- **Workout Plans**: Offers customizable and pre-defined exercise routines.
- **Exercises**: Perform various exercises based on age.
- **AI helper**: For taking food related info.
- **Profile**: Create profile, Registeration and login.

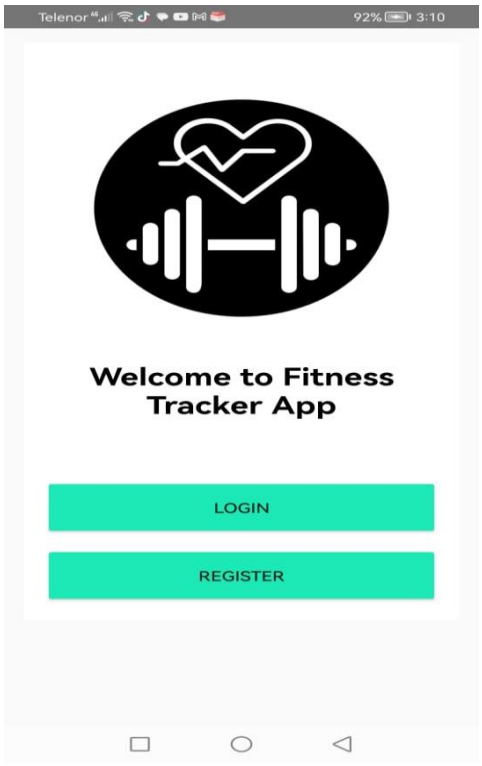

Figure 52: Main Screen

Figure 53: Registeration

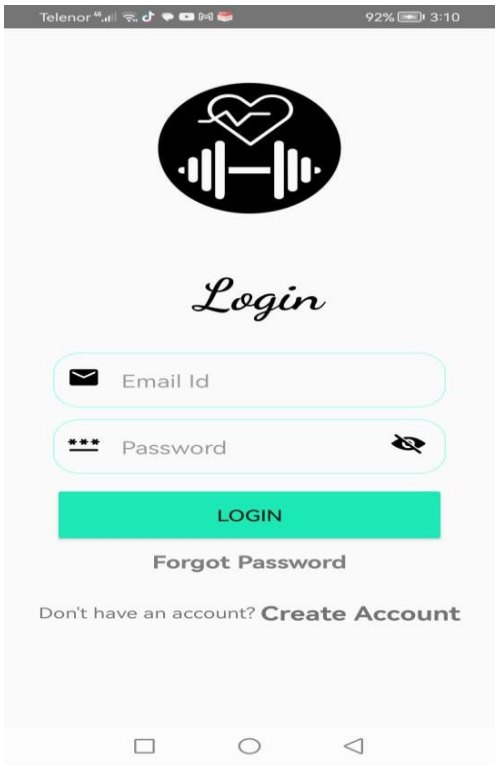

Figure 54: Login

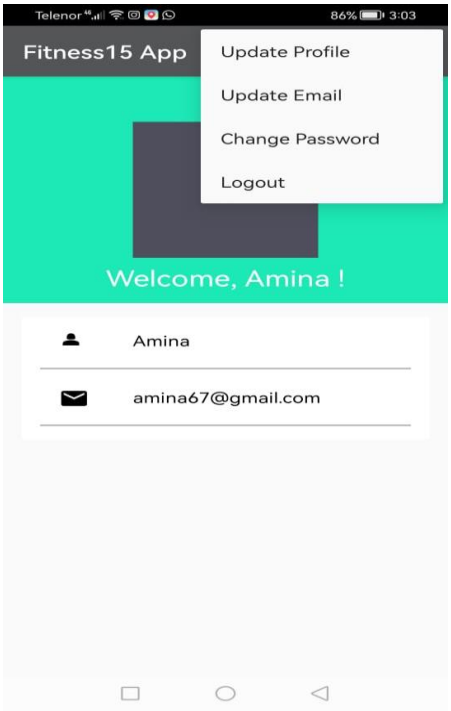

Figure 55: User Profile Details

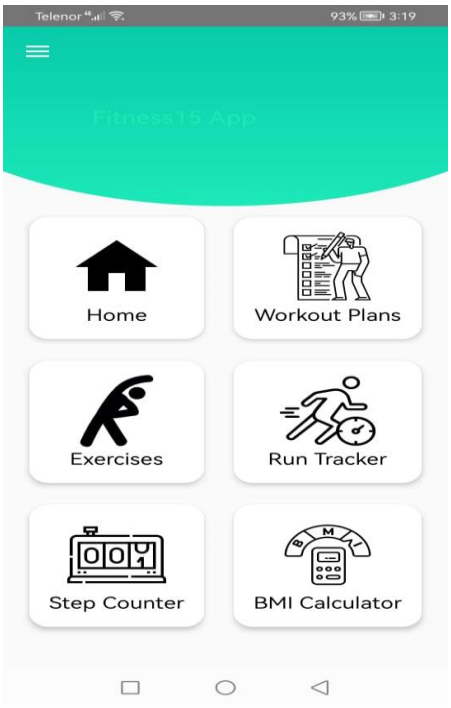

Figure 56: Dashboard

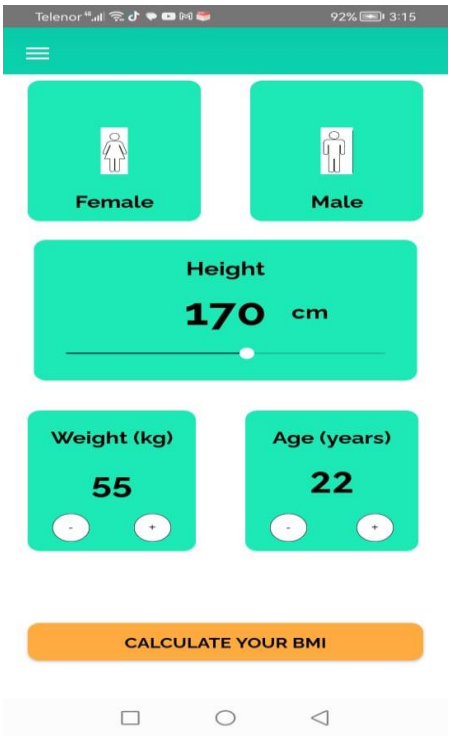

Figure 57: BMI Calculator

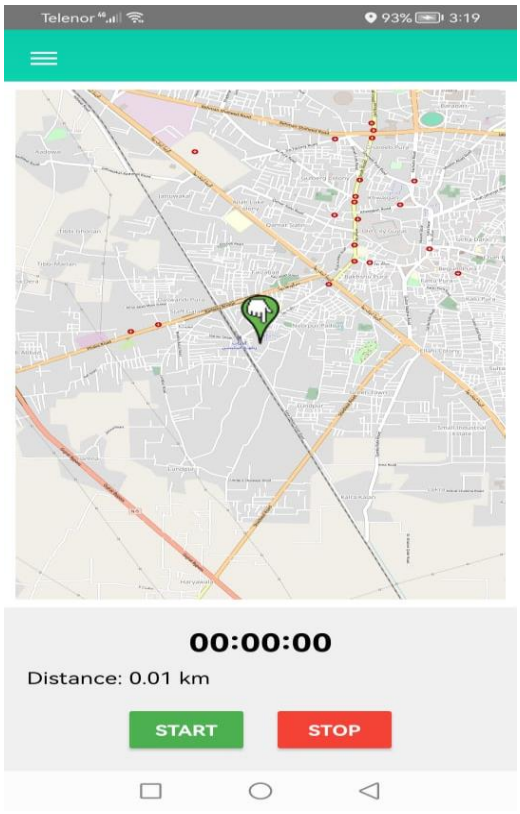

Figure 58: Run Tracker

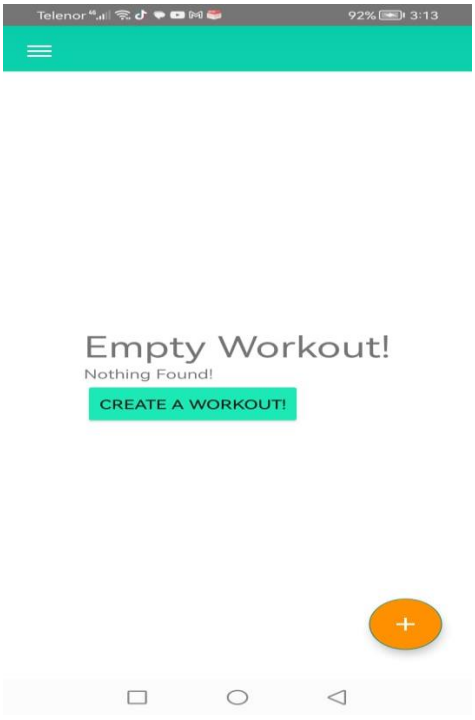

Figure 59: Workout Plan

Figure 60: Nutrition Tracking

Figure 61: Macro Calculator

Figure 62: AI Helper

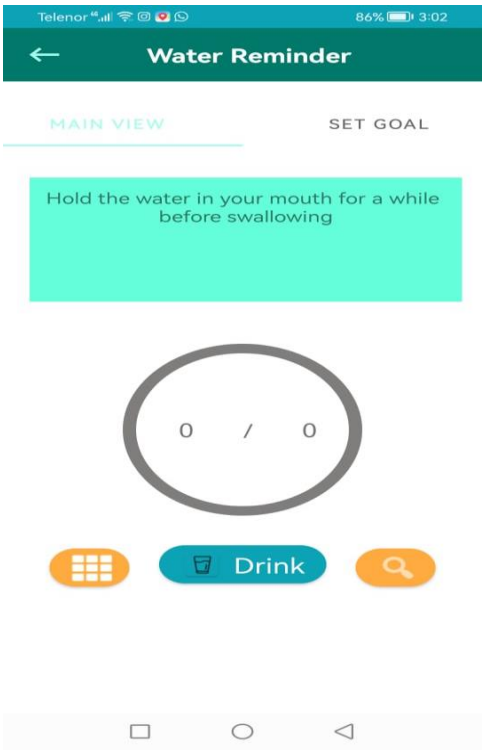

Figure 63: Water Reminder

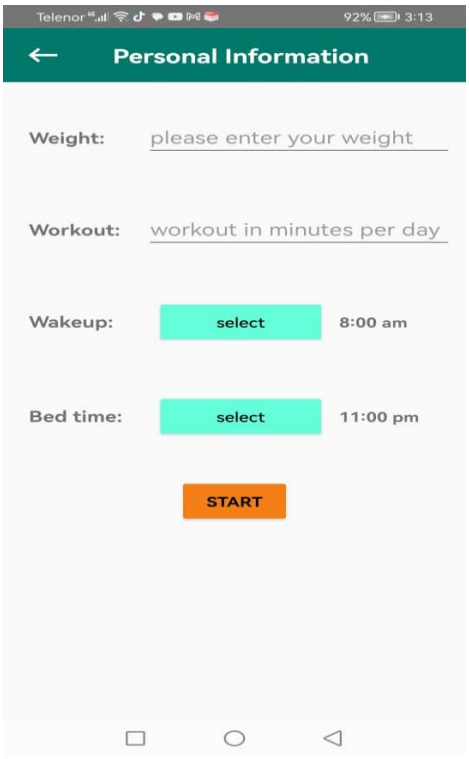

Figure 64: Personal Information

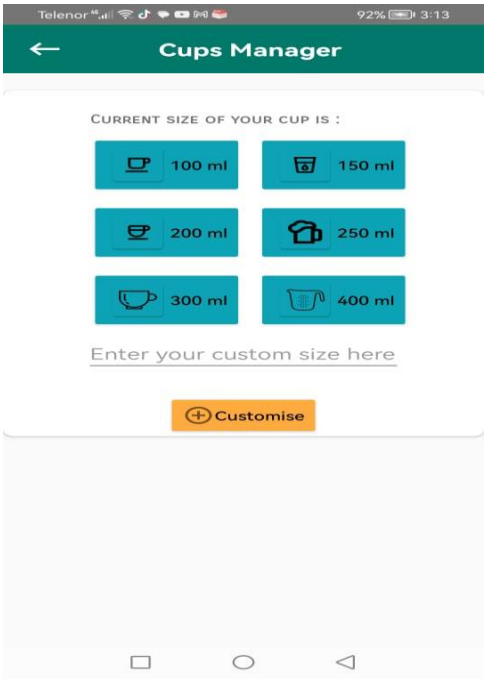

Figure 66: Cups Size Selection

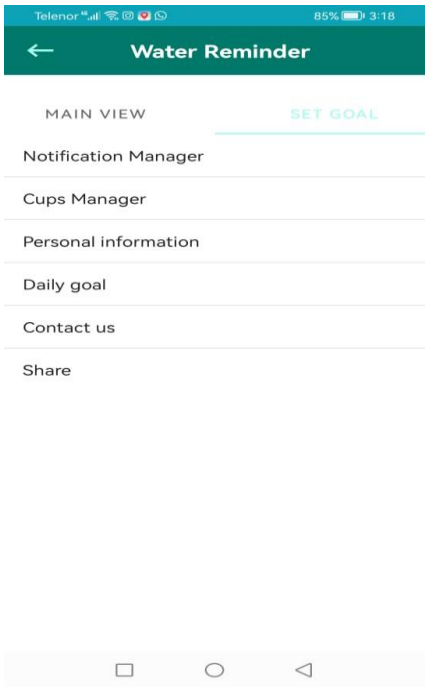

Figure 67: Water Reminder Menu

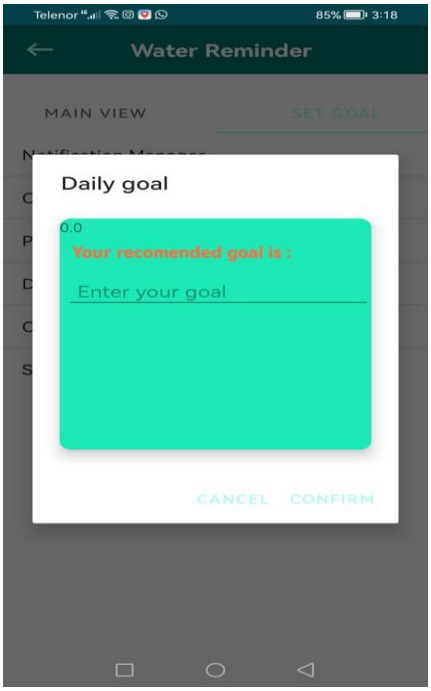

Figure 68: Daily Goal

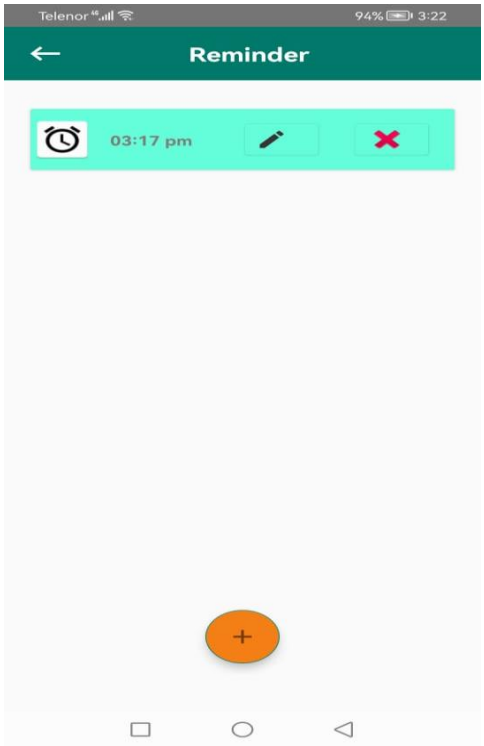

Figure 69: Set Reminder

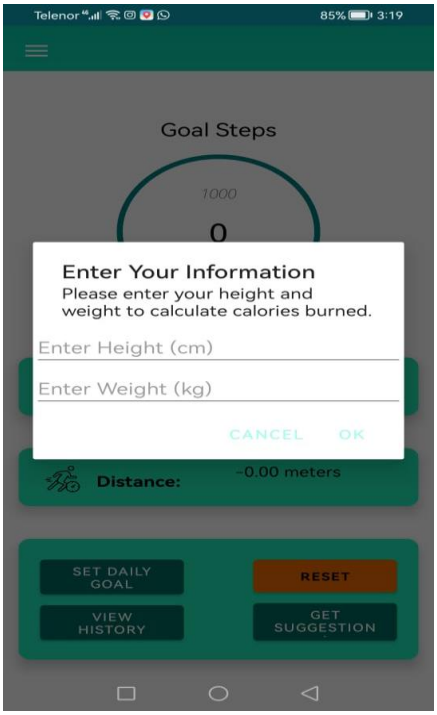

Figure 70: Input Dialogue

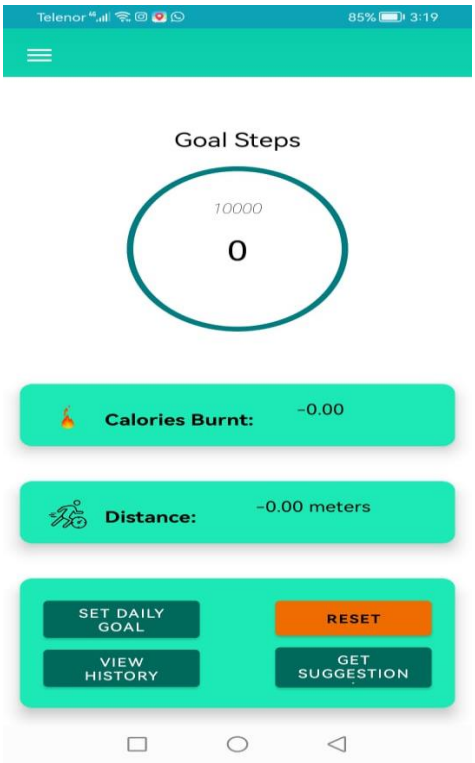

Figure 71: Step Counter

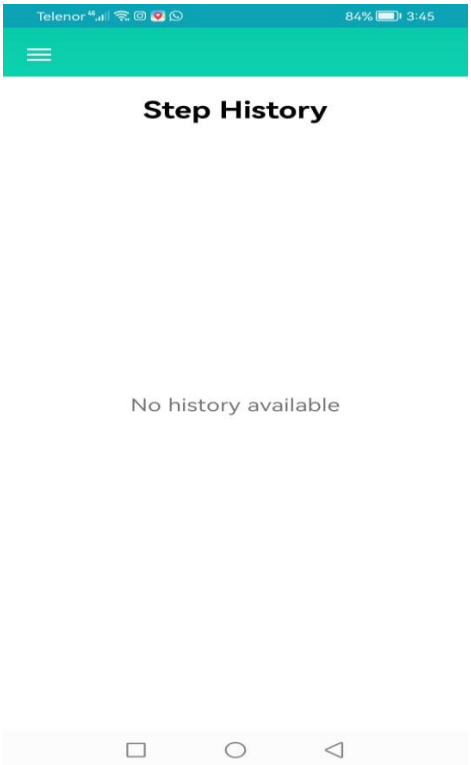

Figure 72: Steps History

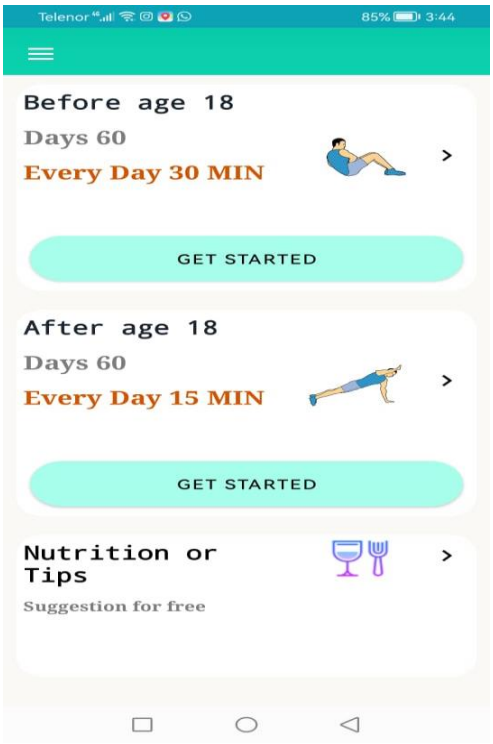

Figure 73: Exercises

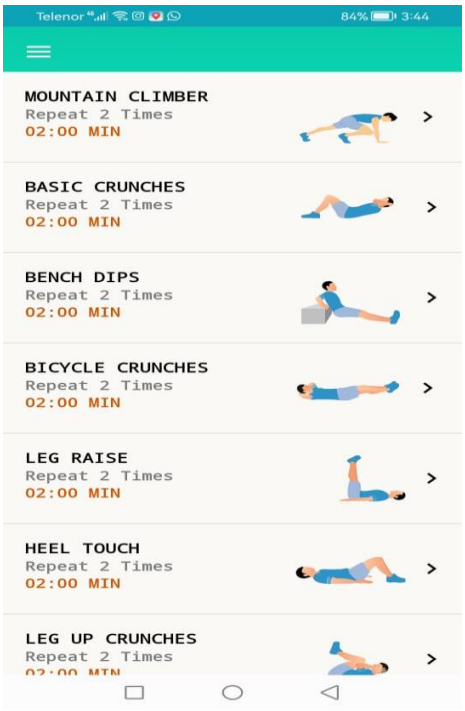

Figure 74: Exercises List

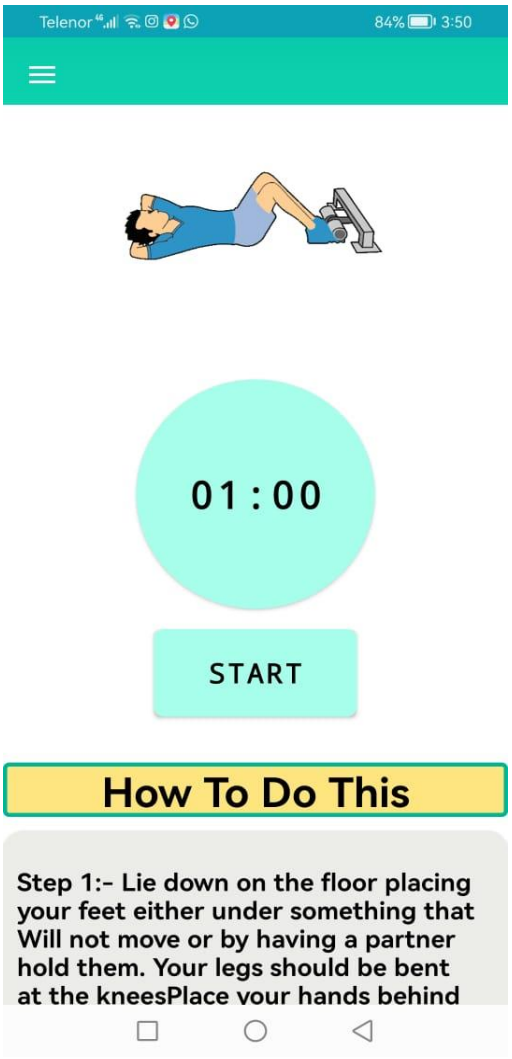

Figure 75: Selected Exercise Description

C. Logging On

A user can register first in the app that login with the same credentials.

8.2 System Menu

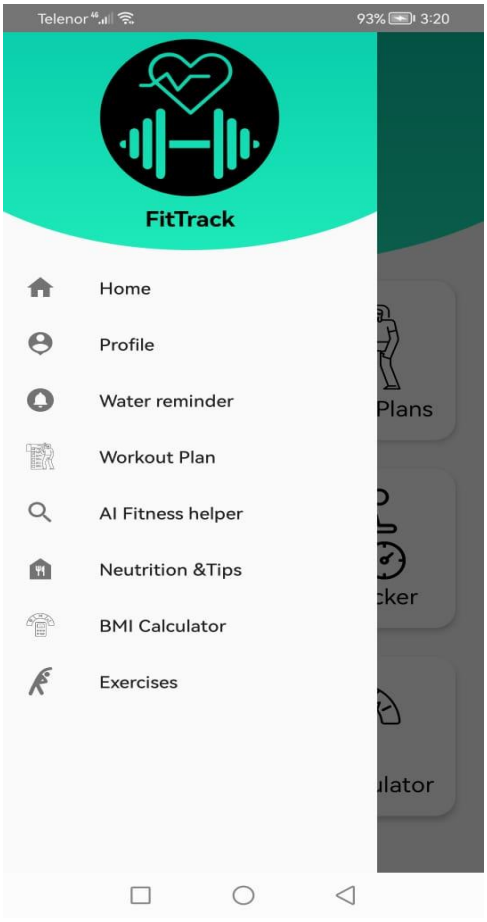

Figure 76: System Menu

Upon opening the app, the **Home Screen** presents the main menu. Each option is clearly labeled and provides access to different features:

- **Dashboard**: View Run Tracker, Step Counter, Exercises, BMI Calculator, Workout Plans and Navigation bar.
- **Exercise Module**: Select and perform exercises.
- **Run Tracker**: Track and log running sessions.
- **Water Reminder**: Schedule and track water reminders.
- **Nutrition Tracker**: Log meals and view summaries.
- **AI helper/chatbot**: For taking food related info.
- **Step Counter**: For performing steps and view calories burnt and distance covered.
- **Workout**: Creating workout for storing information
- **BMI Calculator**: For calculating body mass index by entering age, height, weight and selecting gender

8.3 Changing User ID and Password

1. Navigate to the **Navigation** menu.
2. Select **Profile Settings**.
3. Click on **Change User ID/Password**.
4. Enter the current credentials and provide the new information.
5. Confirm changes and click **Save**.

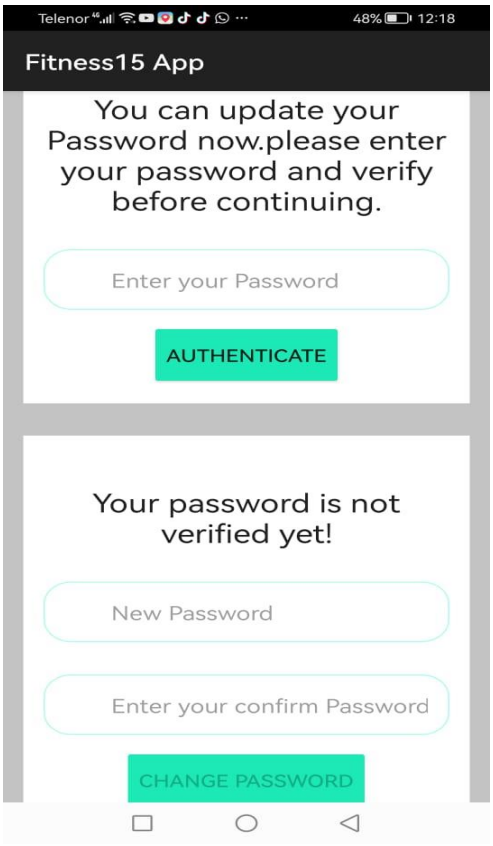

Figure 77: Change Password

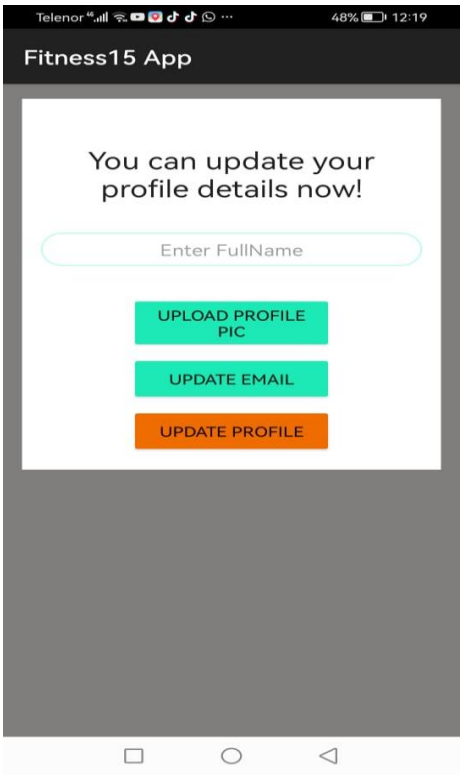

Figure 78: Update Profile

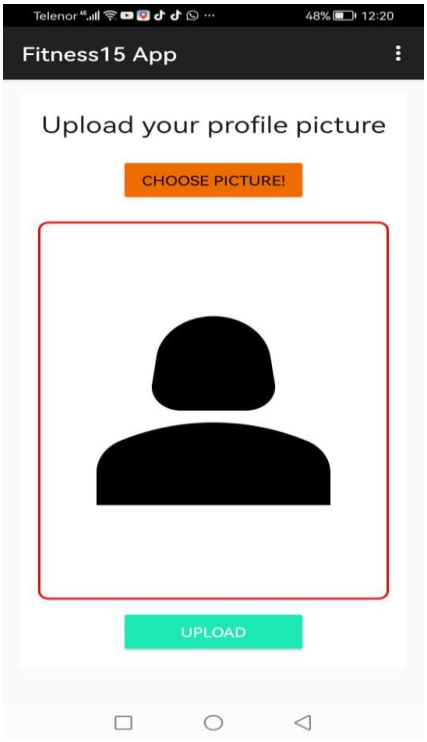

Figure 79: Upload Profile Picture

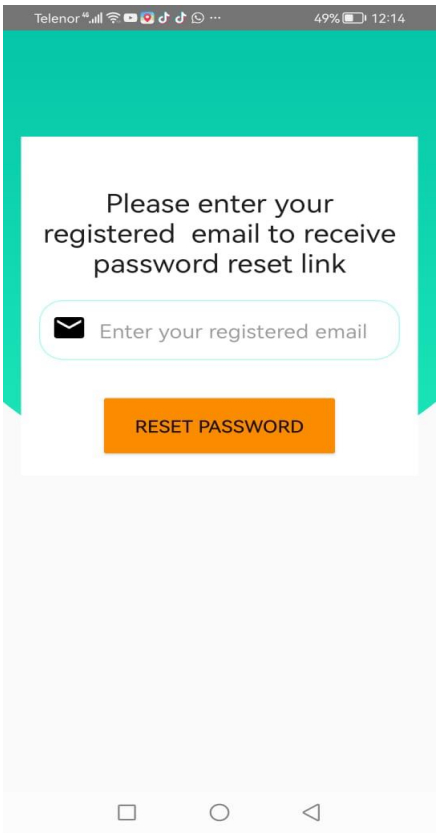

Figure 80: Reset Password

8.4 Exit System

1. Click the **Menu** button.
2. Select **Exit App**.
3. Confirm the action if prompted.

D. USING the SYSTEM

8.5.1 [SYSTEM FUNCTION NAME: USING STEP COUNTER]

1. Access the **Dashboard**.
2. Click on **Step Counter**.
3. Make sure **Physical Activity** Permission is enable.
4. Start walking, and the system will automatically count your steps.
5. Set or adjust your daily step goal under **Set Goal Button**.
6. View history by clicking on **History** from the menu.

8.5.2 [SYSTEM FUNCTION NAME: USING WATER REMINDER]

1. Access the **Dashboard**.
2. Click on **Menu bar**.
3. Click on **Water Reminder**.
4. Enter Personal Information.
5. Set or adjust your daily goal to drink water under **Settings**.
6. Click on **Reminder** and set alarms.
7. Once alarm and notification occur then click on **Drink** button.

8. Also can set **Cup Size** that you have drink.

8.5.3 [SYSTEM FUNCTION NAME: USING NUTRITION TRACKING]

1. Access the **Dashboard**.
2. Click on **Step Counter**.
3. Start walking, and the system will automatically count your steps.
4. Set or adjust your daily step goal under **Settings**.
5. View history by clicking on **History** from the menu.

8.5.4 [SYSTEM FUNCTION NAME: USING BMI CALCULATOR]

1. Access the **Dashboard**.
2. Click on **Step Counter**.
3. Start walking, and the system will automatically count your steps.
4. Set or adjust your daily step goal under **Settings**.
5. View history by clicking on **History** from the menu.

8.5.5 [SYSTEM FUNCTION NAME: USING WORKOUT PLANS]

1. Access the **Dashboard**.
2. Click on **Workout Plan**.
3. Click on **Add** button for creating new workout plan.
4. By Clicking on **Delete** button can also delete the plan.

8.5.6 [SYSTEM FUNCTION NAME: USING EXERCISES]

1. Access the **Dashboard**.
2. Click on **Exercises.**
3. Click on for Age 18 below or Age 18 above exercises.
4. Then select exercise which need to perform.
5. Click on **Start** button for time and can also **Pause.**

8.5.7 [SYSTEM FUNCTION NAME: USING RUN TRACKER]

1. Access the **Dashboard**.
2. Click on **Run Tracker**.
3. Make sure **Location** Permission is enable.
4. Then Click on **Start** button for calculating distance as user run.
5. By Clicking on **Stop** button can also pause.

8.5.8 [SYSTEM FUNCTION NAME: AI CHATBOT]

1. Access the **Dashboard**.
2. Click on **Menu**.
3. Select **AI Fitness Helper**.
4. Ask query by giving prompt.

8.6 SPECIAL INSTRUCTIONS FOR ERROR CORRECTION

1. **Incorrect Step Count**: If steps are inaccurately counted, ensure the device is secured to your body to minimize motion interference.
2. **Water Reminder Not Triggering**: Verify that notifications are enabled in both the app settings and device settings.
3. **Run Tracker GPS Issues**: Ensure location services are enabled and that the app has necessary permissions.
4. **App Crash**: Restart the application and clear cache if necessary. If the issue persists, reinstall the app.

8.7 CAVEATS AND EXCEPTIONS

- **Device Compatibility**: Some features may not function as intended on older devices or without the required sensors (e.g., accelerometer, GPS).
- **Battery Usage**: GPS tracking and notifications may consume more battery. Ensure the device is charged during use.

- **Data Accuracy**: Input data, such as weight and height for BMI calculation, must be precise to ensure accurate results.
- **Network Dependency**: Features like Run Tracker require stable internet or GPS signals for optimal performance.

Chapter 9: REFERENCES

[1] Ideal Expert. (2023). How to build yoga App [Video]. YouTube. https://youtu.be/ZxHdlIcF2Hc?si=IUyUCgRWN53O3hd6
[2] Psyfb2 (2019). [Android-Runner-Tracker]. https://github.com/psyfb2/Android-Runner-Tracker/tree/master/app/src/main/java/com/example/runnertracker
[3] Smith, J., & Jones, D. (2023). The impact of fitness tracking apps on exercise motivation. Journal of Health Psychology, 15(2), 123-135.
[4] Davis, M. (2022). The psychology of exercise. Oxford University Press.
[5] World Health Organization. (2021). Physical activity guidelines. WHO/NMH/NCD/16.2.
[6] Fitbit. (2023). About Fitbit. https://store.google.com/category/watches_trackers?hl=en-US
[7] Johnson, K. (2022). Designing for user engagement in fitness apps. UX Conference, San Francisco, CA.